



Bundesamt
für Sicherheit in der
Informationstechnik

Deutschland
Digital•Sicher•BSI•

BSI Technische Richtlinie 03125

Beweiswerterhaltung kryptographisch signierter Dokumente

Anlage TR-ESOR-F: Formate

Bezeichnung	Formate
Kürzel	BSI TR-ESOR-F (update 1: Freigabe von ASiC-AIP auf Basis von ETSI EN 319 162-1)
Version	1.3 (auf Basis der eIDAS-Verordnung und der ETSI Preservation Standards mit einem neuen Zertifizierungsschema)
Datum	27.08.2026



Änderungshistorie

<i>Version</i>	<i>Datum</i>	<i>Name</i>	<i>Beschreibung</i>
1.3	31.03.2022	BSI	TR-ESOR-F
1.3 update 1	27.08.2026	BSI	TR-ESOR-F+ASiC-AIP

Tabelle 1: Änderungshistorie

Inhalt

1	Einleitung	6
2	Übersicht.....	9
3	Definition der Archivinformationspakete.....	13
3.1	XAIP.....	13
3.1.1	Überblick über die XAIP-Datenstruktur – das <XAIP>-Element	14
3.1.2	Der xaip:packageHeaderType	15
3.1.3	Der xaip:metaDataSectionType.....	20
3.1.4	Der xaip:dataObjectsSectionType	23
3.1.5	Der xaip:credentialsSectionType	27
3.1.6	Das Delta-XAIP-Element <DXAIP>	29
3.2	Logisches XAIP (LXAIP).....	31
3.2.1	Verlinkung der Datenobjekte in einem LXAIP	31
3.2.2	Verlinkung der Metadatenobjekte in LXAIP	32
3.2.3	Verlinkung der Credentialobjekte in LXAIP.....	33
3.2.4	Das Delta-LXAIP-Element	33
3.2.5	Formaterkennung und -validierung eines LXAIP	33
3.2.6	LXAIP-Referenzen	34
3.3	ASiC-AIP	34
3.3.1	ASiC-AIP mit einem logischen XAIP	37
3.3.2	ASiC-AIP mit einem gewöhnlichen XAIP	38
3.3.3	Ein initiales ASiC-AIP.....	39
3.3.4	ASiC-AIP und Verlinkung der XML-Daten.....	40
3.3.5	ASiC-AIP - Umgang mit Versionen.....	40
3.3.6	Referenzen in ASiC-AIP	41
3.3.7	Das Delta-ASiC-AIP	42
3.3.8	Formaterkennung und -validierung eines ASiC-AIP	44
4	Nutzdatenformate	45
4.1	Metadaten.....	45
4.1.1	Extensible Markup Language (XML)	45
4.1.2	XML Schema (XSD)	46
4.2	Inhaltsdaten (Objektdaten).....	46
4.2.1	Dokumente (Schriftgut)	47
4.2.2	SVG – Scalable Vector Graphics	52
4.2.3	Multi-Media Formate.....	52
4.3	Base64 Kodierung.....	55
5	Kryptographische Datenformate	57

5.1	Signatur- und Siegelformate	57
5.1.1	PKCS#7 / CMS / CAdES	57
5.1.2	Digitale XML Signaturen / XAdES	58
5.1.3	Digitale PDF-Signaturen / PAdES	58
5.1.4	Digitale AdES-Signaturen im ASiC-Container	59
5.1.5	Sonstige Signatur- bzw. Siegelformate	59
5.2	Zertifikatsformate	59
5.3	Zertifikatsvalidierungsformate	60
5.3.1	Online Certificate Status Protocol (OCSP / RFC 2560 / RFC 6960)	60
5.3.2	Server-Based Certificate Validation Protocol (SCVP / RFC 5055)	60
5.4	Zeitstempel	61
5.5	Beweisdatenbericht (Evidence Record gemäß RFC 4998 / RFC 6283)	61
5.5.1	EvidenceRecord gemäß RFC 4998	62
5.5.2	<EvidenceRecord> gemäß RFC 6283	63
6	Anhang – Umgang mit digitalen Signaturtechniken innerhalb von (L)XAIP	64
6.1	Umgang mit CAdES	64
6.2	Umgang mit XAdES	65
6.3	Umgang mit PAdES	67
6.4	Umgang mit ASiC	67
6.5	Umgang mit dem Zeitstempel	67
6.6	Umgang mit dem Evidence Record	67
7	Anhang - Umgang mit signierten Metadaten	69
8	Anhang – XML-Schema-Definition	70

Abbildungen

Abbildung 1: Schematische Darstellung der IT-Referenzarchitektur für TR-S.4	7
Abbildung 2: Schematische Darstellung der IT-Referenzarchitektur mit TR-S.512	7
Abbildung 3: Definition von <code>asic:DataObjectReference</code> gem. ASiC-Manifest (vgl. [ETSI EN 319 162-1])	32
Abbildung 4: Aufbau des Elements <code>asic:AsiCManifest</code> (vgl. [ETSI EN 319 162-1])	35
Abbildung 5: ASiC-AIP-Manifest, Definition der globalen Kanonisierungsmethode	36
Abbildung 6: Aufbau des Elements <code>pres:ContainerID</code> (vgl. [ETSI TS 119 512])	36
Abbildung 7: Aufbau des Elements <code>ds:Transform</code>	37
Abbildung 8: Beispiel für ein ASiC-AIP mit einem LXAIP	37
Abbildung 9: Beispiel für ein ASiC-AIP-Manifest mit einem gewöhnlichen XAIP	38
Abbildung 10: Ein beispielhaftes, initiales ASiC-AIP mit einem LXAIP und einer technischen Signatur. ...	39
Abbildung 11: Beispiel für ein ASiC-AIP mit einem LXAIP mit einer Referenz auf das Element <code>xaip:versionManifest</code> in LXAIP	40
Abbildung 12: Ein Beispiel für ein LXAIP mit zwei Versionen und ein korrespondierenden ASiC-AIP	41
Abbildung 13: Exemplarische Darstellung eines Delta-ASiC-AIP mit einem Delta-LXAIP	43
Abbildung 14: Exemplarische Darstellung eines ASiC-AIP mit 2 Versionen und beiden Evidence Records	43
Abbildung 15: Schematische Darstellung einer enveloping XAdES	65
Abbildung 16: Schematische Darstellung einer enveloped XAdES	66
Abbildung 17: Möglichkeiten der Ablage der XML-Inhaltsdaten innerhalb vom (L)XAIP	66

Abbildung 18: Möglichkeiten der Ablage der XML-Signaturen innerhalb vom (L)XAIP.	66
---	----

Tabellen

Tabelle 1: Änderungshistorie	2
Tabelle 2: Wertaufzählung für die Attribute <code>category</code> und <code>classification</code> gem. [XFDU] und [OAS].	22

1 Einleitung

Ziel der Technischen Richtlinie „Beweiswerterhaltung kryptographisch signierter Dokumente“ ist die Spezifikation sicherheitstechnischer Anforderungen für den langfristigen Beweiswerterhalt von kryptographisch signierten elektronischen Dokumenten und Daten nebst zugehörigen elektronischen Verwaltungsdaten (Metadaten).

Eine für diese Zwecke definierte Middleware (TR-ESOR-Middleware) im Sinn dieser Richtlinie umfasst alle diejenigen Module (**M**) und Schnittstellen (**S**), die zur Sicherung und zum Erhalt der Authentizität und zum Nachweis der Integrität der aufbewahrten Dokumente und Daten eingesetzt werden.

Die im Hauptdokument dieser Technischen Richtlinie vorgestellte Referenzarchitektur besteht aus den nachfolgend beschriebenen Schnittstellen, Funktionen und logischen Einheiten:

- der TR-S.4- oder TS119512-Schnittstelle TR-S.512 in der Profilierung **[TR-ESOR-TRANS]** der TR-ESOR-Middleware, die dazu dient, die TR-ESOR-Middleware in die bestehende IT- und Infrastrukturlandschaft einzubetten;
- dem „ArchiSafe-Modul“ (vgl. **[TR-ESOR-M.1]**), welches den Informationsfluss in der Middleware regelt, die Sicherheitsanforderungen an die Schnittstellen zu den IT-Anwendungen umsetzt und für eine Entkopplung von Anwendungssystemen und ECM-/Langzeitspeicher sorgt;
- dem „Krypto-Modul“ (vgl. **[TR-ESOR-M.2]**) nebst den zugehörigen Schnittstellen TR-S.1 und TR-S.3, das alle erforderlichen Funktionen zur Berechnung von Hashwerten, zur Prüfung elektronischer Signaturen bzw. Siegel bzw. Zeitstempel, zur Nachprüfung elektronischer Zertifikate und zum Einholen qualifizierter Zeitstempel sowie (optional) elektronische Signaturen bzw. Siegel für die Middleware zur Verfügung stellt. Darüber hinaus kann es Funktionen zur Ver- und Entschlüsselung von Daten und Dokumenten zur Verfügung stellen;
- dem „ArchiSig-Modul“ (vgl. **[TR-ESOR-M.3]**) mit der Schnittstelle TR-S.6, das die erforderlichen Funktionen für die Beweiswerterhaltung der digital signierten Unterlagen bereitstellt;
- einem ECM-/Langzeitspeicher mit den Schnittstellen TR-S.2 und TR-S.5, der die physische Archivierung/Aufbewahrung und auch das Speichern der beweiswerterhaltenden Zusatzdaten übernimmt.

Dieser ECM-/Langzeitspeicher ist nicht mehr direkt Teil der Technischen Richtlinie, gleichwohl werden über die beiden Schnittstellen, die noch Teil der TR-ESOR-Middleware sind, Anforderungen daran gestellt.

Ebenso wenig ist die Applikationsschicht, die auch einen XML-Adapter enthalten kann, direkter Teil der Technischen Richtlinie, auch wenn dieser XML-Adapter als Teil einer Middleware implementiert werden kann.

Die empfohlene IT-Referenzarchitektur ist in

Abbildung 1 und Abbildung 2 dargestellt und besteht im Wesentlichen aus den in **[TR-ESOR]**, Kap. 7 grob beschriebenen logischen Komponenten und Schnittstellen. Diese werden in Anhängen zur TR weiter detailliert. Die Grafik zeigt zudem die externen Komponenten und Systeme an, die das Bild vervollständigen. Grundsätzlich wird als obere Schnittstelle der TR-ESOR-Middleware entweder die TR-S.4-Schnittstelle gemäß **[TR-ESOR-E]**, die in

Abbildung 1 dargestellt ist, oder die TR-S.512-Schnittstelle gemäß **[ETSI TS 119 512]** in der Profilierung **[TR-ESOR-TRANS]**, die in Abbildung 2 gezeigt wird, unterstützt.

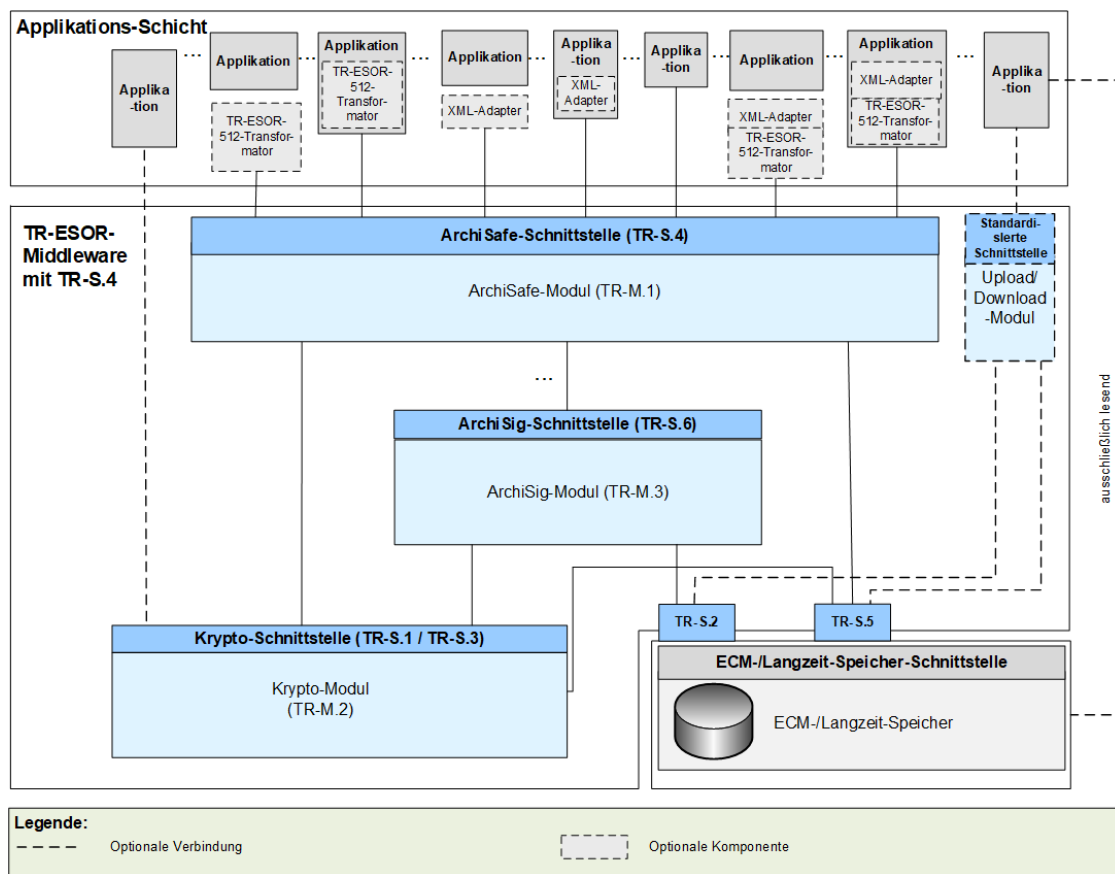


Abbildung 1: Schematische Darstellung der IT-Referenzarchitektur für TR-S.4

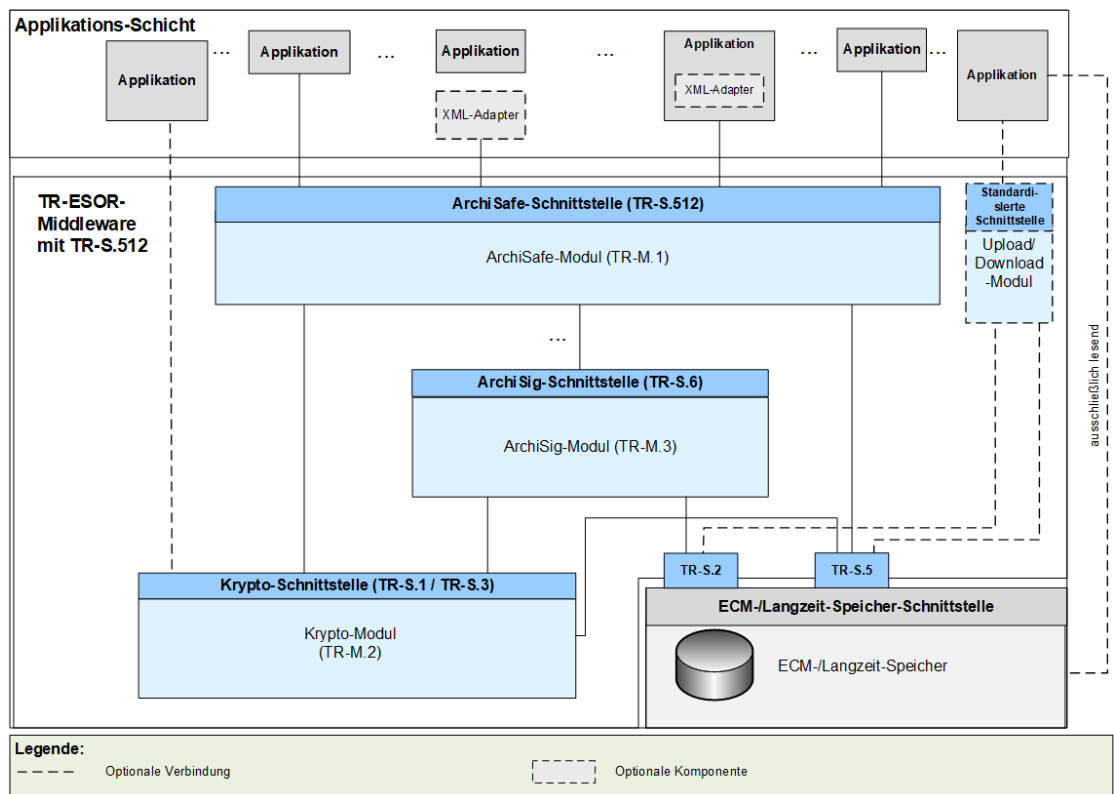


Abbildung 2: Schematische Darstellung der IT-Referenzarchitektur mit TR-S.512

Die in Abbildung 1 bzw. Abbildung 2 dargestellte IT-Referenzarchitektur orientiert sich an der ArchiSafe Referenzarchitektur und soll die logische (funktionale) Interoperabilität künftiger Produkte mit den Zielen und Anforderungen der Technischen Richtlinie ermöglichen und unterstützen.

Sofern der optionale XML-Adapter und/oder der optionale TR-ESOR-512-Transformator¹ vorhanden sind, können beide in folgenden Ausprägungen vorliegen:

- Jeweils eigenständige Komponente mit Schnittstellen zur Applikation sowie zum ArchiSafe-Modul
- Jeweils eigenständige Komponente, jedoch Teil der Applikation mit Schnittstelle zum ArchiSafe-Modul
- XML-Adapter und TR-ESOR-512-Transformator als eine gemeinsame Komponente, die beide Teile enthält mit Schnittstellen zur Applikation sowie zum ArchiSafe-Modul
- XML-Adapter und TR-ESOR-512-Transformator als eine gemeinsame Komponente, die beide Teile enthält und Teil der Applikation ist, mit Schnittstelle zum ArchiSafe-Modul.

Der "ETSI TS119512 TR-ESOR Transformator" ermöglicht Bewahrungsdiensten gemäß [eIDAS-VO], empfangene ETSI TS119512 (V1.1.2) Nachrichten² in TR-S.4 Nachrichten zu transformieren. Diese Nachrichten können dann an ein angeschlossenen TR-ESOR-System³ geschickt werden, ohne irgendwelche Änderungen dieses TR-ESOR-Systems.

Der Einsatz des TR-ESOR-512-Transformators wird EMPFOHLEN, sofern das TR-ESOR-Produkt mit einer TR-S.4-Schnittstelle in Europa zum Einsatz kommt und Interoperabilität mit europäischen (qualifizierten) Bewahrungsdiensten und Bewahrungsprodukten hergestellt werden soll.

Diese Technische Richtlinie ist modular aufgebaut und spezifiziert in einzelnen Anlagen zum Hauptdokument die funktionalen und sicherheitstechnischen Anforderungen an die erforderlichen IT-Komponenten und Schnittstellen der TR-ESOR-Middleware. Die Spezifikationen sind strikt plattform-, produkt-, und herstellerunabhängig.

Das vorliegende Dokument trägt die Bezeichnung „Anlage TR-ESOR-F“ und spezifiziert Datenformate, die aus Sicht der funktionalen und rechtlichen Anforderungen für den Beweiswerterhalt kryptographisch signierter Dokumente geeignet sind.

¹ Siehe [TR-ESOR-TRANS].

² In der Profilierung von [TR-ESOR-TRANS].

³ Siehe <https://www.bsi.bund.de/EN/tr-esor> oder <https://www.bsi.bund.de/DE/tr-esor>.

2 Übersicht

Das Ziel einer auf lange Zeiträume angelegten beweiswerterhaltenden Ablage elektronischer Daten ist die nachprüfbare authentische, d. h. zurechenbare und unversehrte Speicherung, Konservierung und Verfügbarkeit der elektronischen Daten und Metadaten zumindest für die Dauer gesetzlich vorgeschriebener Aufbewahrungspflichten. Zur Sicherung der Verfügbarkeit elektronischer Dokumente gehört dabei auch die Gewährleistung der Verkehrsfähigkeit, d. h. der Lesbarkeit und des Zusammenhangs der Daten und Metadaten mit den ihnen zugrundeliegenden Geschäftsvorfällen über lange Zeiträume auf den zum Zeitpunkt der Verfügbarmachung üblichen IT-Systemen.

Der Inhalt elektronischer Daten und Dokumente ist auf der Anwendungsebene von IT-Systemen in der Regel als ein Strom (Folge) von Zeichen eines endlichen Zeichensatzes Z_1 codiert. Auch die Metadaten sind als Folge eines Zeichensatzes Z_2 codiert. Die Zeichensätze Z_1 und Z_2 können, müssen aber nicht übereinstimmen. Darüber hinaus genügen die Metadaten bestimmten syntaktischen und semantischen Regeln, die in der Spezifikation der Metadatensätze begründet sind.

Die zugehörigen Metadaten lassen sich in zwei Kategorien unterscheiden:

- Ein Metadatensatz M_1 enthält Informationen über den zur Codierung der Unterlagen verwendeten Zeichensatz Z_1 und umfasst somit Informationen zur Darstellung und Formatierung der eigentlichen Inhaltsdaten.
- Ein Metadatensatz M_2 enthält zusätzliche beschreibende Metainformationen zu den digitalen Unterlagen (z. B. Ersteller, Datum, Aktenzeichen, usw.) und stellt somit beispielsweise sicher, dass die Dokumente oder Daten wiedergefunden und einem Geschäftsvorgang zugeordnet werden können.

Das Ziel einer beweiswerterhaltenden Ablage elektronischer Dokumente ist es daher, für die digitalen Inhalte sowie deren Metadaten und Zeichensätze

- die Authentizität (daraus folgt auch die Nichtabstreitbarkeit),
- die Integrität (Unversehrtheit) und die
- Verfügbarkeit

für lange Zeiträume, mindestens aber für die Dauer gesetzlicher Aufbewahrungsfristen zu gewährleisten. Voraussetzung dafür sind standardisierte, zuverlässige und nachprüfbar vertrauenswürdige Dateninfrastrukturen und Funktionsaufrufe für die aufzubewahrenden elektronischen Dokumente.

Um die langfristige Verfügbarkeit der aufbewahrten elektronischen Dokumente sicher zu stellen, sollen deshalb sowohl für die Inhaltsdaten als auch die Metadaten nach Möglichkeit ausschließlich⁴ offene, standardisierte und stabile Datenformate verwendet werden, die eine langfristige und weitgehend system- und plattformunabhängige Interpretierbarkeit der Daten unterstützen. Die vornehmliche Intention dieser Anforderung ist, eine Formattransformation der gespeicherten elektronischen Dokumente zumindest für die Dauer der gesetzlich vorgeschriebenen Aufbewahrungsfristen zu vermeiden, da dies – vor allem bei elektronisch signierten Dokumenten – mit nicht unerheblichem Aufwand verbunden ist.

Für den Nachweis der Integrität und Authentizität der aufzubewahrenden Daten sieht diese Richtlinie den Einsatz vertrauenswürdiger kryptographischer Sicherungsmittel vor, die sowohl die eigentlichen Inhaltsdaten als auch die „beschreibenden“ Metadaten umfassen, zuverlässig mit den geschützten Daten verknüpfen und imstande sind, den Beweiswert der kryptographischen Sicherungsmittel für die Dauer der gesetzlichen Aufbewahrungsfristen zu erhalten bzw. rechtskonform zu erneuern.

Zur Sicherstellung der Authentizität aufbewahrter elektronischer Daten und Dokumente gehört auch, dass die aufbewahrten elektronischen Unterlagen zuverlässig und dauerhaft mit sämtlichen Metainformationen verknüpft werden und bleiben, die für die Auffindbarkeit der Unterlagen sowie die rechts- und

⁴ Sofern die ursprünglichen Daten erst in ein standardisiertes und langfristig stabiles Format überführt werden müssen und die zur Verarbeitung der Dokumente notwendigen IT-Systeme voraussichtlich während der geplanten Aufbewahrungszeit verfügbar sein werden, ist es sinnvoll, zusätzlich die ursprünglichen Daten im Original-Format aufzubewahren.

revisionssichere Nachvollziehbarkeit (Rekonstruktion) der den Daten zugrundeliegenden Geschäftsvorfälle erforderlich sind.

Der Zweck dieser Spezifikation ist es, elektronische Datenformate und Transaktionsprotokolle zu definieren und zu beschreiben, die geeignet sind, die rechtlichen und funktionalen Anforderungen an eine beweiswerterhaltende Langzeitspeicherung im Sinne dieser Richtlinie adäquat abzubilden und umzusetzen.

Der Abschnitt 3 dieses Dokuments spezifiziert mit dem <XAIP>-Element ein XML-basiertes Containerformat (Archivinformationspaket) XAIP für abgelegte Datenobjekte gemäß Kapitel 3.1 bzw. LXAIP gemäß Kapitel 3.2 und ein ZIP- basiertes Containerformat ASiC-AIP gemäß Kapitel 3.3, das von TR-konformen Middleware-Komponenten erzeugt oder von der Applikationsschicht übergeben wird, sowie mit dem Delta-XAIP-Element <DXAIP>-Element gemäß Kapitel 3.1.6 bzw. Delta-LXAIP-Element gemäß Kapitel 3.2.4 bzw. Delta-ASiC-AIP gemäß Kap. 3.3.7 eine beim ArchiveUpdateRequest (vgl. [TR-ESOR-E]) übergebene Delta-(L)XAIP-/ASiC-AIP-Struktur.

Der Abschnitt 4 dieses Dokuments beschreibt hierauf folgend elektronische Datenformate für die Inhaltsdaten (Primärinformationen) und die Metadaten, die zum Zeitpunkt der Veröffentlichung dieser Technischen Richtlinie für die langfristige Aufbewahrung von Nutzdaten vornehmlich unter dem Gesichtspunkt der nachhaltigen Verfügbarkeit sowie maschinellen Lesbarkeit und Interpretierbarkeit empfohlen werden.

Der Abschnitt 5 dieses Dokuments beschreibt Strukturen, Formate und Algorithmen für die Erzeugung und Interpretation kryptographischer Daten, die zum Zeitpunkt der Veröffentlichung dieser Technischen Richtlinie für die langfristige Sicherung der Integrität, Authentizität und Beweiskraft elektronischer Dokumente geeignet sind.

Der diesem Dokument beigelegte Anhang (siehe Abschnitt 8) enthält schließlich das vollständige spezifizierte XML-Schema.

HINWEIS 1

*In der vorliegenden TR-ESOR-Version 1.3 werden die drei Begriffe „(beweiswerterhaltende) **Langzeitspeicherung**“, „(beweiswerterhaltende) **Bewahrung**“ und „(beweiswerterhaltende) **Archivierung**“ synonym verwendet.*

*Ebenso werden die drei Begriffe „**Archivinformationspaket (AIP)**“, „**Archivinformationscontainer**“ und „**Archivdatenobjekt**“ sowie die Begriffe „aufbewahren“ und „archivieren“ synonym verwendet.*

HINWEIS 2

TR-ESOR spezifiziert ein Bewahrungsprodukt (engl. Preservation Product) gemäß [ETSI SR 019 510], [ETSI TS 119 511] und [ETSI TS 119 512] und [eIDAS-VO].

*Die TR 03125 TR-ESOR ist in [ETSI SR 019 510] in dem Kapitel 4.7.3 und 5.2 und B3.2 beschrieben. Die in TR-ESOR erforderlichen grundlegenden Bewahrungstechniken, z.B. das Bewahrungsprotokoll, das Beweisdaten-Format Evidence Record, die Archivdaten-Format (L)XAIP und ASiC-AIP sind in der **ETSI-Publikation [ETSI TS 119 512]** als **normative Elemente** enthalten.*

HINWEIS 3

*Die obere TR-ESOR-Eingangsschnittstelle **TR-S.4** oder die TS119512-Eingangsschnittstelle **TR-S.512** gemäß der „Preservation-API“ in [ETSI TS 119 512] in der Profilierung von [TR-ESOR-TRANS], die logisch äquivalent zur Eingangsschnittstelle TR-S.4 gemäß [TR-ESOR-E] ist wie in der Tabelle 2 in [TR-ESOR-E], Kapitel 4.1 dargestellt, muss benutzt werden. Eine andere Eingangs-Schnittstelle anstelle von TR-S.4 bzw. TR-S.512 ist nicht erlaubt.*

HINWEIS 4

In der vorliegenden TR-ESOR-Version 1.3 umfasst der Begriff „Archivinformationscontainer“ (AIP) in allen TR-ESOR-Anhängen:

- a) das Archivdatenobjekt „XAIP“ gemäß [TR-ESOR-F], Kapitel 3.1 als auch
- b) das logische XAIP „LXAIP“ gemäß [TR-ESOR-F], Kapitel 3.2 und
- c) das „ASiC-AIP“ gemäß [TR-ESOR-F], Kapitel 3.3 auf Basis von [ETSI EN 319162-1].

In TR-ESOR Version V1.3 wird zwischen XAIP, LXAIP und ASiC-AIP differenziert.

Mit (L)XAIP wird XAIP oder LXAIP bezeichnet.

HINWEIS 5

In dieser TR-ESOR Version 1.3 ist „BIN“ beschränkt auf die folgenden Bewahrungsobjekt-Formate (engl. *preservation object formats*):

- CAdES gemäß [ETSI TS 119 512], Annex A.1.1 (<http://uri.etsi.org/ades/CAdES>). Sofern kein MIME Type gesetzt ist, wird als Default application/cms verwendet;
- XAdES gemäß [ETSI TS 119 512], Annex A.1.2 (<http://uri.etsi.org/ades/XAdES>). Sofern kein MIME Type gesetzt ist, wird als Default application/xml verwendet;
- PAdES gemäß [ETSI TS 119 512], Annex A.1.3 (<http://uri.etsi.org/ades/PAdES>). Sofern kein MIME Type gesetzt ist, wird als Default application/pdf verwendet
- ASiC-E gemäß [ETSI TS 119 512], Annex A.1.4 (<http://uri.etsi.org/ades/ASiC/type/ASiC-E>). Sofern kein MIME Type gesetzt ist, wird als Default [application/vnd.etsi.asic-e+zip](http://uri.etsi.org/ades/ASiC/type/ASiC-E) verwendet;
- ASiC-S gemäß [ETSI EN 319 162] (<http://uri.etsi.org/ades/ASiC/type/ASiC-S>). Sofern kein MIME Type gesetzt ist, wird als Default application/vnd.etsi.asic-s+zip verwendet.
- DigestList gemäß [ETSI TS 119 512], Annex A.1.6 (<http://uri.etsi.org/19512/format/DigestList>). Sofern kein MIME Type gesetzt ist, wird als Default application/xml verwendet;
- ASiC-ERS (in TR-ESOR v1.3 mit ASiC-AIP bezeichnet) gemäß [TR-ESOR-F], Kap. 3.3 und gemäß [ETSI TS 119 512], Annex A.3.1 (<http://uri.etsi.org/ades/ASiC/type/ASiC-ERS>) und

Im Falle Upload/Download-Funktion ist zusätzlich nachfolgendes Format erlaubt:

- Binärdaten (BIN) als „Octet Stream“, die ausschließlich in den ECM-/Langzeitspeicher mit „Upload-Request“ gespeichert werden, – aber nur sofern:
 - a) verbunden mit einem korrespondierenden LXAIP und dort referenziert gem. [TR-ESOR-F], Kap. 3.2,
 - b) ggf. mit „Download-Request“ ausgelesen werden, – verbunden mit einem korrespondierenden LXAIP, das mit der „ArchiveRetrieval“-Funktion ausgelesen wurde, – oder eingebettet in einem XAIP und ausgelesen mit der „ArchivRetrievalRequest“-Funktion.
 - c) Der Upload von XAIP oder LXAIP oder ASiC-AIP ist nicht zugelassen.

HINWEIS 6

Im folgenden Text umfasst der Begriff „Digitale Signatur“:

- „fortgeschrittene elektronische Signaturen“ gemäß [eIDAS-VO], Artikel 3 Nr. 11,
- „qualifizierte elektronische Signaturen“ gemäß [eIDAS-VO], Artikel 3 Nr. 12,
- „fortgeschrittenen elektronische Siegel“ gemäß [eIDAS-VO], Artikel 3 Nr. 26 und
- „qualifizierte elektronische Siegel“ gemäß [eIDAS-VO], Artikel 3 Nr. 27.

Insofern umfasst der Begriff „digital signierte Dokumente“ sowohl solche, die fortgeschrittene elektronische Signaturen oder Siegel bzw. qualifizierte elektronische Signaturen oder Siegel tragen.

Mit dem Begriff der „**kryptographisch signierten Dokumente**“ sind in dieser TR neben:

- den gemäß [eIDAS-VO], Artikel 3 Nr. 12 qualifiziert signierten,
- den gemäß [eIDAS-VO], Artikel 3 Nr. 27 qualifiziert gesiegelten oder
- den gemäß [eIDAS-VO], Artikel 3 Nr. 34 qualifiziert zeitgestempelten Dokumenten (im Sinne der eIDAS-

Verordnung)

auch Dokumente:

- *mit einer fortgeschrittenen Signatur gemäß [eIDAS-VO], Artikel 3 Nr. 11 oder*
- *mit einem fortgeschrittenen Siegel gemäß [eIDAS-VO], Artikel 3 Nr. 26 oder*
- *mit einem elektronischen Zeitstempel gemäß [eIDAS-VO], Artikel 3 Nr. 33*

erfasst, wie sie oft in der internen Kommunikation von Behörden entstehen.

Nicht gemeint sind hier Dokumente mit einfachen Signaturen oder Siegeln basierend auf anderen (z. B. nicht-kryptographischen) Verfahren.

3 Definition der Archivinformationspakete

Das vorliegende Dokument spezifiziert drei Archivinformationspaket-Formate: Abschnitt 3.1 spezifiziert das bereits in vorherigen TR-Versionen enthaltene, XML-basierte Archivinformationspaket-Format XAIP. Abschnitt 3.2 spezifiziert mit dem „logischen XAIP“ (LXAIP) eine Variante des XAIP, bei dem auf extern abgelegte Datenobjekte verwiesen werden kann, und Abschnitt 3.3 spezifiziert schließlich ein auf dem in der Europäischen Norm [ETSI EN 319 162-1] definierten ASiC-E-Format basierendes Archivinformationspaket-Format (ASiC-AIP).

In der vorliegenden Richtlinie wird der Begriff AIP⁵ im Sinne von XAIP, LXAIP oder ASiC-AIP verwendet.

(A3–1) Für den Beweiserhalt kryptographisch signierter Dokumente muss das in diesem Abschnitt beschriebene und näher spezifizierte XAIP-Format gemäß Kapitel 3.1 verwendet werden. Zusätzlich kann das LXAIP-Format gemäß Kapitel 3.2 oder das ASiC-AIP-Format gemäß Kapitel 3.3 genutzt werden.

(A3–2) Ergänzend zu Anforderung (A3-1) müssen bei einer Abfrage oder einem Export der aufzubewahrenden Daten und Dokumente diese in einem abgeschlossenen und selbsterklärenden XML-Daten-Austauschformat (L)XAIP bzw. in einem abgeschlossenen und selbsterklärenden ZIP-Daten-Austauschformat ASiC-AIP gemäß [TR-ESOR-F] zurück geliefert werden – unabhängig vom tatsächlich verwendeten Speicherformat im ECM-/Langzeitspeicher.

HINWEIS 7

Es darf eine Transformation zwischen dem eingereichten Format des Archivdatenobjekts beim ArchiveSubmissionRequest und dem angeforderten Format beim ArchiveRetrievalRequest durchgeführt werden. Auf dieser Weise kann beispielsweise ein zuvor abgelegten LXAIP (ArchiveSubmission) als XAIP angefordert und abgerufen (ArchiveRetrieval) werden.

3.1 XAIP

Ein Archivdatenobjekt, d. h. ein für die langfristige Ablage in einem elektronischen Bewahrungs- bzw. Archivsystem bestimmtes elektronisches Dokument im Sinne dieser Richtlinie, ist ein selbst-beschreibendes und wohlgeformtes XML-Dokument, das gegen ein gültiges und autorisiertes XML-Schema geprüft werden kann (im Weiteren auch: **XML formatted Archival Information Package** oder kurz: **XAIP**).

Ein solches Archivinformationspaket enthält sämtliche Inhaltsdaten (Primärinformationen) und Meta-informationen, die für eine zuverlässige und vollständige Rekonstruktion von Geschäfts- oder Verwaltungsvorgängen bis zum Ablauf der gesetzlich vorgeschriebenen Aufbewahrungsfristen erforderlich sind.

Die Beschreibung der Archivinformationspakete durch ein gültiges XML-Schema gewährleistet, dass:

- die Archivinformationspakete vor der Übergabe an den elektronischen Langzeitspeicher auf syntaktische Richtigkeit evaluiert werden können,
- erforderliche Ergänzungen oder Erweiterungen der Metadaten mit wenig Aufwand durch Ergänzung und Erweiterung vorhandener Metadatenstrukturen und/oder Einschluss zusätzlicher XML-Schemata möglich sind, sowie
- die für den Nachweis der Authentizität und Integrität aufbewahrungspflichtiger Daten aus rechtlichen Erfordernissen benötigten kryptographischen Sicherungsmittel, wie digitale Signaturen oder elektronische Zeitstempel, dauerhaft und zuverlässig mit den zu sichernden Daten verknüpft werden können.

Die folgenden Abschnitte definieren und beschreiben grundsätzliche syntaktische und semantische Strukturen, die ein zu den Zielen und Anforderungen dieser Technischen Richtlinie konformes

⁵ Archivinformationspaket

Archivinformationspaket implementieren soll. Sie stützen sich in weiten Teilen auf Vereinbarungen und Erfahrungen des ArchiSafe Projektes der Physikalisch-Technischen Bundesanstalt [PTB 05] sowie auf Konzepten des OAIS Referenzmodells [OAIS], des Metadata Encoding and Transmission Standards [METS], der Victorian Electronic Records Strategy [VERS] sowie dem XML Formatted Data Unit [XFDU] Standards des Consultative Committee for Space Data Systems (CCSDS) der amerikanischen Luft- und Raumfahrtbehörde.

In der Definition und Beschreibung der XML Datenstrukturen unterscheidet diese Spezifikation zwischen verbindlichen (obligatorischen) und optionalen Datenelementen. Ein verbindliches Datenelement muss in einem zu dieser Spezifikation konformen Archivinformationspaket vorhanden sein und durch zu dieser Richtlinie konforme elektronische Bewahrungs- und Archivsysteme interpretierbar sein. Ein optionales Element kann auftreten. Es muss nicht notwendig interpretiert werden können, darf jedoch die maschinelle Verarbeitung (Interpretation) anderer Elemente nicht behindern. Ein optionales Element muss auftreten, wenn sein Vorhandensein an bestimmte Bedingungen, wie bspw. das Vorhandensein elektronischer Signaturen, geknüpft ist und diese Bedingungs Voraussetzungen erfüllt sind.

3.1.1 Überblick über die XAIP-Datenstruktur – das <XAIP>-Element

Ein Archivinformationspaket gemäß dieser Richtlinie besteht aus einem <XAIP>-Element, das folgendermaßen definiert ist:

```
<xs:element name="XAIP" type="xaip:XAIPType"/>
```

Der **xaip:XAIPType** ist folgendermaßen definiert:

```
<xs:complexType name="XAIPType">
  <xs:sequence>
    <xs:element name="packageHeader" type="xaip:packageHeaderType"/>
    <xs:element name="metaDataSection" type="xaip:metaDataSectionType"
      minOccurs="0"/>
    <xs:element name="dataObjectsSection" type="xaip:dataObjectsSectionType"
      minOccurs="0"/>
    <xs:element name="credentialsSection" type="xaip:credentialsSectionType"
      minOccurs="0"/>
  </xs:sequence>
  <xs:attribute name="XAIPVersion" use="required" fixed="1.3.0"/>
</xs:complexType>
```

Der **xaip:XAIPType** enthält folgende Elemente und Attribute:

<packageHeader> [required]

Das <packageHeader>-Element enthält generelle Informationen zum Archivinformationspaket und beschreibt beispielsweise die logische Struktur desselben. Weitere Informationen zum **xaip:packageHeaderType** finden sich in Abschnitt 3.1.2.

<metaDataSection> [optional]

Das <metaDataSection>-Element enthält Metainformationen zur Beschreibung des Geschäfts- und Archivierungskontextes, sofern solche vorhanden sind. Die `metaDataSection` soll alle Informationen enthalten, die zur transparenten und nachhaltigen Interpretation des Geschäfts- und Archivierungskontextes benötigt werden. Weitere Informationen zum **xaip:metaDataSectionType** finden sich in Abschnitt 3.1.3.

<dataObjectsSection> [optional]

Das <dataObjectsSection>-Element enthält die Nutzdaten des Archivinformationspakets, sofern solche vorhanden sind. Dieses Element kann beispielsweise dafür genutzt werden, Inhaltsdaten in verschiedenen plattform- oder anwendungsspezifischen Datenformaten in einem XAIP-Container zu speichern oder ganze Akten mit vielen unterschiedlichen Dokumenten gemeinsam zu archivieren. Weitere Informationen zum **xaip:dataObjectsSectionType** finden sich in Abschnitt 3.1.4.

<credentialsSection> [optional]

Das <credentialsSection>-Element enthält bei Bedarf Beweisdaten in Form von Evidence Records oder beweisrelevante Informationen, wie z. B. Signaturen, Siegel, Zeitstempel, Zertifikate oder Signaturprüfinformationen. Weitere Informationen zum **xaip:credentialsSectionType** finden sich in Abschnitt 3.1.5.

@XAIPVersion [required, fixed]

Mit dem XAIPVersion-Attribut wird die festgelegte Version des XAIP manifestiert.

3.1.2 Der xaip:packageHeaderType

Der **xaip:packageHeaderType** wird für die Definition des XAIP-Elementes genutzt und ist folgendermaßen definiert:

```
<xs:complexType name="packageHeaderType">
  <xs:sequence>
    <xs:element name="AOID" type="xs:string" minOccurs="0"/>
    <xs:element name="packageInfo" type="xs:string" minOccurs="0"/>
    <xs:element name="versionManifest" type="xaip:versionManifestType"
      minOccurs="0" maxOccurs="unbounded"/>
    <xs:element ref="ds:CanonicalizationMethod" minOccurs="0"/>
    <xs:element name="extension" type="xaip:extensionType" minOccurs="0"/>
  </xs:sequence>
  <xs:attribute name="packageID" type="xs:ID" use="required"/>
</xs:complexType>
```

<AOID> [optional⁶]

Das <AOID>-Element dient als eindeutiger Identifikator des Archivinformationspakets, das in der Regel durch die TR-ESOR Middleware oder den ECM-/Langzeitspeicher erzeugt wird. Die interne Struktur dieses Identifikators ist nicht durch diese Spezifikation festgelegt, sondern bleibt dem Hersteller bzw. Anwender der TR-ESOR-Middleware überlassen.

<packageInfo> [optional]

Das <packageInfo>-Element kann Basisinformationen über das Archivinformationspaket im Textformat enthalten, die einen Nutzer auch in Zukunft in die Lage versetzen, das Format des XAIP-Dokumentes zu verstehen und die Inhalte zu interpretieren.

<versionManifest> [required, unbounded]

Das <versionManifest>-Element dient als "versionsspezifisches Inhaltsverzeichnis" und kann mehrfach auftreten. Es enthält jeweils Informationen zu einer Version des Archivinformationspakets. Die Struktur des **xaip:VersionManifestType** ist unten näher erläutert.

<ds:CanonicalizationMethod> [optional]

Das <CanonicalizationMethod>-Element spezifiziert die anzuwendende Kanonisierungsmethode. Sofern dieses Element fehlt, erfolgt die Kanonisierung durch die TR-ESOR-Middleware mit dem Standard-Algorithmus (C14N - Canonical XML [C14N]).

<extension> [optional]

Durch das optionale <extension>-Element können benutzerspezifische Erweiterungen geschaffen werden. Die Struktur des **xaip:extensionType** ist unten dargestellt.

⁶ Das <AOID>-Element ist optional, um im Rahmen des ArchiveSubmissionRequest (siehe Anlage E) die Erzeugung der AOID durch die aufgerufene Komponente zu ermöglichen. Sofern dieses Element nicht bereits vorhanden ist, muss es durch die TR-ESOR-Middleware oder den ECM-/Langzeitspeicher erzeugt werden.

Es wird empfohlen, diese Erweiterungen und hierdurch entstehende XAIP-Profile mit dem Bundesamt für Sicherheit in der Informationstechnik abzustimmen⁷.

@packageID [required]

Mit dem packageID-Attribut steht ein eindeutiger Identifikator des <packageHeader>-Elementes bereit, der bei Bedarf als Bezugspunkt innerhalb des Archivinformationspaket dienen kann. Sofern das <packageHeader>-Element kryptographisch geschützt werden soll, wird durch ein <protectedObjectPointer>-Element auf dieses packageID-Attribut verwiesen. Das packageID-Attribut kann von der Geschäftsanwendung erzeugt werden und in dieser als weiterer Identifikator dienen.

Die Struktur des **xaip:extensionType** ist folgendermaßen definiert:

```
<xs:complexType name="extensionType">
  <xs:sequence>
    <xs:any processContents="lax" minOccurs="0" maxOccurs="unbounded"/>
  </xs:sequence>
</xs:complexType>
```

Die Struktur des **xaip:VersionManifestType** ist folgendermaßen gegeben:

```
<xs:complexType name="versionManifestType">
  <xs:sequence>
    <xs:element name="versionInfo" type="xs:string" minOccurs="0"/>
    <xs:element name="preservationInfo" type="xaip:preservationInfoType"/>
    <xs:element name="submissionInfo" type="xaip:submissionInfoType"
      minOccurs="0"/>
    <xs:element name="packageInfoUnit" type="xaip:packageInfoUnitType"
      maxOccurs="unbounded"/>
    <xs:element name="idAssignmentList" type="xaip:idAssignmentListType"
      minOccurs="0"/>
    <xs:element name="extension" type="xaip:extensionType" minOccurs="0"/>
  </xs:sequence>
  <xs:attribute name="VersionID" type="xs:ID" use="required"/>
</xs:complexType>
```

<versionInfo> [optional]

Das <versionInfo>-Element enthält Informationen zur entsprechenden Version des Archivinformationspakets im Textformat.

<preservationInfo> [required]

Das <preservationInfo>-Element enthält Informationen zur Aufbewahrung (z. B. Aufbewahrungsdauer, Bewertung des Archivs) des Archivinformationspakets. Die Struktur des **xaip:preservationInfoType** ist unten näher erläutert⁸.

<submissionInfo> [optional]

Das <submissionInfo>-Element enthält Informationen über den Absender des Archivinformationspakets. Die Struktur des **xaip:submissionInfoType** ist unten näher erläutert.

<packageInfoUnit> [required, unbounded]

Das <packageInfoUnit>-Element enthält Informationen zu einer bestimmten Inhaltsdateneinheit. Dieses Element kann mehrmals auftreten. Die Struktur des **xaip:packageInfoUnitType** ist unten näher erläutert.

⁷ Darüber hinaus wird den Bundesbehörden empfohlen, diese Erweiterungen und/oder XAIP-Profile zusätzlich mit dem Bundesarchiv abzustimmen.

⁸ Stellen des Bundes wird empfohlen, sich bei der Aufbewahrung anbieterpflichtiger Unterlagen mit dem Bundesarchiv abzustimmen. Erfolgt die Paketierung der Datenobjekte für die Aufbewahrung koordiniert, kann eine gesonderte Datenaufbereitung im Rahmen der Aussonderung entfallen. So lassen sich Aufwände minimieren.

<idAssignmentList> [optional]

Das **<idAssignmentList>**-Element enthält eine Liste von **<idAssignmentPointer>**-Elementen, die jeweils einem **dataObjectID**-Attribut bzw. **metaDataID**-Attribut eine Prüfsumme zuweisen, falls entsprechend das **<binaryData>**-Element oder **<binaryMeta-data>**-Element verwendet wird. In den beiden Fällen werden gem. der Ausführungen zu **<protectedObjectPointer>**-Element auf Seite 19 die angesprochenen IDs nicht geschützt⁹. Mit Hilfe dieser Struktur kann die vorhandene Zuordnung ID – Objekt abgesichert werden. Die Struktur des **xaip:idAssignmentListType** ist auf Seite 20 erläutert.

<extension> [optional]

Durch das optionale **<extension>**-Element können benutzerspezifische Erweiterungen geschaffen werden. Die Struktur des **xaip:extensionType** ist auf Seite 16 dargestellt.

Es wird empfohlen, diese Erweiterungen und hierdurch entstehende XAIP-Profile mit dem Bundesamt für Sicherheit in der Informationstechnik abzustimmen⁷.

@VersionID [required]

Mit dem **VersionID**-Attribut steht ein eindeutiger Identifikator der Version des Archivinformationspakets bereit. Das **VersionID**-Attribut soll in der Form “v1”, “v2”, “v3” etc. gebildet werden.

Die Struktur des **xaip:preservationInfoType** im **xaip:VersionManifestType** ist folgendermaßen gegeben:

```
<xs:complexType name="preservationInfoType">
  <xs:sequence>
    <xs:element name="retentionPeriod" type="xs:date"/>
    <xs:element name="status" type="xs:string" minOccurs="0"/>
    <xs:element name="extension" type="xaip:extensionType" minOccurs="0"/>
  </xs:sequence>
</xs:complexType>
```

<retentionPeriod> [required]

Das **<retentionPeriod>**-Element spezifiziert das Datum bis zu dem die Unterlagen aufbewahrt werden müssen. Dieses Element wird beim **ArchiveDeletionRequest** (vgl. [TR-ESOR-E]) ausgewertet.

<status> [optional]

Das **<status>**-Element kann Informationen über den Status des Archivinformationspakets enthalten, die vor dem Löschen des Archivinformationspakets ausgewertet werden können.¹⁰

Die konkrete Belegung und Auswertung dieses Elementes ist nicht Gegenstand der vorliegenden Spezifikation. Vielmehr sollen derartige Festlegungen Gegenstand von XAIP-Profilen sein. Es wird empfohlen, solche XAIP-Profile mit dem Bundesamt für Sicherheit in der Informationstechnik abzustimmen⁷.

⁹ Da binäre Daten prinzipiell ohne umschließende XML-Elemente verhasht werden, ist die Zuordnung von zugehörigen ID zu den Daten nicht abgesichert. So ist es in speziellen Fällen möglich, dass die Inhalte von zwei Datenobjekten (z. B. zwei signierten PDF-Dokumenten) vertauscht werden und die nachgeordnete Prüfung eines hierzu vorhandenen EvidenceRecords NICHT fehlschlägt. Die ggf. auch vorhandenen Metadaten würden in dem Fall immer noch korrekte IDs aber falsche Inhaltsdaten referenzieren.

¹⁰ Mit diesem Element kann insbesondere der im behördlichen Umfeld benötigte “Bewertungsvermerk” realisiert werden. Stellen des Bundes wird empfohlen, sich bei der Aufbewahrung anbieterpflichtiger Unterlagen mit dem Bundesarchiv abzustimmen. Erfolgt die Paketierung der Datenobjekte für die Aufbewahrung koordiniert, kann eine gesonderte Datenaufbereitung im Rahmen der Aussonderung entfallen. So lassen sich Aufwände minimieren.“

<extension> [optional]

Durch das optionale **<extension>**-Element können benutzerspezifische Erweiterungen geschaffen werden. Die Struktur des **xaip:extensionType** ist auf Seite 16 dargestellt.

Es wird empfohlen, diese Erweiterungen und hierdurch entstehende XAIP-Profile mit dem Bundesamt für Sicherheit in der Informationstechnik abzustimmen⁷.

Die Struktur des **xaip:submissionInfoType** im **xaip:VersionManifestType** ist folgendermaßen gegeben:

```
<xs:complexType name="submissionInfoType">
  <xs:sequence>
    <xs:element name="clientID" type="saml:NameIDType"/>
    <xs:element name="submissionUnit" type="saml:NameIDType" minOccurs="0"/>
    <xs:element name="submissionAuthor" type="saml:NameIDType" minOccurs="0"/>
    <xs:element name="submissionTime" type="xs:dateTime" minOccurs="0"/>
    <xs:element name="extension" type="xaip:extensionType" minOccurs="0"/>
  </xs:sequence>
</xs:complexType>
```

<clientID> [required]

Das **<clientID>**-Element enthält den Identifikator der aufrufenden Geschäftsanwendung. Weitere Details zu diesem Element sind in **[SAMLv2]** Abschnitt 2.2.2 spezifiziert. Dieses Element kann von der TR-ESOR-Middleware aus den verfügbaren Authentisierungsinformationen abgeleitet werden.

<submissionUnit> [optional]

Das **<submissionUnit>**-Element bezeichnet bei Bedarf die Organisationseinheit der aufrufenden Geschäftsanwendung. Die Struktur des Elements ist in **[SAMLv2]** Abschnitt 2.2.2 spezifiziert.

<submissionAuthor> [optional]

Das **<submissionAuthor>**-Element bezeichnet bei Bedarf den Autor bzw. Absender des Archivinformationspakets. Die Struktur des Elements ist in **[SAMLv2]** Abschnitt 2.2.2 spezifiziert.

<submissionTime> [optional]

Das **<submissionTime>**-Element soll von der aufrufenden Geschäftsanwendung eingefügt werden und gibt den Zeitpunkt der Übergabe des Archivinformationspakets an.

<extension> [optional]

Durch das optionale **<extension>**-Element können benutzerspezifische Erweiterungen geschaffen werden. Die Struktur des **xaip:extensionType** ist auf Seite 16 dargestellt.

Es wird empfohlen, diese Erweiterungen und hierdurch entstehende XAIP-Profile mit dem Bundesamt für Sicherheit in der Informationstechnik abzustimmen⁷.

Die Struktur des **xaip:packageInfoUnitType** im **xaip:VersionManifestType** ist folgendermaßen gegeben:

```
<xs:complexType name="packageInfoUnitType">
  <xs:sequence>
    <xs:element name="unitType" type="xs:string" minOccurs="0"/>
    <xs:element name="textInfo" type="xs:string" minOccurs="0"/>
    <xs:element name="protectedObjectPointer" type="xs:IDREF"
      minOccurs="0" maxOccurs="unbounded"/>
    <xs:element name="unprotectedObjectPointer" type="xs:IDREF" minOccurs="0"
      maxOccurs="unbounded"/>
    <xs:element name="packageInfoUnit" type="xaip:packageInfoUnitType"
      minOccurs="0" maxOccurs="unbounded"/>
    <xs:element name="extension" type="xaip:extensionType" minOccurs="0"/>
  </xs:sequence>
  <xs:attribute name="packageUnitID" type="xs:ID" use="required"/>
</xs:complexType>
```

<unitType> [optional]

Das **<unitType>**-Element gibt den Typ der betreffenden Inhaltsdateneinheit an.

Die möglichen Ausprägungen des `<unitType>`-Elementes sollen im Rahmen von XAIP-Profilen festgelegt werden. Es wird empfohlen, die XAIP-Profile mit dem Bundesamt für Sicherheit in der Informationstechnik abzustimmen⁷.

`<textInfo>` [optional]

Das `<textInfo>`-Element enthält Informationen zur abgelegten Inhaltsdateneinheit.

`<protectedObjectPointer>` [required, unbounded]

Durch die Menge der hier angegebenen `<protectedObjectPointer>`-Elemente wird definiert, welche Teile des Archivinformationspakets in die Hashwertbildung einfließen und deshalb vom entsprechenden Evidence Record geschützt werden. Hinsichtlich der Details der Hashwertbildung wird zwischen Nutzdaten und beweisrelevanten Daten einerseits und Metadaten und sonstigen XAIP-spezifischen XML-Strukturen, wie z. B. `VersionManifest` und `packageInfoUnit`, unterschieden:

Bei Nutzdaten, binär abgelegten Metadaten (`<binaryMetaData>`-Element) und beweisrelevanten *Daten* werden die eigentlichen Daten – **ohne** umschließende XML-Tags und in ihrem ursprünglichen Format (d. h. ohne Base64-Codierung) – der Hashwertbildung zugeführt, so dass solche bereits durch Evidence Records geschützte Daten nahtlos importiert und in einen XAIP-Container eingebettet werden können.

Bei Metadaten, die nicht binär abgelegt wurden (`<xmlMetaData>`-Element), und sonstigen XAIP-spezifischen XML-Strukturen hingegen wird das komplette XML-Element – **inklusive** der umschließenden XML-Tags und der möglicherweise darin enthaltenen XML-Attribute – mit dem spezifizierten Kanonisierungsverfahren kanonisiert und sodann der Hashwertbildung zugeführt.¹¹

Die in dieser Weise gebildeten Hashwerte bilden eine Datenobjekt-Gruppe (data object group) im Sinne von [RFC4998] und [RFC6283].

`<unprotectedObjectPointer>` [optional, unbounded]

Durch einen hier angegebenen `unprotectedObjectPointer` wird klargestellt, dass das Objekt, auf das hier verwiesen wird, logisch zur angegebenen XAIP-Version gehört. Allerdings fließt dieses Objekt *nicht* in die Hashwertbildung ein und es ist deshalb nicht von einem Evidence Record umfasst und kryptographisch geschützt. Beispielsweise kan mit einem solchen Element auf einen zu dieser Version gehörenden Evidence Record verwiesen werden, der in der Credential Section abgelegt ist.

`<packageInfoUnit>` [optional, unbounded]

Ein `packageInfoUnit`-Element kann wiederum eine Folge von `packageInfoUnit`-Elementen enthalten, so dass hierdurch hierarchisch verschachtelte Strukturen abgebildet werden können.

`<extension>` [optional]

Durch das optionale `<extension>`-Element können benutzerspezifische Erweiterungen geschaffen werden. Die Struktur des **xaip:extensionType** ist auf Seite 16 dargestellt.

Es wird empfohlen, diese Erweiterungen und hierdurch entstehende XAIP-Profile mit dem Bundesamt für Sicherheit in der Informationstechnik abzustimmen⁷.

`@packageUnitID` [required]

Das `packageUnitID`-Attribut fungiert als eindeutiger Identifikator der Inhaltsdateneinheit.

¹¹ Beispielsweise könnte hier auf das `packageID`-Attribut (siehe Seite 15) verwiesen werden, um das `packageHeader`-Element in die Hashbaumbildung einzubeziehen. Allerdings ist in diesem Fall zu berücksichtigen, dass sich die `packageHeader`-Struktur durch weitere Updates verändern würde und deshalb der kryptographische Schutz des `packageHeader`-Elementes oft nur dann sinnvoll ist, sofern keine weiteren Aktualisierungen mehr zu erwarten sind.

Die Struktur des **xaip:idAssignmentListType** im **xaip:VersionManifestType** ist folgendermaßen gegeben:

```
<xs:complexType name="idAssignmentListType">
  <xs:sequence>
    <xs:element ref="xaip:idAssignmentPointer" maxOccurs="unbounded"/>
  </xs:sequence>
  <xs:attribute name="idAssignmentListID" type="xs:ID" use="required"/>
</xs:complexType>
```

<idAssignmentPointer> [required, unbounded]

Das **<idAssignmentPointer>**-Element ordnet ein **dataObjectID**-Attribut bzw. **metaDataID**-Attribut den zugehörigen binären Daten zu. Die Struktur des **xaip:idAssignmentPointerType** ist weiter unten erläutert.

@idAssignmentListID [required]

Das **idAssignmentListID**-Attribut fungiert als eindeutiger Identifikator dieser Struktur.

Die Struktur des **xaip:idAssignmentPointerType** ist folgendermaßen gegeben:

```
<xs:complexType name="idAssignmentPointerType">
  <xs:sequence>
    <xs:element ref="xaip:checkSum"/>
  </xs:sequence>
  <xs:attribute name="objectRef" type="xs:IDREF" use="required"/>
</xs:complexType>

<xs:element name="idAssignmentPointer" type="xaip:idAssignmentPointerType"/>
```

<checkSum> [required, unbounded]

Das **<checkSum>**-Element beinhaltet einen Hashwert über den Nettoinhalt des referenzierten Daten- bzw. Metadatenobjekts. Die Details zur Struktur des **xaip:checkSumType** sind auf der Seite 25 zu finden.

@objectRef [required]

Das **objectRef**-Attribut stellt klar, welches Daten- (**dataObjectID**-Attribut) bzw. Metadatenobjekt (**metaDataID**-Attribut) gemeint ist.

3.1.3 Der xaip:metaDataSectionType

Die Struktur des **xaip:metaDataSectionType** ist folgendermaßen gegeben:

```
<xs:complexType name="metaDataSectionType">
  <xs:sequence>
    <xs:element ref="xaip:metaDataObject" maxOccurs="unbounded"/>
  </xs:sequence>
</xs:complexType>

<xs:element name="metaDataObject" type="xaip:metaDataObjectType"/>
```

<metaDataObject> [required, unbounded]

Der **xaip:metaDataSectionType** besteht aus einer Folge von **xaip:metaDataObject**-Elementen vom Typ **xaip:metaDataObjectType** in denen die entsprechenden Metadaten abgelegt sind.

Die Struktur des **xaip:metaDataObjectType** ist folgendermaßen gegeben:

```
<xs:complexType name="metaDataObjectType">
  <xs:sequence>
    <xs:choice>
      <xs:element ref="xaip:binaryMetaData"/>
      <xs:element name="xaip:xmlMetaData" type="dss:AnyType"/>
    </xs:choice>
    <xs:element ref="xaip:checkSum" minOccurs="0"/>
  </xs:sequence>
  <xs:attribute name="metaDataID" type="xs:ID" use="required"/>
  <xs:attribute name="relatedObjects" type="xs:IDREFS" use="required"/>
  <xs:attribute name="category" type="xs:string"/>
  <xs:attribute name="classification" type="xs:string"/>
  <xs:attribute name="type" type="xs:string"/>
</xs:complexType>
```

Demnach ist ein **xaip:metaDataObject** wie folgt strukturiert¹²:

<binaryMetaData> [choice]

Mit Hilfe von diesem Element können die Metadaten als ein Base64-enkodierten Datenstrom abgelegt werden. Das Element ist vom Typ **xaip:binaryMetaDataType**.

<xmlMetaData> [choice]

Dieses Element wird für die Ablage von XML-Metadaten verwendet, wobei der in [OASIS-DSS] definierte **dss:AnyType** zum Einsatz kommt.

<checkSum> [optional]

Dieses Element enthält bei Bedarf die kryptographische Prüfsumme der Metadaten. Gegenstand der Berechnung der Prüfsumme bei **<binaryMetaData>**-Elementen sind Base64-dekodierte Metadaten und bei **<xmlMetaData>**-Elementen die gemäß **<packageHeader>/<CanonicalizationMethod>** kanonisierten XML-Metadaten. Weitere Details des **xaip:checkSumType** sind im Kap. 3.1.4 näher erläutert. Die Checksumme bezieht sich dabei immer auf die Metadaten, wie sie tatsächlich im XAIP vorliegen, also insbesondere unter Berücksichtigung möglicher, bereits durchgeführter Transformationen.

@metaDataID [required]

Das **metaDataID**-Attribut fungiert als eindeutiger Identifikator des Metadatenobjektes, so dass auf dieses Attribut Bezug genommen werden kann.

@relatedObjects [required]

Mit dem **relatedObjects**-Attribut wird klargestellt, auf welche Datenobjekte¹³ oder auf welche XAIP-Strukturelemente sich das vorliegende Metadatenobjekt bezieht. Allerdings ist hierbei zu bedenken, dass die **dataObjectID** eines binären Datenobjektes (vgl. Abschnitt 3.1.4) nicht durch kryptographische Mechanismen geschützt ist und deshalb durch den oben erläuterten **protectedObjectPointer**-Mechanismus ein Metadatenobjekt später unbemerkt einem anderen Datenobjekt zugeordnet werden könnte. Um diese Mehrdeutigkeit zu vermeiden, soll ein Metadatenobjekt, das auf ein binäres Datenobjekt verweist, ein unten erläutertes **dataObjectCheckSum**-Element enthalten, durch das das Datenobjekt eindeutig bezeichnet wird.

@category [optional]

Das **category**-Attribut bestimmt die generelle Kategorie, dem das Metadatenobjekt zugeordnet ist.

¹² Eine Empfehlung zum Umgang mit signierten Metadaten ist dem Anhang 7 zu entnehmen.

¹³ Ein Meta-Datenobjekt kann einem (oder mehreren) Nutzdatenobjekt (**dataObject**), einem (oder mehreren) Beweisdatenobjekt (**credential**) oder einem anderen (oder mehreren anderen) Meta-Datenobjekt (**metaDataObject**) oder einem (oder mehreren) XAIP-Strukturelement zugeordnet sein.

@classification [optional]

Das `classification`-Attribut liefert weitere Informationen zur näheren Klassifizierung des Metadatenobjektes innerhalb der über das `category`-Attribut definierten Kategorie.

@type [optional]

Das `type`-Attribut gibt den konkreten Typ des Metadatenobjektes an. Dieses Attribut bestimmt die interne syntaktische Struktur und Semantik des `metaDataObject`-Elementes. Die verwendeten `type`-Attribute und die dazu korrespondierenden Kind-Elemente des `xaip:metaDataObject`-Elementes sollen in XAIP-Profilen festgeschrieben werden. Es wird empfohlen, diese XAIP-Profile mit dem Bundesamt für Sicherheit in der Informationstechnik abzustimmen⁷.

Sofern nicht durch eine XAIP-Profilierung etwas anderes bestimmt ist, sollen für die `category` und `classification` Attribute die folgenden, in [XFDU] definierten und dem OAIS-Modell entsprechenden Werte genutzt werden:

Tabelle 2: Wertauflistung für die Attribute `category` und `classification` gem. [XFDU] und [OAIS].

Kategorie	Klassifikation	Beschreibung
DMD ¹⁴	DESCRIPTION	beschreibende (fachliche) Metadaten, die das referenzierte Datenobjekt näher beschreiben, um beispielsweise das Auffinden und den Zugriff zu ermöglichen oder zu unterstützen.
DMD	OTHER	Sonstige beschreibende Metadaten.
REP ¹⁵	SYNTAX	Metadaten, die die Syntax eines anderen Datenobjektes näher beschreiben. ¹⁶
REP	DED ¹⁷	Metadaten, die die Semantik eines anderen Datenobjektes näher beschreiben. ¹⁸
REP	OTHER	Sonstige Metadaten, die die Darstellung eines anderen Datenobjektes unterstützen.
PDI ^{19 20 21}	REFERENCE	Metadaten, die die Bildung von Identifikatoren beschreiben und externen Systemen den Zugriff auf ein Archivdatenobjekt ermöglichen.
PDI	CONTEXT	Metadaten, die die Umgebung beschreiben, in der das Archivdatenobjekt entstanden ist.

¹⁴ Die Abkürzung DMD bedeutet Descriptive Meta Data.

¹⁵ Die Abkürzung REP bedeutet Representation Meta Data.

¹⁶ Hier soll das verwendete Datenformat in einer für die spätere Verarbeitung hinreichend präzisen Weise beschrieben werden. Beispielsweise können hier XML-Schema-Dateien abgelegt werden, die bei einer Schema-Validierung von anwendungsspezifischen Datenobjekten eingesetzt werden können.

¹⁷ DED = Data Entity Dictionary

¹⁸ Beispielsweise können hier Ontologie-Dateien abgelegt werden, in denen die Semantik von anwendungsspezifischen Datenobjekten beschrieben ist.

¹⁹ PDI = Preservation Description Information

²⁰ Es sei angemerkt, dass im XAIP keine Metadaten der Klasse FIXITY auftreten, da die Elemente zum Schutz der Integrität und Authentizität im XAIP in der Credential-Section abgelegt werden.

²¹ Bei den Metadaten der Kategorie PDI kann es sich sowohl um fachliche (z. B. Aktenzusammenhang, Bearbeitungs- und Protokollinformationen) als auch technische (z. B. Informationen zu erfolgten Konvertierungen sowie zur Soft- und Hardwareumgebung) Metadaten handeln.

Kategorie	Klassifikation	Beschreibung
PDI	PROVENANCE	Metadaten, die die Entstehungsgeschichte des Archivdatenobjektes dokumentieren. ²²
PDI	OTHER	Sonstige aufbewahrungsspezifische Metadaten. Mit der Kategorie "OTHER" können Metadaten abgelegt werden, die sich nicht eindeutig einer der vorgenannten Kategorien (DMD, REP, PDI) zuordnen lassen.

Weiterhin besteht eine Möglichkeit, binäre Metadaten durch die Verwendung des **xaip:binaryMetaData**-Elements innerhalb des **xaip:metaDataObject**-Elements abzulegen.

```
<xs:element name="binaryMetaData" type="xaip:binaryMetaDataType"/>
```

Der **xaip:binaryMetaData**-Element wird durch den **xaip:binaryMetaDataType** definiert, dessen Struktur wie folgt aufgebaut ist:

```
<xs:complexType name="binaryMetaDataType" xmlns:mime="*/*>
  <xs:simpleContent>
    <xs:extension base="xs:base64Binary">
      <xs:attribute name="MimeType" type="xs:string" use="optional"/>
    </xs:extension>
  </xs:simpleContent>
</xs:complexType>
```

Der Typ **xaip:binaryMetaDataType** erweitert den Typ **xs:base64Binary**. Die darin abgelegten binären Daten müssen gemäß [RFC4648] Base64 kodiert sein.

@MimeType [required]

Der Typ der Daten soll im MimeType-Attribut gemäß der von IANA gepflegten Liste der registrierten Media-Types²³ angegeben werden.

```
<xs:element name="dataObjectChecksum" type="xaip:checksumType"/>
```

<dataObjectChecksum> [optional]

Durch das **dataObjectChecksum**-Element kann bei Bedarf der kryptographische Hashwert des Datenobjektes, auf das sich das vorliegende Metadatenobjekt bezieht und auf das mit dem unten genannten **dataObjectID**-Attribut verwiesen wird, in das Metadatenobjekt eingefügt werden, so dass die Zuordnung zwischen dem zu Grunde liegenden Datenobjekt und dem vorliegenden Metadatenobjekt mit kryptographischen Mitteln sichergestellt wird.

3.1.4 Der xaip:dataObjectsSectionType

Die Struktur des **xaip:dataObjectsSectionType** ist folgendermaßen gegeben:

```
<xs:complexType name="dataObjectsSectionType">
  <xs:sequence>
    <xs:element ref="xaip:dataObject" maxOccurs="unbounded" minOccurs="1"/>
  </xs:sequence>
</xs:complexType>

<xs:element name="dataObject" type="xaip:dataObjectType"/>
```

<dataObject> [required, unbounded]

Der **xaip:dataObjectsSectionType** besteht aus einer Folge von **xaip:dataObject**-Elementen vom Typ **xaip:dataObjectType**.

²² Für die Spezifikation von Metadaten der Klasse PROVENANCE können beispielsweise die in [PREMIS] definierten Ereignisse genutzt werden.

²³ Siehe <http://www.iana.org/assignments/media-types/media-types.xhtml>.

Die Struktur des **xaip:dataObjectType** ist folgendermaßen gegeben:

```
<xs:complexType name="dataObjectType">
  <xs:sequence>
    <xs:choice>
      <xs:element ref="xaip:binaryData"/>
      <xs:element name="xmlData" type="dss:AnyType"/>
    </xs:choice>
    <xs:element ref="checksum" minOccurs="0"/>
    <xs:element name="transformInfo" type="xaip:transformInfoType" minOccurs="0"/>
  </xs:sequence>
  <xs:attribute name="dataObjectID" type="xs:ID" use="required"/>
  <xs:attribute name="relatedObjects" type="xs:IDREFS" use="optional"/>
</xs:complexType>
```

<binaryData> [choice]

Dieses Element wird für die Ablage von Binärdaten verwendet und ist vom Typ **xaip:binaryDataType**.

(A3.1-1) Als besondere Archivinformationspakete vom Typ **xaip:binaryDataType** sind nur die in HINWEIS 4 aufgeführten Bewahrungsobjekt-Formaten (engl. preservation object formats) mit den dort aufgeführten Mime-Types zulässig.

<xmlData> [choice]

Dieses Element wird für die Ablage von XML-Daten verwendet, wobei der in [OASIS-DSS] definierte **dss:AnyType** zum Einsatz kommt.

Die TR-ESOR-Middleware soll eine entsprechende Validierung der übergebenen Daten bezüglich administrativ festgelegter XML-Schema-Dateien durchführen.

<checksum> [optional]

Dieses Element enthält bei Bedarf die kryptographische Prüfsumme des Datenobjektes. Gegenstand der Berechnung der Prüfsumme ist bei **<binaryData>**-Elementen das Base64-dekodierte Nutzdatenobjekt und bei **<xmlData>**-Elementen die gemäß **<packageHeader>/<CanonicalizationMethod>** kanonisierten XML-Daten. Weitere Details des **xaip:checksumType** sind unten näher erläutert. Die Checksumme bezieht sich dabei immer auf die Nutzdaten wie sie tatsächlich im XAIP vorliegen, also insbesondere unter Berücksichtigung möglicher, bereits durchgeführter Transformationen.

<transformInfo> [optional]

Dieses Element wird dazu benutzt, um eine oder mehrere Operationen (Transformationen), die auf den Originaldaten vor der Aufnahme in die Bewahrung ausgeführt wurden²⁴, zu dokumentieren, um diese Schritte bei Bedarf erneut durchführen oder – sofern es sich um eine reversible Transformation handelt – automatisiert rückgängig machen zu können. Die Syntax des **xaip:transformInfoType** ist unten definiert.

Die Festlegung weiterer Details muss im Rahmen eines entsprechenden XAIP-Profiles erfolgen. Es wird empfohlen, XAIP-Profile mit dem Bundesamt für Sicherheit in der Informationstechnik abzustimmen⁷.

Sofern eine Transformation auch manuelle Abläufe oder unvollständig spezifizierte Prozessschritte umfasst, die nicht vollständig automatisiert rückgängig gemacht werden könnten, soll das **transformInfo**-Element nicht genutzt werden. Vielmehr wird in einem solchen Fall die Verwendung

²⁴ Beispielsweise können hierdurch verschlüsselt abgelegte Daten zur Verarbeitung entschlüsselt werden. Darüber hinaus kann das **transformInfo**-Element dazu genutzt werden, um notwendige Decodierungs- und Konvertierungsschritte zu beschreiben. Die Beschreibung der obligatorischen Base64-Codierung bei der Verarbeitung von Binärdaten muss jedoch nicht angegeben werden. Vielmehr bezieht sich die Prüfsumme hier ohnehin auf die ursprünglichen Originaldaten.

eines geeigneten Metadatenobjektes mit `category=PDI` und `classification=PROVENANCE` empfohlen.

@dataObjectID [required]

Das dataObjectID-Attribut fungiert als eindeutiger Identifikator des Datenobjektes.

@relatedObjects [optional]

Das relatedObjects-Attribut kann optional Referenzen auf weitere Objekte innerhalb des (L)XAIP beinhalten, die in einer Relation mit diesem konkreten Datenobjekt stehen (z. B. Referenzen auf die diesem Datenobjekt zugewiesenen Metadaten etc.).

Das **xaip:binaryData**-Element basiert auf dem **xaip:binaryDataType**, der folgendermaßen beschrieben ist:

```
<xs:element name="binaryData" type="xaip:binaryDataType" />

<xs:complexType name="binaryDataType" xmlns:xmime="http://www.w3.org/2003/11/xmime">
  <xs:simpleContent>
    <xs:extension base="xs:base64Binary">
      <xs:attribute name="MimeType" type="xs:string" use="optional"/>
    </xs:extension>
  </xs:simpleContent>
</xs:complexType>
```

Der Typ **xaip:binaryDataType** erweitert den Typ **xs:base64Binary**. Die darin abgelegten binären Daten müssen gemäß [RFC4648] Base64 kodiert sein.

@MimeType [required]

Der Typ der Daten soll im MimeType-Attribut gemäß der von IANA gepflegten Liste der registrierten Media-Types²⁵ angegeben werden.

(A3.1-2) Die TR-ESOR-Middleware soll eine entsprechende Validierung der übergebenen Daten hinsichtlich des durch das MimeType-Attribut spezifizierten Datenformates durchführen.

Die Struktur des **xaip:checksumType** ist folgendermaßen gegeben:

```
<xs:element name="checksum" type="xaip:checksumType"/>

<xs:complexType name="checksumType">
  <xs:sequence>
    <xs:element name="checksumAlgorithm" type="xs:anyURI"/>
    <xs:element name="checksum" type="xs:hexBinary"/>
  </xs:sequence>
</xs:complexType>
```

<checksumAlgorithm> [required]

Dieses Element spezifiziert den für die Erstellung der Prüfsumme verwendeten Algorithmus. Eine Auflistung der hierfür geeigneten Algorithmen ist der jeweils aktuellen Version von [ETSI TS 119 312] oder [SOG-IS] zu entnehmen.

Eine Einschränkung der zu verwendenden Algorithmen oder die Spezifikation zusätzlicher Algorithmen kann im Rahmen eines XAIP-Profiles erfolgen.

<checksum> [required]

Dieses Element enthält die mit dem oben angegebenen Algorithmus berechnete kryptographische Prüfsumme.

²⁵ Siehe <http://www.iana.org/assignments/media-types/media-types.xhtml>.

Die Struktur des **xaip:transformInfoType** umfasst eine Folge von `transformObject`-Elementen und ist folgendermaßen gegeben:

```
<xs:complexType name="transformInfoType">
  <xs:sequence>
    <xs:element name="transformObject" type="xaip:transformObjectType"
      maxOccurs="unbounded"/>
  </xs:sequence>
</xs:complexType>
```

Die Struktur des **xaip:transformObjectType** ist folgendermaßen gegeben:

```
<xs:complexType name="transformObjectType">
  <xs:sequence>
    <xs:element name="transformAlgorithm" type="xs:anyURI"/>
    <xs:element name="Parameters" type="xs:anyType" minOccurs="0"/>
  </xs:sequence>
  <xs:attribute name="transformObjectID" type="xs:ID" use="required"/>
  <xs:attribute name="order" type="xs:string" />
</xs:complexType>
```

`<transformAlgorithm>` [required]

Dieses Element spezifiziert den für die Transformation verwendeten Algorithmus, wobei es sich beispielsweise um einen Kompressions-, Kanonisierungs- oder Verschlüsselungsalgorithmus handeln kann.

`<parameters>` [optional]

Dieses Element enthält bei Bedarf weitere Parameter, welche für die Durchführung der Transformation benötigt werden. Die detaillierte Struktur dieses Elements ist abhängig vom verwendeten Transformationsalgorithmus.

`@transformObjectID` [required]

Das `transformObjectID`-Attribut fungiert als eindeutiger Identifikator des Transformationsobjektes.

`@order` [optional]

Sofern mehrere reversible Transformationen vorhanden sind, spezifiziert das `order`-Attribut in welcher Reihenfolge die Folge der Transformationen rückgängig gemacht werden kann. Angelehnt an [XFDU] ist das `order`-Attribut eine positive ganze Zahl und der Beginn der Bearbeitung startet bei der Zahl "1" und wird in jedem Schritt inkrementiert.

Eine Festlegung der zulässigen Algorithmen und Parameter muss im Rahmen eines XAIP-Profils erfolgen. Es wird empfohlen, ein solches XAIP-Profil mit dem Bundesamt für Sicherheit in der Informationstechnik abzustimmen⁷.

3.1.5 Der xaip:credentialsSectionType

Die Struktur des **xaip:credentialsSectionType** ist folgendermaßen gegeben:

```
<xs:complexType name="credentialsSectionType">
  <xs:sequence>
    <xs:element ref="xaip:credential" maxOccurs="unbounded"/>
  </xs:sequence>
</xs:complexType>
```

```
<xs:element name="credential" type="xaip:credentialType"/>
```

<credential> [required, unbounded]

Der **xaip:credentialsSectionType** besteht aus einer Folge von **xaip:credential**-Elementen vom Typ **xaip:credentialType**, die "technische Beweisdaten" oder "beweisrelevante Daten"²⁶ enthalten.

Die Struktur des **xaip:credentialType** ist folgendermaßen gegeben:

```
<xs:complexType name="credentialType">
  <xs:choice>
    <xs:element ref="dss:SignatureObject"/>
    <xs:element name="certificateValues" type="xades:CertificateValuesType"/>
    <xs:element name="revocationValues" type="xades:RevocationValuesType"/>
    <xs:element ref="xaip:evidenceRecord"/>
    <xs:element ref="vr:VerificationReport"/>
    <xs:element name="other" type="xaip:extensionType"/>
  </xs:choice>
  <xs:attribute name="relatedObjects" type="xs:IDREFS"/>
  <xs:attribute name="credentialID" type="xs:ID" use="required"/>
</xs:complexType>
```

<dss:SignatureObject> [choice]

In diesem Element können bei Bedarf Signaturobjekte und Zeitstempel für Nutz- oder Metadatenobjekte abgelegt werden, auf die mit dem **relatedObjects**-Attribut verwiesen wird. Die Struktur des **dss:SignatureObject**-Elementes ist in [OASIS-DSS] definiert.

<certificateValues> [choice]

In diesem Element können bei Bedarf²⁷ Zertifikate abgelegt werden, die für die Prüfung von digitalen Signaturen und Zeitstempeln benötigt werden. Die Struktur dieses Elementes ist in [XAdES] definiert. Das **relatedObjects**-Attribut soll in diesem Fall auf die entsprechenden digitalen Signaturen oder Zeitstempel verweisen.

²⁶ „Technische Beweisdaten“ dienen dem Nachweis der Unversehrtheit, der Integrität und Authentizität der bewahrten bzw. archivierten Datenobjekte. In Übereinstimmung mit den Spezifikationen des ERS-Standards der IETF enthält ein technischer Beweisdatensatz Archivzeitstempel ausreichender Qualität über die gespeicherten (signierten) Datenobjekte, die die Unversehrtheit der Daten nachweisen und zusätzlich Informationen die die Richtigkeit und die Gültigkeit digitaler Signaturen zum Signaturzeitpunkt sowie die rechtzeitige und rechtskonforme Signatur- bzw. Siegelerneuerung belegen. „Beweisrelevante Daten“ sind Signaturen bzw. Siegel bzw. Zeitstempel zu genau einem Datenobjekt bzw. Dokument und enthalten auch die für die Prüfung der Signatur bzw. Siegel bzw. Zeitstempelsignatur bzw. - Siegel notwendigen Prüfdaten wie z. B. Zertifikate sowie CRL-Listen und OCSP-Responses zu diesen Zertifikaten. Die Beweisdaten belegen, dass das Dokument ab dem Zeitpunkt der Bewahrung bzw. Archivierung nicht mehr verändert wurde. Die beweisrelevanten Daten belegen, dass die ggf. außerhalb des Bewahrung- bzw. Archivsystems erzeugten digitalen Signaturen und Zeitstempel zum Erstellungszeitpunkt bzw. Bewahrungs- oder Archivierungszeitpunkt gültig waren.

²⁷ Wie in Abschnitt **Fehler! Verweisquelle konnte nicht gefunden werden.** erläutert, soll dieses Element nur bei sonstigen Signaturformaten (vgl. Abschnitt 5.1.5), die *nicht* dem PKCS#7/CMS/CAdES- oder XML/XAdES-Standard genügen, genutzt werden.

<revocationValues> [choice]

In diesem Element können bei Bedarf²⁸ Zertifikatsstatusinformationen²⁹ abgelegt werden, die für die Prüfung von digitalen Signaturen und Zeitstempeln benötigt werden. Die Struktur dieses Elementes ist in [XAdES] definiert. Das relatedObjects-Attribut soll in diesem Fall auf die entsprechenden Signaturen oder Siegel oder Zeitstempel verweisen.

<evidenceRecord> [choice]

In diesem Element können bei Bedarf Beweisdaten in Form von Evidence Records gemäß [RFC4998] und [RFC6283]³⁰ abgelegt werden. Die Struktur des xaip:evidenceRecord-Elementes ist unten näher erläutert. Gemäß der vorliegenden Spezifikation bezieht sich ein Evidence Record im Regelfall auf eine bestimmte XAIP-Version und das relatedObjects-Attribut soll auf das entsprechende versionID-Attribut des <versionManifest>-Elementes verweisen. Durch diesen Verweis können bei Bedarf die entsprechenden <protectedObjectPointer>-Elemente ermittelt werden, wodurch klargestellt ist, auf welche Datenobjekte sich der Evidence Record genau bezieht. Darüber hinaus ist bei der Beschreibung des <protectedObjectPointer>-Elementes (siehe Seite 19) klargestellt, wie die Hashwertbildung bei den verschiedenen Objekten genau erfolgt.

<vr:VerificationReport> [choice]

In diesem Element sollen die Ergebnisse der bei der Übergabe durchgeführten Signatur- bzw. Siegel- bzw. Zeitstempelprüfung abgelegt werden. Wie in [OASIS-VR] näher erläutert, enthält ein vr:VerificationReport-Element eine Folge von IndividualReport-Elementen, in denen sich Prüfergebnisse für individuelle Beweisdatenobjekte befinden. Sofern ein vr:VerificationReport mehrere IndividualReport-Elemente enthält, muss im WhichDocument-Attribut des IndividualReport/SignedObjectIdentifier-Elementes auf das geprüfte Beweisdatenobjekt verwiesen werden. Details zum vr:VerificationReport-Element sind in [OASIS-VR] definiert.

<other> [choice]

Im other-Element können andere beweisrelevante Daten und Beweisdaten abgelegt werden.

@relatedObjects [optional]

Das relatedObjects-Attribut verweist bei Bedarf auf das oder die Objekte, auf das sich das Beweisdatenobjekt (Credential) bezieht. Falls es sich bei dem Beweisdatenobjekt um ein evidenceRecord-Element handelt, verweist das relatedObjects-Attribut auf das zugehörige VersionID-Attribut. Bei einem vr:VerificationReport-Element verweist das vorliegende Attribut auf das geprüfte Objekt, das beispielsweise ein dss:SignatureObject oder ein evidenceRecord sein kann.

@credentialID [required]

Das credentialID-Attribut fungiert bei Bedarf als eindeutiger Identifikator des Beweisdatenobjektes (Credential).

²⁸ Wie in Abschnitt **Fehler! Verweisquelle konnte nicht gefunden werden.** erläutert, soll dieses Element nur bei sonstigen Signaturformaten (vgl. Abschnitt 5.1.5), die *nicht* dem PKCS#7/CMS/CAdES- oder XML/XAdES-Standard genügen, genutzt werden.

²⁹ Sofern für das zu prüfende Zertifikat sowohl Sperrinformationen in Form von CRLs als auch OCSP-Responses vorliegen sollen hier OCSP-Responses verwendet werden.

³⁰ [RFC4998] muss, [RFC6283] kann unterstützt werden.

Das `xaip:evidenceRecord`-Element basiert auf dem **`xaip:evidenceRecordType`**, der entweder auf dem in [RFC6283] näher beschriebenen **`ers:EvidenceRecordType`** oder auf dem in [RFC4998] näher beschriebenen **`asn1EvidenceRecord`** gestützt ist und folgendermaßen definiert ist:

```
<xs:element name="evidenceRecord" type="xaip:EvidenceRecordType" />

<xs:complexType name="EvidenceRecordType" >
  <xs:choice>
    <xs:element name="xmlEvidenceRecord" type="ers:EvidenceRecordType"/>
    <xs:element name="asn1EvidenceRecord" type="xs:base64Binary"/>
  </xs:choice>
  <xs:attribute name="AOID" type="xs:string"/>
  <xs:attribute name="VersionID" type="xs:string"/>
</xs:complexType>
```

`<xmlEvidenceRecord>` [choice]

XML-basierte Evidence Records gemäß [RFC6283] werden im `xaip:xmlEvidenceRecord`-Element abgelegt.

`<asn1EvidenceRecord>` [choice]

ASN.1-basierte Evidence Records gemäß [RFC4998] werden Base64-codiert im `xaip:asn1EvidenceRecord`-Element abgelegt.

@AOID [optional]

Mit Hilfe von AOID-Attribut kann auf das zugehörige Archivinformationspaket eindeutig verwiesen werden.

@VersionID [optional]

Mit Hilfe von VersionID-Attribut kann auf die zugehörige Version des Archivinformationspakets verwiesen werden.

Sofern das `xaip:evidenceRecord`-Element wie hier in ein entsprechendes `xaip:credential`-Element innerhalb eines `xaip:XAIP`-Elementes eingebettet ist, kann auf die zusätzlichen Attribute (AOID und VersionID) verzichtet werden.

3.1.6 Das Delta-XAIP-Element <DXAIP>

Im Rahmen der ArchiveUpdate-Funktion (siehe Anlage [TR-ESOR-E]) wird ein `<DXAIP>`-Element übergeben, das zusätzlich zur üblichen `<XAIP>`-Struktur (siehe Abschnitt 3.1) eine zusätzliche `updateSection` vom Typ **`xaip:updateSectionType`** enthält und folgendermaßen spezifiziert ist:

```
<xs:element name="DXAIP" type="xaip:DXAIPType" />

<xs:complexType name="DXAIPType">
  <xs:complexContent>
    <xs:extension base="xaip:XAIPType">
      <xs:sequence>
        <xs:element name="updateSection" type="xaip:updateSectionType"/>
      </xs:sequence>
    </xs:extension>
  </xs:complexContent>
</xs:complexType>
```

Hiermit enthält das `<DXAIP>`-Element folgende Kindelemente:

`<packageHeader>` [required]

Das `<packageHeader>`-Element im `<DXAIP>`-Element umfasst folgende Kindelemente:

- `<AOID>` [required] – spezifiziert das durch das Delta-XAIP zu ergänzende Archivdatenobjekt.
- `<packageInfo>` [optional] – sofern dieses Element noch nicht im XAIP existiert, wird es entsprechend angelegt. Sofern das Element bereits vorhanden ist, wird das übergebene

Element ignoriert und es wird eine entsprechende Warnung zurückgeliefert.

- `<versionManifest>` [required] – muss genau einmal vorhanden sein und spezifiziert die neue Version des Archivinformationspakets. Das in diesem Element enthaltene `versionID`-Attribut darf nicht unter den bereits im existierenden `<XAIP>`-Element vergebenen Identifikatoren sein und soll durch Inkrementierung der im `<updateSection>`-Element angegebenen Version ermittelt werden. Die hierin enthaltenen `<protectedObjectPointer>` und `<unprotectedObjectPointer>`-Elemente müssen entweder auf neu übergebene Meta-, Nutz- oder Credential-Datenobjekte, oder auf entsprechende Platzhalter im `<updateSection>`-Element verweisen.

Daten können in einer neuen Version logisch gelöscht werden, indem die entsprechenden `protectedObjectPointer`- bzw. `unprotectedObjectPointer`-Elemente entfernt werden.

`<metaDataSection>` [optional]

Das `<metaDataSection>`-Element kann zusätzliche Metainformationen zur Beschreibung des Geschäfts- und Archivierungskontextes enthalten. Weitere Informationen zum **xaip:metaDataSectionType** finden sich in Abschnitt 3.1.3.

`<dataObjectsSection>` [optional]

Das `<dataObjectsSection>`-Element kann zusätzliche Nutzdaten des Archivinformationspakets. Weitere Informationen zum **xaip:dataObjectsSectionType** finden sich in Abschnitt 3.1.4.

`<credentialsSection>` [optional]

Das `<credentialsSection>`-Element kann zusätzliche Beweisdaten in Form von Evidence Records oder beweisrelevante Informationen, wie z. B. Signaturen, Siegel, Zeitstempel, Zertifikate oder Signatur- bzw. Siegel- bzw. Zeitstempelprüfinformationen enthalten. Weitere Informationen zum **xaip:credentialsSectionType** finden sich in Abschnitt 3.1.5.

`<updateSection>` [required]

Das `<updateSection>`-Element muss die Information auf welche Version des Archivinformationspakets sich das Update bezieht und bei Bedarf weitere Platzhalter-Elemente enthalten, die das `ID`-Attribut eines unverändert aus einer Vorversion übernommenen Meta-, Nutz- oder Credential-Datenobjektes tragen.

Die Struktur des **xaip:updateSectionType** ist folgendermaßen gegeben:

```
<xs:complexType name="updateSectionType">
  <xs:sequence>
    <xs:element name="prevVersion" type="xs:string"/>
    <xs:element name="placeholder" type="xaip:placeholderType"
      maxOccurs="unbounded" minOccurs="0"/>
  </xs:sequence>
</xs:complexType>
```

`<prevVersion>` [required]

Das `<prevVersion>`-Element muss den eindeutigen Identifikator der Version des Archivinformationspakets enthalten, auf das sich der Updatevorgang bezieht. Weitere Informationen zum `versionID`-Attribut des `versionManifest`-Elementes finden sich auf Seite 17. Damit der Update-Vorgang erfolgreich durchgeführt werden kann, muss das übergebene `<prevVersion>`-Element dem `versionID`-Attribut der aktuellsten Version des Archivinformationspakets entsprechen.

`<placeholder>` [optional, unbounded]

Die `<placeholder>`-Elemente stellen bei Bedarf die fehlenden `ID`-Attribute von unverändert aus einer Vorversion übernommenen Meta-, Nutz- oder Credential-Datenobjekten bereit.

Die Struktur des **xaip:placeholderType** ist folgendermaßen gegeben:

```
<xs:complexType name="placeholderType">
  <xs:attribute name="objectId" type="xs:ID" use="required"/>
</xs:complexType>
```

Somit enthält das oben genannte `<placeholder>`-Element genau das `objectId`-Attribut:

@objectId [required]

Das `objectId`-Attribut des `<placeholder>`-Elementes muss dem `ID`-Attribut eines bereits in einer Vorversion des Archivinformationspakets enthaltenen Meta-, Nutz- oder Credential-Datenobjektes entsprechen.

3.2 Logisches XAIP (LXAIP)

Basierend auf der XAIP-Definition aus Abschnitt 3.1 wird nachfolgend die Definition eines logischen XAIP (LXAIP) dargelegt. Ein LXAIP unterscheidet sich von einem XAIP, dadurch, dass die Inhalte, d.h. die Sequenz der Elemente `xaip:dataObject` aus der `xaip:dataObjectsSection`, `xaip:metaDataObject` aus der `xaip:metaDataSection` oder `xaip:credential` aus der `xaip:credentialsSection` herausgenommen werden, in separate Datenobjekte (z. B. Dateien) abgelegt werden und dafür in das XAIP eine Sequenz von Referenzen auf die entsprechenden `dataObject(s)`, `metaDataObject(s)` oder `credential(s)` eingefügt werden, so dass im XAIP eine entsprechende Verlinkung auf die ausgelagerten Datenobjekte entsteht.

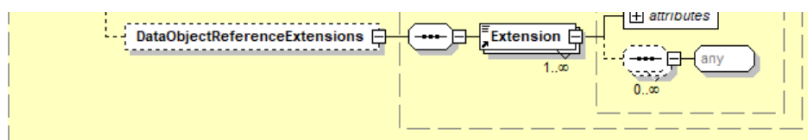
3.2.1 Verlinkung der Datenobjekte in einem LXAIP

Die einzelnen Datenobjekte werden im XAIP innerhalb des Elements `xaip:dataObjectsSection` abgelegt. Hierbei stehen zwei mögliche Ablageoptionen zur Verfügung (vgl. Kap. 3.1.4):

- Ablage als Binärdaten innerhalb des Elements `xaip:binaryData`,
- Ablage als beliebige XML-Daten innerhalb des Elements `xaip:xmlData`.

Der Inhalt der abgelegten Daten wird bislang von einer TR-ESOR-Implementierung nicht näher analysiert und „interpretiert“.

Der Weg für die Erzeugung eines LXAIP, bei dem entsprechende Verweise in ein XAIP-Objekt eingebracht werden, ohne dabei das XAIP-Schema ändern zu müssen, besteht darin, dass ein spezifisches, XML-basiertes, mit einer besonderen Semantik versehenes Element innerhalb des Elements `xaip:xmlData` abgelegt wird, so dass eine TR-ESOR-Implementierung diesem Element besondere Bedeutung beimessen und eine entsprechende Behandlung des spezifischen Inhalts vornehmen kann.



das wiederum ein
auf die relevanten
Abbildung 3

). Sollte eine Instanz

des Elements `asic:DataObjectReference` im Element `xaip:xmlData` vorzufinden sein, dann bedeutet es, dass es sich nicht direkt um die Inhaltsdaten handelt, sondern um eine Referenz, die zunächst vom Kryptomodul aufgelöst werden muss, um die eigentlichen Inhaltsdaten erhalten zu können. Es gilt dabei folgende Bedeutung für die Belegung der einzelnen Unterelemente:

- `URI` – Attribut des Elements `asic:DataObjectReference` – die Ablage der URI, die eindeutig die verlinkten Daten referenziert (z. B. eine URL) – vgl. hierzu Kap. 3.2.6,
- `ds:DigestMethod` – gibt den verwendeten Hashalgorithmus für den obligatorischen Hashwert an, abgelegt im Attribut `Algorithm` (z. B. <http://www.w3.org/2001/04/xmlenc#sha256>),
- `ds:DigestValue` – beinhaltet den Wert des obligatorischen Hashwertes,
- `asic:DataObjectReferenceExtensions` – kann eine beliebige Erweiterung zu der Referenz beinhalten. Dieses Element bleibt im Rahmen dieser Spezifikation unberührt.

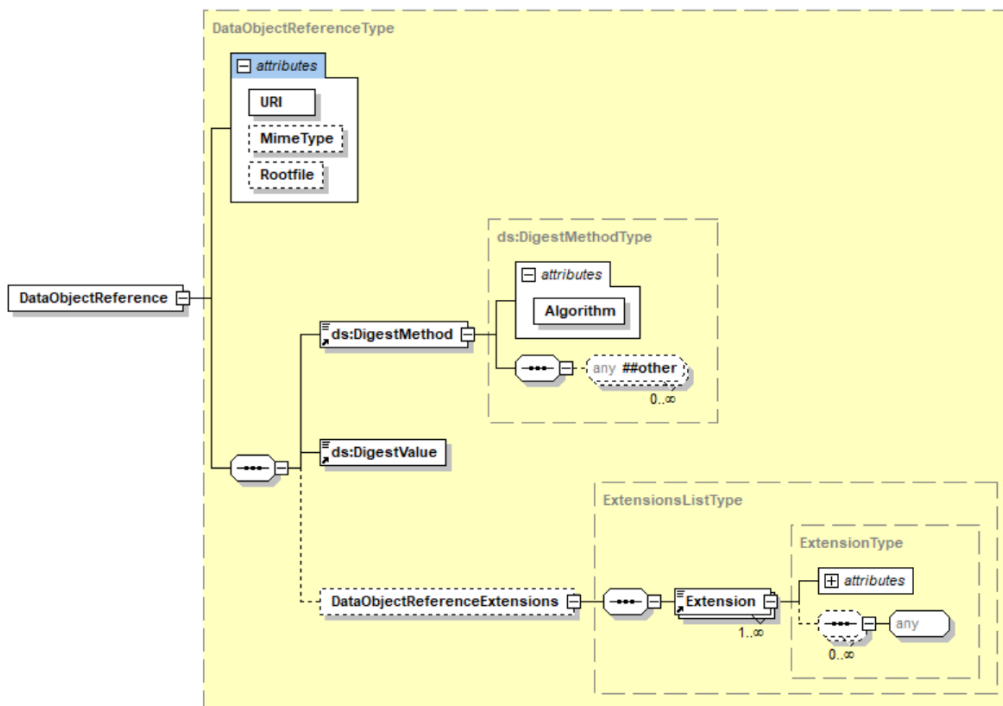


Abbildung 3: Definition von *asic:DataObjectReference* gem. ASiC-Manifest (vgl. [ETSI EN 319 162-1])

Basierend auf dem Element `asic:DataObjectReference` enthält das folgende Codefragment ein Beispiel einer in `xaip:dataObjectsSection` eingebetteten Verlinkung eines LXAIP. Eine Textdatei (hier DO-03.txt) wird durch den Inhalt des Elements `xaip:dataObject` mit dem Attribut `dataObjectID="DO-03"` referenziert. Die Attribut `URI` des Elements `asic:DataObjectReference` verweist auf die o. g. Datei mit Inhaltsdaten und die Elemente `ds:DigestMethod` und `ds:DigestValue` beinhalten den obligatorischen Integritätsschutz in Form einer kryptographischen Prüfsumme. .

```
[...]
<xaip:dataObjectsSection>
  <xaip:dataObject dataObjectID="DO-03">
    <xaip:xmlData>
      <asic:DataObjectReference URI="DO-03.txt">
        <ds:DigestMethod Algorithm="http://www.w3.org/2001/04/xmlenc#sha256"/>
        <ds:DigestValue>4763AA261F8EF67E[...]E649FFD73BC5D73C2E</ds:DigestValue>
      </xaip:xmlData>
    </xaip:dataObject>
  </xaip:dataObjectsSection>
```

[...]

3.2.2 Verlinkung der Metadatenobjekte in LXAIP

Die einzelnen Metadatenobjekte werden im XAIP innerhalb des Elements `xaip:metaDataSection` abgelegt. Hierbei stehen zwei mögliche Ablageoptionen zur Verfügung (vgl. Kap. 3.1.3):

- Ablage als Binärdaten innerhalb des Elements `xaip:binaryMetaData`,
- Ablage als beliebige XML-Daten innerhalb des Elements `xaip:xmlMetaData`.

Die Ablage der ausgewiesenen Metadaten als ein referenziertes Metadatenobjekt innerhalb von LXAIP erfolgt analog wie im Falle von Datenobjekte (vgl. Kap. 3.2.1). Sollte ein entsprechendes Element `asic:DataObjectReference` im Element `xaip:xmlMetaData` abgelegt werden, so handelt sich dabei nicht um die tatsächlichen Metadaten, sonder um eine Referenz, unter welcher die tatsächlichen Metadaten, z. B. mit Hilfe des Upload-Download-Moduls, ermittelt werden können.

Nachfolgendes Codefragment enthält beispielhafte Darstellung eines als Referenz (im Sinne von LXAIP) abgelegten Metadatenobjekts – eine XML-Datei mit dem Namen „MTDO-01.xml“.

```
[...]
<xaip:metaDataSection>
  <xaip:metaDataObject metaDataID="MTDO-01" relatedObjects="DO-03">
    <xaip:xmlMetaData>
      <asic:DataObjectReference URI="MTDO-01.xml">
        <ds:DigestMethod Algorithm="http://www.w3.org/2001/04/xmlenc#sha256"/>
        <ds:DigestValue>F456AA261F8EF67E[...]E649FFD73BC5D73EE4</ds:DigestValue>
      </xaip:xmlMetaData>
    </xaip:metaDataObject>
  </xaip:metaDataSection>
```

[...]

3.2.3 Verlinkung der Credentialobjekte in LXAIP

Die einzelnen Credentialobjekte werden im XAIP innerhalb des Elements `xaip:credentialsSection` abgelegt. Hierbei stehen diverse Ablageoptionen zur Verfügung (vgl. Kap. 3.1.5):

- Ablage in einem dedizierten Format: `dss:SignatureObject`, `xaip:certificateValues`, `xaip:revocationValues`, `xaip:evidenceRecord` und `vr:VerificationReport`, oder
- Ablage als beliebige XML-Daten innerhalb des Elements `xaip:other`, falls nicht beweisrelevanten Daten (Signaturen, Siegel, Zeitstempel, Zertifikate, Sperrlisten, OCSP-Responses, Signatur- bzw. Siegel- bzw. Zeitstempelprüfinformationen etc.) oder technischen Beweisdaten (Evidence Records).

Die Ablage der ausgewiesenen Credentialdaten als ein referenziertes Credentialobjekt innerhalb von LXAIP erfolgt analog wie im Falle von Datenobjekte (vgl. Kap. 3.2.1). Sollte ein entsprechendes Element `asic:DataObjectReference` im Element `xaip:other` abgelegt werden, so handelt sich dabei nicht um die tatsächlichen Credentialdaten, sondern um eine Referenz, unter welcher die tatsächlichen Credentialdaten, z. B. mit Hilfe des Upload-Download-Moduls, ermittelt werden können.

Nachfolgendes Codefragment enthält beispielhafte Darstellung eines als Referenz (im Sinne von LXAIP) abgelegten Credentialobjekts – eine abgesetzte CAdES mit dem Namen „DO-03.txt.p7s“.

```
[...]
<xaip:credentialsSection>
  <xaip:credential credentialID="CR-01" relatedObjects="DO-03">
    <xaip:other>
      <asic:DataObjectReference URI="./DO-03.txt.p7s">
        <ds:DigestMethod Algorithm="http://www.w3.org/2001/04/xmlenc#sha256"/>
        <ds:DigestValue>C356AA261F8EF67E[...]E649FFD73BC5D7AB54</ds:DigestValue>
      </xaip:other>
    </xaip:credential>
  </xaip:credentialsSection>
```

[...]

3.2.4 Das Delta-LXAIP-Element

Die im Abschnitt 3.1.6 ausgeführte Regeln für ein Delta-XAIP-Element gelten gleichwohl für ein Delta-LXAIP-Element, wenn die Regeln für den Aufbau eines LXAIP aus den Abschnitten 3.2.1, 3.2.2 und 3.2.3 entsprechend Ihre Anwendung finden.

3.2.5 Formaterkennung und -validierung eines LXAIP

Die Validierung eines LXAIP samt den darin enthaltenen beweisrelevanten Daten erfolgt grundsätzlich gem. dem im Kapitel 7.4.6 des Hauptdokuments (vgl. [TR-ESOR]) beschriebenen Ablauf. Es müssen im Gegensatz zur Validierung eines physischen XAIP jedoch einige zusätzliche Schritte erfolgen:

Im ersten Schritt muss sichergestellt werden, dass es sich um ein LXAIP handelt. Für diesen Zweck wird der Inhalt jedes einzelnen Elements `xaip:dataObject` aus dem Element `xaip:dataObjectsSection`, `xaip:metaDataObject` aus dem Element `xaip:metaDataSection` und `xaip:credential` aus dem Element `xaip:credentialsSection` überprüft.

Wenn

- das Element `xaip:dataObject` ein Element `xaip:xmlData` beinhaltet, das wiederum ein Element `asic:DataObjectReference` beinhaltet, oder
- das Element `xaip:metaDataObject` ein Element `xaip:xmlMetaData` beinhaltet, das wiederum ein Element `asic:DataObjectReference` beinhaltet, oder
- das Element `xaip:credential` ein Element `xaip:other` beinhaltet, das wiederum ein Element `asic:DataObjectReference` beinhaltet,

→ so handelt sich dabei um ein LXAIP.

Im nächsten Schritt muss sichergestellt werden, dass alle im LXAIP enthaltenen Referenzen auf die zugehörigen Datenobjekte erfolgreich aufgelöst werden können. Es folgt die Überprüfung der obligatorischen korrespondierenden Prüfsummen (vgl. die Elemente `ds:DigestMethod` und `ds:DigestValue` aus dem Element `asic:DataObjectReference`). Bei positivem Ausgang der o. g. Prüfungen kann die LXAIP analog zu einem XAIP validiert werden, wobei die dem `DataObjectReference`-Element beigelegte Prüfsumme im `DigestValue`-Element ausschließlich der Transportabsicherung (z. B. beim initialen `SubmissionRequest`) dient und die für die Berechnung oder Verifikation des Hasbaums benötigte Hashwerte ausschließlich über den Netto-Inhalt der durch das `URI`-Attribut des `DataObjectReference`-Elements referenzierten Objekte berechnet werden muss.

3.2.6 LXAIP-Referenzen

Der Umgang mit den innerhalb von einem LXAIP verwendeten Referenzen auf die externen Datenobjekte, Metadatenobjekte und Credentials wird nachfolgend anhand einiger Beispiele erläutert.

HINWEIS 8

Die nachfolgend aufgeführten Beispiellansätze weisen ausdrücklich einen informativen Charakter auf. Die konkrete Umsetzung kann herstellerabhängig variieren.

Zugriff auf die lokal verfügbaren Datenobjekte, Metadatenobjekte und Credentials:

- durch die Eingabe von zu der Ablageort des korrespondierenden LXAIP relativen Pfade, z. B. „./lxaip/do-01.txt“,
- durch die Eingabe von absoluten Dateipfaden:
 - als einfache Pfadeingabe, z. B.: „/home/user1/data/lxaip/mdo-01.xml“, oder
 - als eine URL, z. B. „file:///home/user1/data/lxaip/do-01.txt.p7s“.

Zugriff auf die entfernt verfügbaren Datenobjekte, Metadaten, Credentials³¹:

- durch die Eingabe einer URL, z. B.:
 - „https://server01.com/data/lxaip?AOID=AOID-123-234-434-455&ID=DO-01“, oder
 - „fts://server01.com/data/lxaip/AOID-123-234-434-455_do-01.txt.p7s“.

3.3 ASiC-AIP

Der Associated Signature Container (ASiC) ist in [ETSI EN 319 162-1] spezifiziert und gilt in der Ausprägung ASiC Baseline Profile gem. eIDAS-VO als einer der zu unterstützenden Signaturformate (vgl. [eIDAS-VO], [2015/1506/EU]). Es handelt sich dabei um einen Binärcontainer, dessen Inhalte mit Hilfe des ZIP-Algorithmus (vgl. [ISO21320]) platzsparend wie reversibel komprimiert sind.

Die erweiterte Variante von ASiC, ASiC-E³², erlaubt grundsätzlich die Ablage einer beliebigen Anzahl von digitalen Signaturen oder Zeitstempeln bzw. Evidence Records (*time assertions*-Objekte) sowie von

³¹ Vgl. hierzu auch A7.4-8 in [TR-ESOR].

³² Je nach Anzahl der aufzubewahren Signaturen wird dabei zwischen zwei Varianten des ASiC unterschieden:

- Einfacher Container – ASiC-S,

zugehörigen Datenobjekten direkt im Wurzelverzeichnis der Container-Datei. Die digitalen Signaturen oder Zeitstempel bzw. Evidence Records (*time assertion*-Objekte) sowie (bei Bedarf) weitere Anwendung-relevante Dateien werden in einem dedizierten Verzeichnis *META-INF* abgelegt. Datenobjekte dagegen sind in der Regel direkt im Wurzelverzeichnis platziert. Die logische Verbindung zwischen einem Signatur- oder „*time assertion*“-Objekt und den zugehörigen Datenobjekten wird explizit in einer dedizierten Manifest-Datei (ASiCManifest.xml) innerhalb des *META-INF*-Verzeichnisses abgelegt (vgl. Kap. 4.4.2 [ETSI EN 319 162-1] und Abbildung 4).

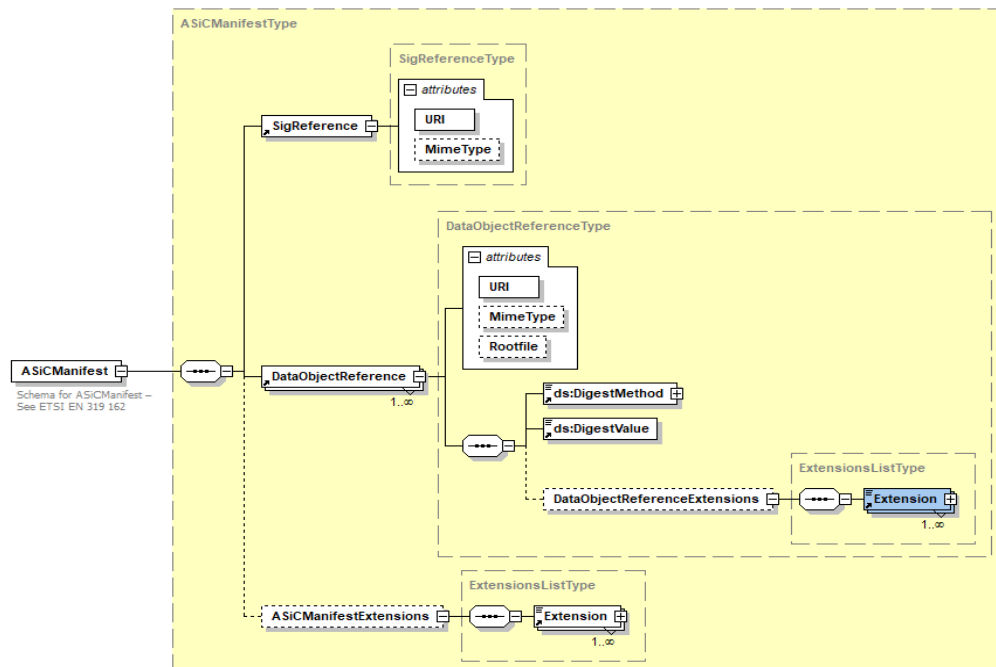


Abbildung 4: Aufbau des Elements *asic:AsiCManifest* (vgl. [ETSI EN 319 162-1]).

Angelegt an die Bedürfnisse eines AIP und unter Berücksichtigung der Regeln zum Aufbau eines ASiC-E, wurde folgende Definition eines ASiC-AIP erarbeitet. Generell werden zwei Ausprägungen eines ASiC-AIP, in Abhängigkeit von der Art, wie die Inhaltsdaten behandelt werden, unterschieden:

1. ASiC-AIP mit einem logischen XAIP mit den abgesetzten Inhaltsdaten,
2. ASiC-AIP mit einem gewöhnlichen XAIP mit den darin eingebetteten Inhaltsdaten.

Einen zusätzlichen Aspekt stellt die Berücksichtigung der Protected Pointers dar, die auf einen Teil des XAIPs verweisen, das im XML-Format vorliegt, z.B. das Element *xaip:VersionManifest* etc.

Das zugrundeliegende Konzept besteht darin, alle in einer Version eines XAIP aufgezählten Protected Pointers entsprechend in einem ASiC-Manifest abzubilden, wobei dieser um notwendige Transformationen- und Kanonisierungseingaben vervollständigt wird.

Gem. [TR-ESOR-F] enthält das Element */xaip:XAIP/xaip:packageHeader/ds:CanonicalizationMethod* (weiter als globale Kanonisierungsmethode bezeichnet) Eingaben, welche Kanonisierungsmethode global auf das XAIP angewandt werden muss, bevor dessen Inhalte für die Erstellung bzw. Prüfung der Beweisdaten verhasht werden. Das Fehlen dieses optionalen Elements impliziert die Anwendung des Standard-Algorithmus (C14N - Canonical XML [C14N]) (vgl. [TR-ESOR-F], Kap. 3.2). Diese Information muss entsprechend auch in einem ASiC-AIP-Manifest enthalten sein. Für die Ablage der globalen Kanonisierungsmethode wird eine Instanz des Elements *asic:ASiCManifestExtensions* (eine durch ASiC

- Erweiterter Container – ASiC-E.

Da ASiC-S über eine Einschränkung verfügt, die die Ablage von nur einem Signatur- oder „*time assertion*“-Objekt erlaubt, ist dieses Format für weitere Untersuchungen zwecks Nutzung als AIP-Format zur Aufnahme von mehreren Dateien in TR-ESOR ungeeignet.

definierte Methode für die Ablage notwendigen Erweiterungen) verwendet. Nachfolgende Abbildung illustriert die beschriebene Vorgehensweise (vgl. Abbildung 5)³³.

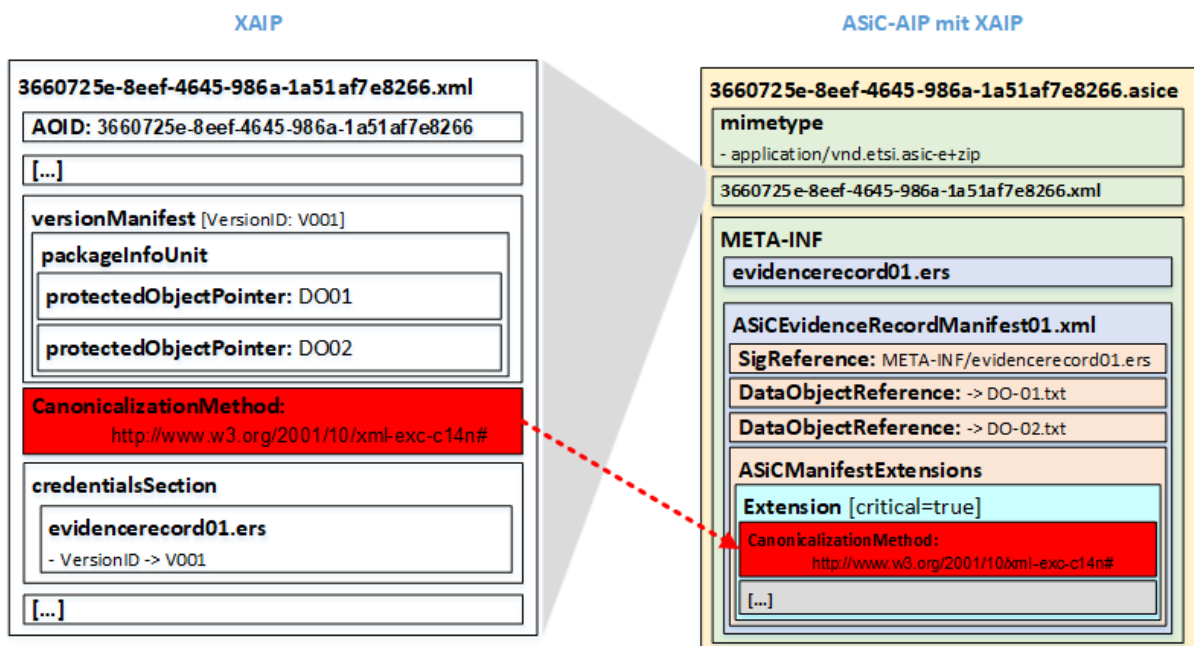


Abbildung 5: ASiC-AIP-Manifest, Definition der globalen Kanonisierungsmethode.

Die rechte Seite der Abbildung 5 beinhaltet ein beispielhaftes ASiC-AIP, das wiederum ein beispielhaftes XAIP beinhaltet (dargestellt auf der linken Seite der Abbildung 5). Die innerhalb des XAIP dargestellte globale Kanonisierungsmethode (links in rot) wird als eine kritische³⁴ Erweiterung in das ASiC-Manifest (rechts in rot) übernommen.

Eine weitere Erweiterung, die für die Umsetzung von ASiC-AIP benötigt wird, beschreibt die AOID und die VersionID. Für diesen Zweck wurde das Element `pres:ContainerID` gem. [ETSI TS 119 512] verwendet (vgl. nachfolgende Abbildung 6).

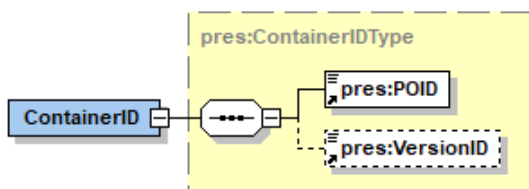


Abbildung 6: Aufbau des Elements `pres:ContainerID` (vgl. [ETSI TS 119 512]).

Es ergibt sich dabei folgende Belegung der einzelnen Elemente:

- `pres:POID` – entspricht dem Element `xaip:AOID`
- `pres:VersionID` – entspricht dem Attribut `VersionID` des entsprechenden Elements

³³ Die globale Kanonisierungsmethode ist nur dann zwingend notwendig, wenn XML-Teile von einem (L)XAIP durch einen ER geschützt werden. Nur in solchen Fällen ist die Erweiterung als kritisch abzulegen. In allen anderen Fällen ist die Erweiterung optional und kann auch gänzlich ausgelassen werden.

³⁴ Der Wert des Attributs `critical` besagt, ob die Erweiterung bei der Prüfung verstanden werden muss. Sollten keine Teile des XAIP geschützt werden, so ist diese Erweiterung nicht zwingend zu verstehen und somit auch nicht kritisch. Im Falle, dass das ASiC-Manifest (somit auch der Evidence Record) einige Teil des XAIP referenziert (z. B. `asic:VersionManifest`), so muss diese Erweiterung verstanden werden und somit ist sie als kritisch zu markieren.

³⁵ Es gilt: `xmlns:pres="http://uri.etsi.org/019512/v1.0#"`

xaip:versionManifest.

Letztendlich kann mit Hilfe des Elements `ds:Transform`³⁶ folgende Erweiterung definiert werden, welche die notwendige Transformation im Sinne der Base64-Dekodierung vorschreibt.

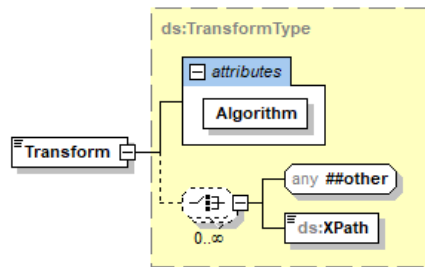


Abbildung 7: Aufbau des Elements `ds:Transform`.

Die Bedeutung der einzelnen Unterelemente und Attribute des Elements `ds:Transform` ist dem Kapitel 4.3.3.4 [XMLDSIG] zu entnehmen. Im Sinne dieser Spezifikation wird auf folgenden möglichen Wert des Attributs `Algorithm` hingewiesen (vgl. [XMLDSIG], Kap. 6.6.2):

<http://www.w3.org/2000/09/xmlsig#base64> – die Daten müssen Base64 dekodiert werden (vgl. [RFC4648]).

3.3.1 ASiC-AIP mit einem logischen XAIP

Im Falle eines logischen XAIP (LXAIP, vgl. Kap. 3.2) sind die Inhaltsdaten bereits aus dem XAIP extrahiert worden und können somit direkt aus dem ASiC-AIP-Manifest referenziert werden. Nachfolgende Abbildung zeigt das Zusammenspiel eines LXAIP und des korrespondierenden ASiC-AIP-Manifests³⁷.

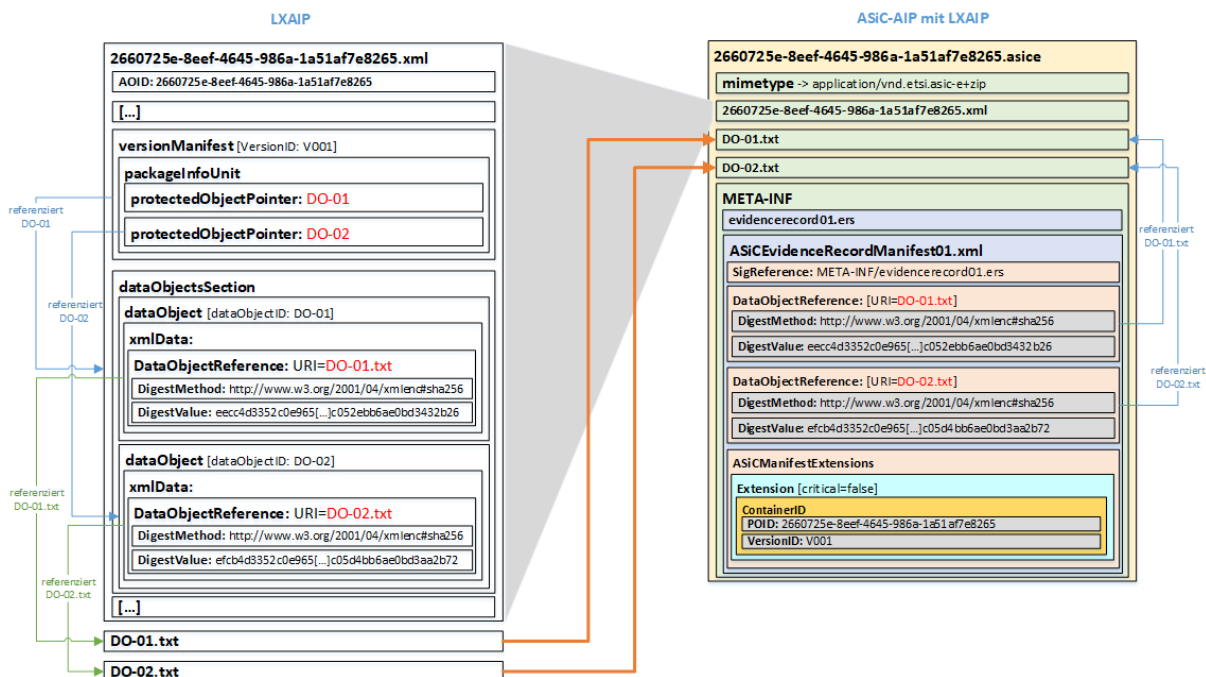


Abbildung 8: Beispiel für ein ASiC-AIP mit einem LXAIP.

³⁶ Um die größtmögliche Kompatibilität mit den zukünftigen EU-Normen zum Thema Beweiswerterhaltung (z. B. [TS-119512]) zu erreichen, wurde an dieser Stelle auf die Verwendung des aus dem XAIP-Umfeld stammenden Elements `xaip:transformInfo` zu Gunsten des Elements `ds:Transform` (vgl. [XMLDSIG]) verzichtet.

³⁷ In diesem Beispiel ist die Erweiterung zur Definition der globalen Kanonisierungsmethode nicht notwendig, daher wurde sie ausgelassen.

Auf der rechten Seite der Abbildung ist ein beispielhaftes ASiC-AIP dargestellt (2660725e-8eef-4645-986a-1a51af7e8265.asice), das wiederum ein logisches XAIP (2660725e-8eef-4645-986a-1a51af7e8265.xml) und zwei dazugehörige Datenobjekte (DO-01.txt und DO-02.txt) beinhaltet (siehe die linke Seite der Abbildung 8). Innerhalb des ASiC-AIP werden sowohl das LXAIP als auch die Inhaltsdaten in dem Wurzelverzeichnis abgelegt. Weiterhin beinhaltet das o. g. ASiC-AIP einen über die Version V001 des o. g. LXAIP berechneten Evidence Record (META-INF/evidencerecord01.ers) sowie das zugehörige ASiC-Manifest-Datei (META-INF/ASiCEvidenceRecordManifest01.xml). Die Integrität der innerhalb des ASiC-Manifests referenzierten Inhaltsdaten wird durch die jeweilige Eingabe einer Prüfsumme (Elemente `DigestMethod` und `DigestValue` innerhalb des Elements `DataObjectReference`) zusätzlich geschützt. Da die Inhaltsdaten bereits außerhalb LXAIP liegen, sind diese nicht transformiert und somit ist die Anwendung der Base64-Transformation (vgl. Kap. 3.3) nicht notwendig. Die übrigen Erweiterungen zur genauer Spezifikation des gemeinten Containers (`AOID` und `VersionID` innerhalb des Elements `pres:ContainerID`) sowie die globale Kanonisierungsmethode für LXAIP müssen in diesem Beispiel nicht zwingend verstanden werden und sind somit als unkritisch eingestuft worden³⁸.

3.3.2 ASiC-AIP mit einem gewöhnlichen XAIP

In einem gewöhnlichen XAIP sind die Inhaltsdaten vollständig in das XAIP eingebettet und werden daher aus dem ASiC-AIP-Manifest direkt im besagten XAIP referenziert. Nachfolgende Abbildung stellt ein Beispiel eines XAIP, in dem die Version V001 auf zwei eingebettete Inhaltsdatenobjekte DO-01 und DO-02 verweist, sowie ein zugehöriges ASiC-AIP-Manifest dar³⁹.

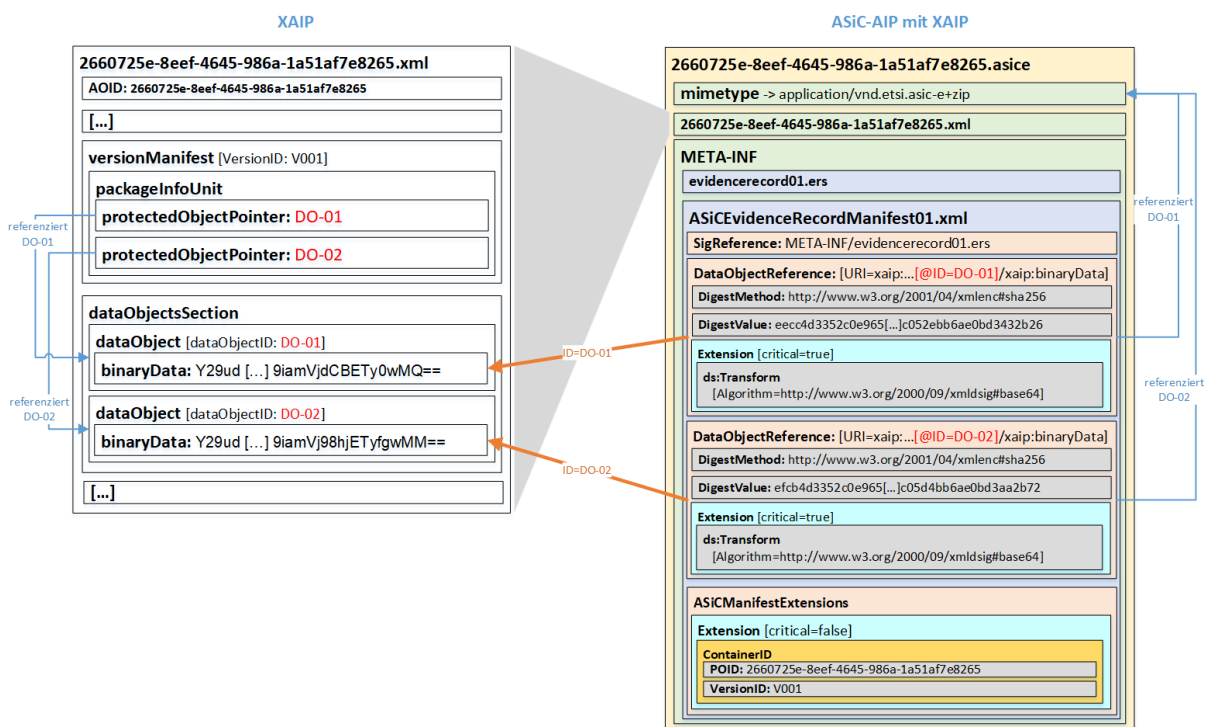


Abbildung 9: Beispiel für ein ASiC-AIP-Manifest mit einem gewöhnlichen XAIP.

(A3.3-1) Das Referenzieren der Daten innerhalb des XAIP erfolgt durch eine URI und muss außer der Referenz auf die XAIP-Datei selbst auch den XPath-Ausdruck beinhalten, der den Weg zu den Daten innerhalb des XAIP beschreibt.

³⁸ Das auf dieser Weise aufgebaute Beispiel kann ohne Weiteres durch einen gem. [ETSI EN 319 162-1] aufgebauten out-of-the-box-Validator erfolgreich validiert werden.

³⁹ Die Erweiterung zur Definition der globalen Kanonisierungsmethode wurde als nicht notwendig ausgelassen.

Die folgende URI:

- `xaip:3660725e-8eef-4645-986a-1a51af7e8266.xml?q="//[@dataObjectID='DO-01']/esor:binaryData"` (siehe Kap. **Fehler! Verweisquelle konnte nicht gefunden werden.**).

referenziert beispielsweise den Inhalt des Elements `xaip:dataObject` (die binären Daten), dessen Attribut `dataObjectID` den Wert „DO-01“ besitzt.

3.3.3 Ein initiales ASiC-AIP

Gemäß [EN 319 162-1] muss ein gültiges ASiC-Objekt zwingend mindestens ein kryptografisches Artefakt beinhalten, z. B. CAdES, XAdES, Zeitstempel oder Evidence Record. Da es sich um ein initiales ASiC-AIP handelt, ist der zugehörige Evidence Record noch nicht vorhanden. Es muss ein alternativer Mechanismus angewandt werden, um die o.g. Bedingung zu erfüllen. In der Abbildung 10 ist ein Beispiel eines gültigen, initialen ASiC-AIPs mit einem LXAIP⁴⁰ dargestellt werden.

In dem Falle wurde im ASiC-AIP zusätzlich zu dem LXAIP (hier `2660725e-8eef-4645-986a-1a51af7e8265-v1.xml`) und den zu LXAIP zugehörigen Datenobjekten (hier das Datenobjekt `DO-01.txt`), eine sog. technische Signatur (`signature.p7s`, eine CAdES-B) und zugehörige `ASiCManifest.xml`-Datei abgelegt. Die Manifest-Datei referenziert dabei das obligatorische LXAIP (vgl. `asic:DataObjectReference`-Element im Manifest) und die technische Signatur schützt die `ASiCManifest`-Datei selbst (vgl. `asic:SigReference`-Element im Manifest). Auf dieser Weise aufgebautes ASiC-Objekt erfüllt die Vorgaben aus [EN 319 162-1] vollumfänglich und stellt ein valides ASiC im Sinne dieser Norm bereit.

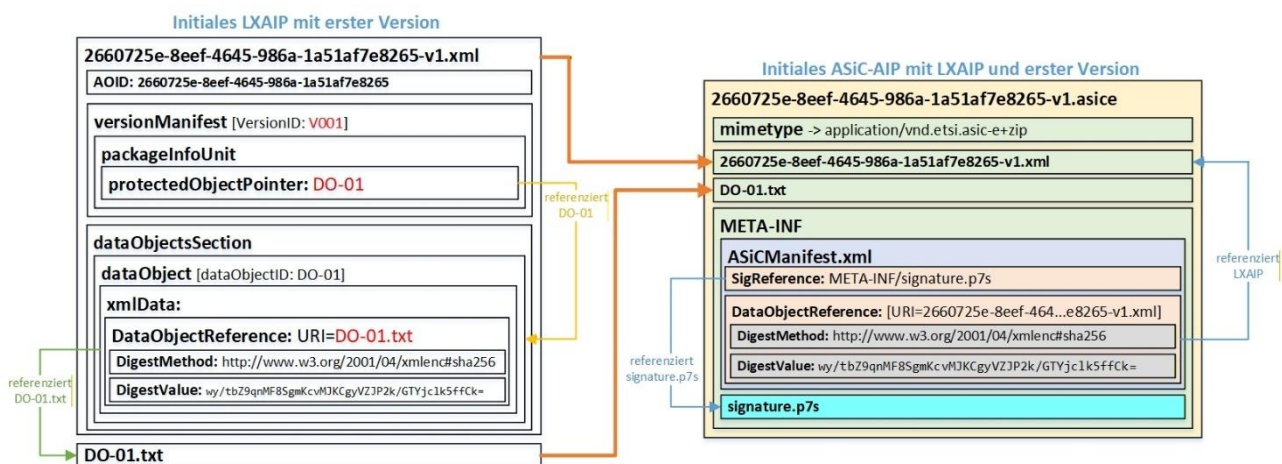


Abbildung 10: Ein beispielhaftes, initiales ASiC-AIP mit einem LXAIP und einer technischen Signatur.

HINWEIS 9

Ein initiales Delta-ASiC-AIP muss analog mit einer technischen Signatur ausgestattet werden. Für mehr Informationen und ein Beispiel vgl. Kap. 0.

HINWEIS 10

Die technische Signatur wird bis zum Zeitpunkt der Erstellung des zugehörigen Evidence Records benötigt und kann dann entfernt werden.

⁴⁰ Im Falle eines XAIP anstatt von LXAIP wird analog vorgegangen.

3.3.4 ASiC-AIP und Verlinkung der XML-Daten

Die Verlinkung der XML-Daten kann sowohl im Falle des gewöhnlichen als auch des logischen XAIP vorkommen. Es können einerseits die Inhaltsdaten, die in dem Element `xaip:xmlData` abgelegt werden, sowie andererseits auch beliebige Teile des XAIP selbst mit einem Protected Pointer (innerhalb XAIP) geschützt werden.

Nachfolgende Abbildung stellt ein Beispiel dar, in dem die ASiC-AIP-Instanz aus dem Kap. 3.3.1 zusätzlich zu den zwei Inhaltsobjekten noch das gesamte Element `xaip:versionManifest` der Version V001 als dritten Protected Pointer schützt⁴¹, das mit folgenden URI referenziert werden kann:

- `xaip:2660725e-8eef-4645-986a-1a51af7e8265-1a51af7e8265.xml?q="//xaip:versionManifest[@VersionID='V001']"` (siehe Kap. 3.3.6)

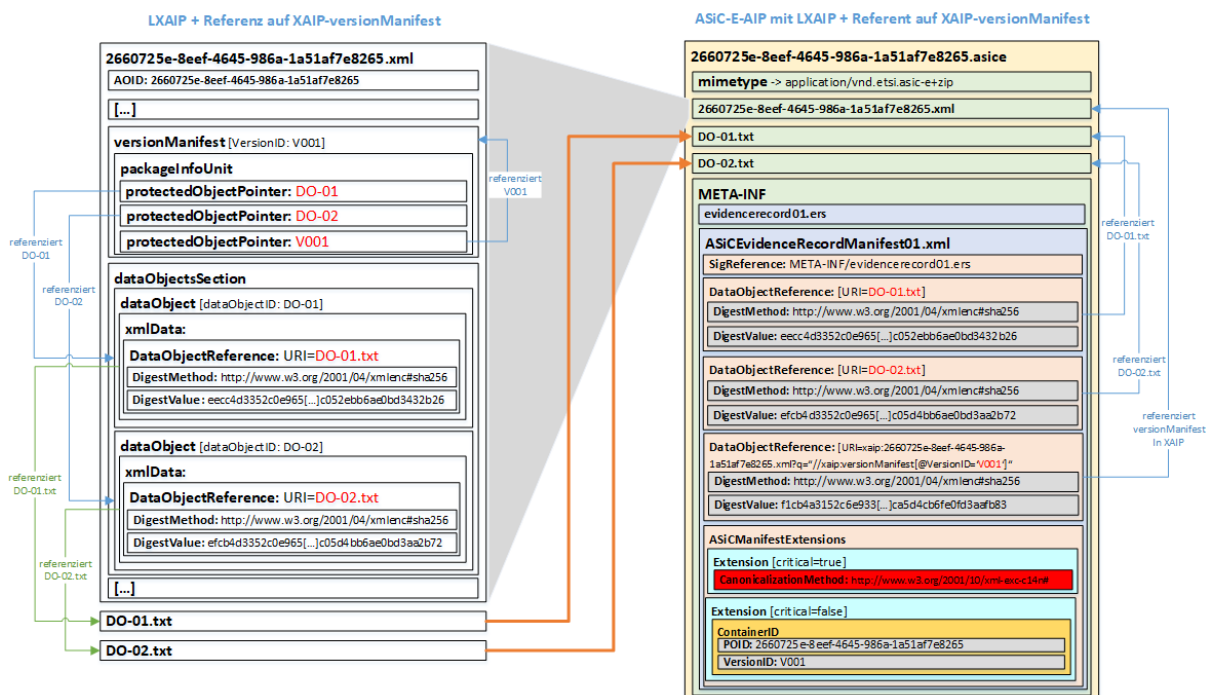


Abbildung 11: Beispiel für ein ASiC-AIP mit einem LXAIP mit einer Referenz auf das Element `xaip:versionManifest` in LXAIP.

Ein weiterer Unterschied zu der Darstellung aus dem Kap. 3.3.1 besteht in der Kritikalität der Erweiterung zur Spezifikation der globalen Kanonisierungsmethode. Diese Erweiterung ist nämlich in diesem Beispiel als kritisch eingestuft worden und darf somit bei der Verarbeitung des vorliegenden ASiC-AIP nicht ignoriert werden.

3.3.5 ASiC-AIP - Umgang mit Versionen

Im Falle von (L)XAIP werden die Versionen des Archivdatenobjekts mit Hilfe des Attributs `VersionID` des Elements `xaip:versionManifest` innerhalb des Elements `xaip:packageHeader` abgebildet (vgl. Kap. 3.1.1). Dieses Verhalten kann auf das ASiC-AIP projiziert werden, indem jeder der im (L)XAIP vorhandenen Versionen ein korrespondierendes ASiC-Manifest zugeordnet wird.

⁴¹ Die Erweiterung zur Definition der globalen Kanonisierungsmethode ist zwingend erforderlich, daher wurde diese mit der Kritikalität gesetzt auf „true“ in das Manifest aufgenommen.

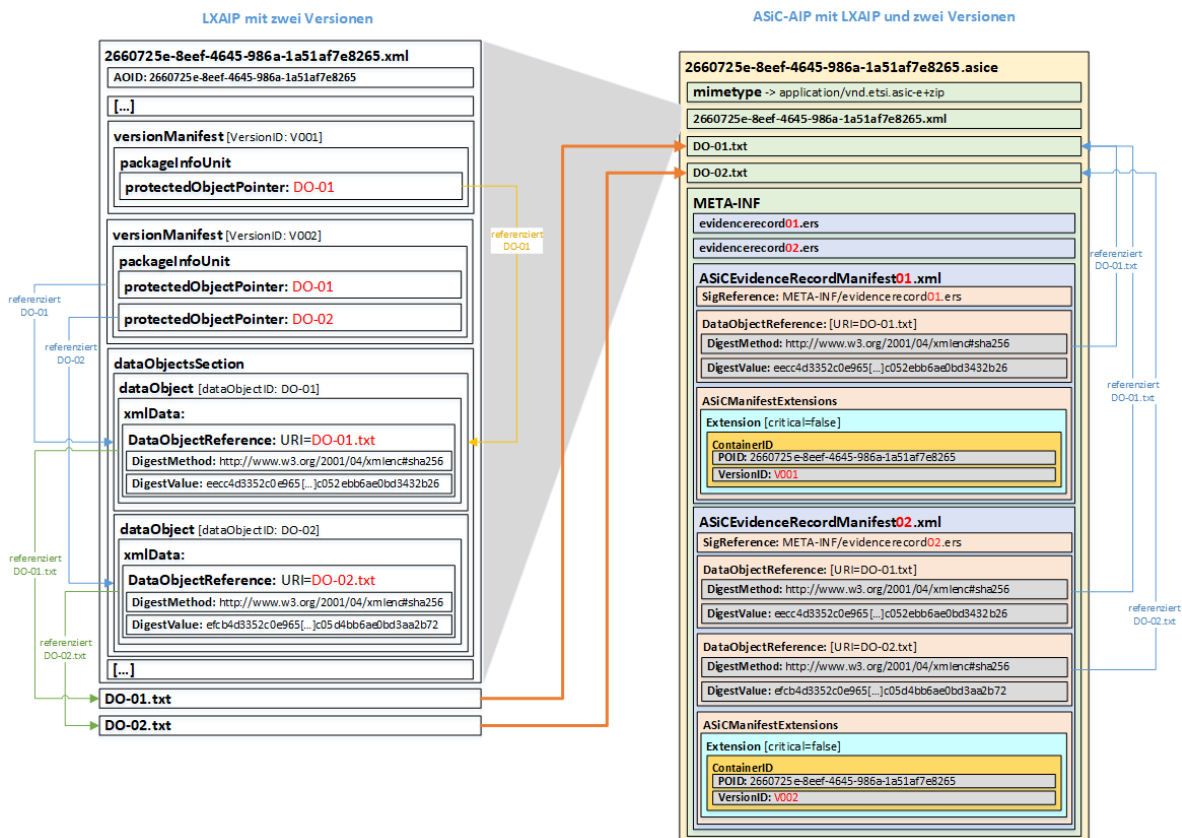


Abbildung 12: Ein Beispiel für ein LXAIP mit zwei Versionen und ein korrespondierendes ASiC-AIP.

Abbildung 12 stellt ein LXAIP dar, das über zwei Versionen verfügt. Die erste Version schützt ein Datenobjekt (DO-01.txt). Die zweite Version dagegen schützt zwei Datenobjekte (DO-01.txt und DO-02.txt). Entsprechend sind im korrespondierenden ASiC-AIP-Container zwei ASiC-Manifest-Dateien (ASiCEvidenceRecordManifest01.xml und ASiCEvidenceRecordManifest02.xml) enthalten, die jeweils entsprechend die Versionen V001 und V002 innerhalb des Containers abbilden.

3.3.6 Referenzen in ASiC-AIP

Die innerhalb eines ASiC-AIP abgelegten Datenobjekte, Metadatenobjekte und Credentials (weiterhin in diesem Kapitel zusammengefasst als Datenobjekte genannt), sowie ggf. Teile des zugehörigen (L)XAIP müssen aus den korrespondierenden ASiC-Manifest-Dateien entsprechend referenziert werden.

Die Aufbauregel für solche Referenzen sind im nachfolgenden Kapitel, abhängig vom zugrundeliegenden Anwendungsfall, beschrieben.

3.3.6.1 Anwendungsfall LXAIP

Wird bereits LXAIP verwendet (vgl. Kap. 3.3.1), so sind die referenzierten Datenobjekte bereits aus dem LXAIP ausgelöst und innerhalb des ASiC-Datenstruktur abgelegt worden. Diese werden gem. [ETSI EN 319 162-1] unter der Eingabe des zugehörigen Dateinamens aus dem korrespondierenden ASiC-Manifest-Datei referenziert, z. B.:

- URI="DO-01.txt" – referenziert eine Textdatei.

3.3.6.2 Anwendungsfall XAIP

Wird ein XAIP verwendet (vgl. Kap. 3.3.2), so sind die referenzierten Datenobjekte inline im XAIP enthalten und das XAIP selbst ist innerhalb der ASiC-Datenstruktur abgelegt worden. Um die Zugriffe aus der

korrespondierenden ASiC-Manifest-Datei zu steuern, wurde ein dediziertes Verfahren konzipiert, das mit Hilfe vom folgenden URI-Muster initiiert wird:

- `xaip:<xaip-file-name>?q=<xpath-expression>`.

Den einzelnen Teilen des Musters wird folgende Bedeutung zugeschrieben:

- **xaip** – signalisiert des zu verwendeten Musterzugriffstyps,
- **xaip-file-name** – der Dateiname des referenzierten XAIP, das im ASiC-Container abgelegt worden ist,
 - z. B. „2660725e-8eef-4645-986a-1a51af7e8265.xml“
- **xpath-expression** – der XPath-Ausdruck, der innerhalb des referenzierten XAIP angewandt werden muss, um das gewünschte Datenobjekt zu extrahieren,
 - z. B. „//*[@dataObjectID='DO-01']/esor:binaryData“.

3.3.6.3 Anwendungsfall XML-Teile in (L)XAIP

Wird direkt einen Teil aus dem (L)XAIP referenziert (vgl. Kap. 3.3.3), so erfolgt der Zugriff aus dem korrespondierenden ASiC-Manifest analog wie im Kap. 3.3.6.2 beschreiben.

3.3.7 Das Delta-ASiC-AIP

Das Delta-ASiC-AIP, analog zum ASiC-AIP selbst (vgl. Kap. 3.3.1 und 3.3.2), setzt entsprechend auf Delta-XAIP (vgl. Kap. 3.1.6) oder respektive Delta-LXAIP (vgl. Kap. 3.2.4) auf, die bereits hinreichend das Update-Verhalten beschreiben. Der gleiche Mechanismus wird im Falle des korrespondierenden ASiC-AIPs angewandt. Eine exemplarische Darstellung eines Delta-ASiC-AIPs mit einem Delta-LXAIP ist der Abbildung 13 zu entnehmen.

Es wird eine Version des ASiC-AIP mit der `versionID` gleich `v002` abgebildet, die zusätzlich zu bereits in der Version mit der `versionID` gleich `v001` existierenden Datenobjekt `DO-01.txt` ein Datenobjekt `DO-02.txt` beinhaltet. Auch hier in der gewohnten Manner eines Delta-Objektes müssen nur die neu dazugekommenen Datenobjekte physikalisch im ASiC-AIP enthalten sein. Die bereits aus den vorherigen Versionen des ASiC-AIPs referenzierte Datenobjekte sind bereits dem TR-ESOR-System bekannt und können via entsprechende `xaip:placeholder`-Elemente im `xaip:updateSection`-Element referenziert werden.

Analog zum initialen ASiC-AIP muss auch ein Delta-ASiC-AIP mit einer technischen Signatur ausgestattet werden (vgl. hierzu auch Kap. 3.3.3). In diesem Beispiel bilden die Dateien `ASiCManifest.xml`, die das obligatorische Delta-LXAIP (analog kann auch ein Delta-XAIP verwendet werden) referenziert, und `signature.p7s`, die eine CAdES-B über `ASiC-Manifest.xml` enthält, die besagte technische Signatur ab.

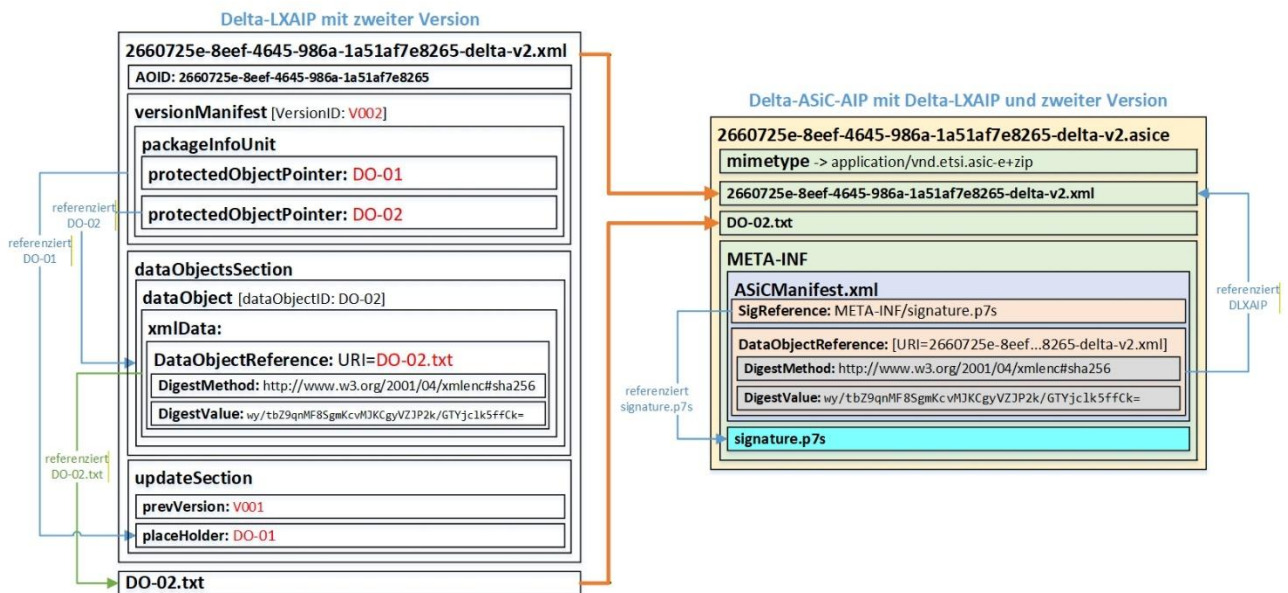


Abbildung 13: Exemplarische Darstellung eines Delta-ASiC-AIP mit einem Delta-LXAIP.

Nach einem erfolgreichen Aufruf der Funktion `archiveUpdate` (TR.S.4) bzw. `updatePOC` (TR.S.512) mit dem in der Abbildung 13 dargestellten Delta-ASiC-AIP und einer erfolgreichen Berechnung der zugehörigen Evidence Records (jeder Version wird ein separates Evidence Record zugeordnet) ergibt sich ein exemplarisches ASiC-AIP mit zwei Versionen und beiden Evidence Records, wie dargestellt in der Abbildung 14.

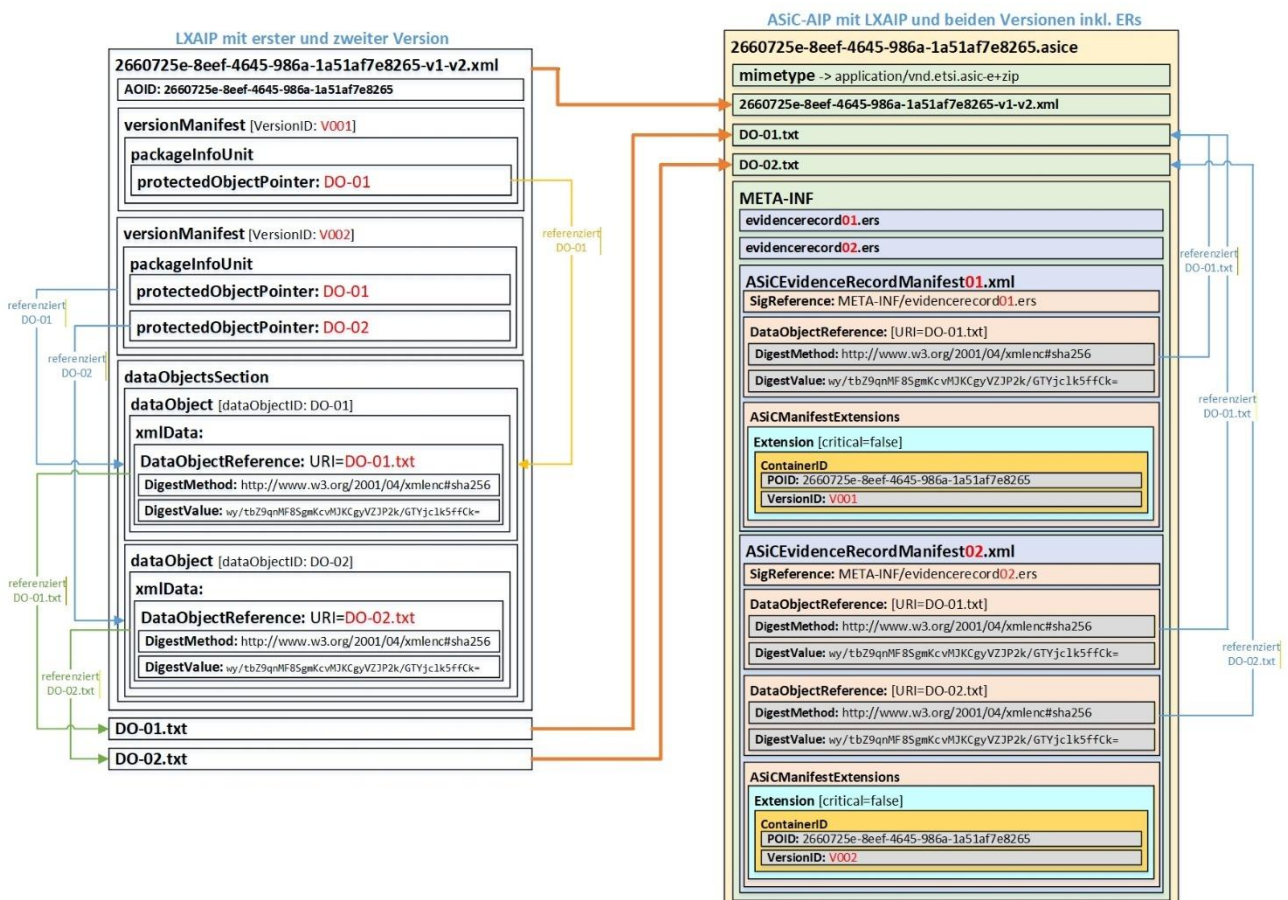


Abbildung 14: Exemplarische Darstellung eines ASiC-AIP mit 2 Versionen und beiden Evidence Records.

3.3.8 Formaterkennung und -validierung eines ASiC-AIP

Abhängig von der Art des verwendeten Validators, sind entsprechende Vorbedingungen zu erfüllen, damit ein ASiC-AIP erfolgreich validiert werden kann. Im weiteren Verlauf werden dafür grundsätzlich zwei Arten von Validatoren kurz betrachtet:

1. ein ASiC-E-konformer Validator (vgl. [ETSI EN 319 162-1]),
2. ein TR-ESOR-XAIP-konformes Validator (vgl. [TR-ESOR-F]).

3.3.8.1 Verwendung eines ASiC-E-konformen Validators

Damit ein ASiC-E-konformer Validator ein ASiC-AIP erfolgreich prüfen kann, müssen grundsätzlich folgende zusätzliche Voraussetzungen erfüllt werden:

1. der Validator muss die Erweiterung für die Definition der globalen Kanonisierungsmethode verstehen und diese entsprechend auf das enthaltene XAIP anwenden können (vgl. Kap. 3.3),
2. der Validator muss die, für den Umgang mit den Base64-kodierten Daten, definierten Transformationen verstehen und anwenden können (vgl. Kap. 3.3),
3. der Validator muss in der Lage sein, die entsprechenden Referenzen aus den ASiC-Manifest-Dateien (vgl. Kap. 3.3.6) aufzulösen,
4. der Validator soll die Bedeutung des im Rahmen einer Erweiterung angebrachtes Elementes `pres:ContainerID` (vgl. Kap. 3.3) erkennen und interpretieren können.

Alle XML-basierte Daten, die aus dem zugehörigen XAIP für die Prüfung des korrespondierenden Evidence Record extrahiert werden, müssen zuvor mit Hilfe der globalen Kanonisierungsmethode in entsprechendes Format überführt werden (vgl. Punkt 1 oben).

Sollten binäre Daten aus der zugehörigen XAIP extrahiert werden, so müssen diese entsprechend der definierten Base64-Transformation in die binäre Repräsentation überführt werden (vgl. Punkt 2 oben), bevor diese weiter interpretiert werden.

Es soll stets mit Hilfe der im Element `pres:ContainerID` der zugehörigen Erweiterung abgelegten Referenzen auf `AOID` (hier `POID`) und die Version, auf die Übereinstimmung mit den im zugehörigen XAIP geführten Daten geprüft werden (vgl. Punkt 4 oben).

Weiterer Verlauf der Verifikation ist [ETSI EN 319 162-1] zu entnehmen.

3.3.8.2 Verwendung eines TR-ESOR-konformes Validators

Ein TR-ESOR-konformer Validator prüft ein vorliegendes ASiC-AIP entweder indem ein ASiC-E-konformes Validator unter Einhaltung der Voraussetzungen aus dem Kap. 3.3.8.1 verwendet wird, oder indem das enthaltene (L)XAIP sowie zugehörige Evidence Record aus dem ASiC-AIP extrahiert werden und die Prüfregel für ein XAIP bzw. LXAIP angewandt werden (vgl. hierzu auch Kap. 7.4.6 im Hauptdokument).

4 Nutzdatenformate

Der folgende Abschnitt beschreibt elektronische Datenformate, die zum Zeitpunkt der Veröffentlichung dieser Technischen Richtlinie für die langfristige Aufbewahrung von Nutzdaten vornehmlich unter dem Gesichtspunkt der nachhaltigen Verfügbarkeit sowie maschinellen Lesbarkeit und Interpretierbarkeit empfohlen werden.⁴²

Zu den Nutzdaten gehören sowohl die eigentlichen Inhaltsdaten (Primärinformationen oder auch Objektdaten) als auch die den Geschäfts- und Archivierungskontext beschreibenden Metadaten.

4.1 Metadaten

Metadaten sind im weitesten Sinne Daten, die andere Daten beschreiben. Metadaten sind Auszeichnungsdaten (Markups), die die Strukturen und den Zusammenhang von Daten bei der Verarbeitung der Daten durch IT-Systeme beschreiben, die die Daten erzeugen, bearbeiten, verwalten und speichern. Metadaten eines Archivdatenobjektes dienen der Identifikation und Rekonstruktion des Verwaltungs- oder Geschäftskontextes der gespeicherten Inhaltsdaten.

Für die strukturelle Auszeichnung (Beschreibung) elektronischer Dokumente, d. h. die Abbildung und maschinenlesbare Beschreibung von Dokumentstrukturen haben sich heute XML-basierte Lösungen als globaler Standard für einen plattformübergreifenden Datenaustausch fest etabliert.⁴³

Ein Archivdatenobjekt, d. h. ein für die langfristige Ablage in einem elektronischen Archivsystem bestimmtes elektronisches Dokument im Sinne dieser Richtlinie ist deshalb ein selbst-beschreibendes und wohlgeformtes XML-Dokument, das gegen ein gültiges und autorisiertes XML-Schema geprüft werden kann.

4.1.1 Extensible Markup Language (XML)

Die Extensible Markup Language [XML] ist eine vor allem für das Internet entwickelte Formatbeschreibungssprache für den Austausch strukturierter Daten und wurde 1997 vom World Wide Web Consortium (W3C) standardisiert.⁴⁴

Auf syntaktischer Ebene unterstützt XML als textbasierte Meta-Auszeichnungssprache nicht nur die Beschreibung, sondern vor allem die automatisierte Darstellung, Manipulation und Verarbeitung logisch strukturierter Daten und zeichnet sich darüber hinaus durch eine gute Erweiterbarkeit und eine große Flexibilität aus.

Auf semantischer Ebene unterstützen Regeln und Strukturdefinitionen in XML-Syntax (XML-Schema) die Abbildung strukturierter Inhaltsmodelle. XML-Schemata erlauben nicht nur die formale und maschinenlesbare Beschreibung eines für den Datenaustausch erlaubten XML-Vokabulars, sondern darüber hinaus den Aufbau komplexer Datenstrukturen und die Formulierung von Verarbeitungsanweisungen. Ein XML-Schema legt mittels einer formalen Grammatik fest, welche XML-Elemente definiert sind, welche Verarbeitungsregeln wie umgesetzt werden sollen und welche Bedeutung die einzelnen Elemente besitzen.

Die Vertraulichkeit von XML-Dokumenten kann durch „XML Encryption“ [XMLENC] sichergestellt werden, die Integrität und Authentizität durch „XML Signatures“ [XMLDSIG]. Dabei werden verschiedene Formen

⁴² Weitere Festlegungen und Empfehlungen zu geeigneten Dokumentenformaten finden sich auch in BMI, Architekturrichtlinie für die IT des Bundes, Version 2020 https://www.cio.bund.de/Web/DE/Architekturen-und-Standards/Architekturrichtlinie-IT-Bund/architekturrichtlinie_it_bund_node.html

⁴³ Die Architekturrichtlinie des Bundes fordert u.a. die Nutzung von XML für den Datenaustausch. Weitere Informationen finden sich hier: https://www.cio.bund.de/Web/DE/Architekturen-und-Standards/Architekturrichtlinie-IT-Bund/architekturrichtlinie_it_bund_node.html.

⁴⁴ mehr unter: <http://www.w3c.org/XML/>

von XML-Signaturen unterschieden, abhängig davon, ob die Signatur innerhalb oder außerhalb des XML-Dokumentes liegt.

Verwendung

Für sämtliche Daten/Dokumente und als Zeichensatz und Beschreibungssprache für alle zu dieser technischen Richtlinie konformen Archivdatenobjekte: Ein gültiges XML formatiertes Archivdatenobjekt muss die XML Spezifikation des World Wide Web Konsortium (W3C) erfüllen [XML].

4.1.2 XML Schema (XSD)

XML Schema ist eine XML Auszeichnungssprache zur Definition der Struktur von XML Dokumenten (XML Instanzen). Mit Hilfe eines XML Schemas ist es möglich, eine in der Form von Regeln oder Strukturmodellen ausgedrückte Formalisierung von Strukturen und Beschränkungen, die auf eine Klasse von XML Dokumenten zutreffen, zu formulieren. XML Schemas werden i. d. R. dazu verwendet, um ein zutreffendes und gültiges XML-Vokabular für den Datenaustausch innerhalb eines Geschäftsbereiches oder Branche zu dokumentieren.

Der hauptsächliche Zweck eines XML Schemas ist dabei die Validierung von XML Dokumenten. Mit der Validierung eines XML Dokumentes gegen ein Schema kann man sicherstellen, dass der Inhalt eines XML-Dokumentes den vereinbarten Regeln entspricht.

Ein Archivdatenobjekt im Sinne dieser Richtlinie muss vor der Ablage im elektronischen Langzeitspeicher gegen ein gültiges und autorisiertes XML-Schema geprüft werden können. Die Autorisierung muss sicherstellen, dass der Ersteller (Besitzer) des Schemas eindeutig identifiziert und das Schema nicht unbemerkt manipuliert werden kann.

Verwendung

Für alle zu dieser Richtlinie konformen Archivdatenobjekte: Ein zu dieser Richtlinie konformes Schema muss die normativen Empfehlungen der XML Schema Working Group des W3C [XSD] umsetzen⁴⁵.

4.2 Inhaltsdaten (Objektdaten)

Für eine dauerhafte und rechtskonforme Aufbewahrung von elektronischen Primärinformationen (Inhaltsdaten) muss sichergestellt sein, dass die Verkehrsfähigkeit und (maschinelle) Lesbarkeit der aufbewahrten elektronischen Informationen mindestens für die Dauer der gesetzlich vorgeschriebenen Aufbewahrungsfristen durch geeignete Maßnahmen gewährleistet werden kann.

Um die langfristige Verkehrsfähigkeit sicher zu stellen, sollen daher ausschließlich standardisierte und langfristig stabile Datenformate verwendet werden, deren Beschreibung offen gelegt wurde.⁴⁶ Die Verwendung von Standardformaten verringert nicht nur die Abhängigkeit von Hard- und Softwareumgebungen, sondern auch die Notwendigkeit künftiger Formattransformationen, die insbesondere beim Einsatz digitaler Signaturen mit nicht unerheblichem Aufwand verbunden ist.

⁴⁵ Ein grundlegender Satz von Anforderungen, der in der XML Schema Requirements Note des W3C formal beschrieben vorliegt, führt eine Anzahl von Anwendungsfällen für Schemas und die ihnen zugrunde liegenden Entwurfsrichtlinien auf (Siehe unter: <http://www.w3.org/TR/NOTE-xml-schema-req>).

⁴⁶ Siehe dazu auch: Drucksache 16/5927 des Deutschen Bundestages vom 04.07.2007. Danach sollen Standards dann als offen betrachtet werden, „wenn sie den Austausch zwischen verschiedenen Plattformen und Applikationen ermöglichen und ausreichend dokumentiert sind. Die Schnittstellen müssen offengelegt und die technischen Spezifikationen umsetzbar sein. Die Ausgestaltung der Nutzungsbedingungen soll dabei den Vorgaben der internationalen Standardisierungsorganisationen entsprechen.“

4.2.1 Dokumente (Schriftgut)

Das Organisationskonzept elektronische Verwaltungsarbeit⁴⁷ und die Architekturrichtlinie Bund **[AR-Bund]**⁴⁸ oder die **[DIN31644]**⁴⁹, **[ISO14721]**⁵⁰ empfehlen, für die langfristige Ablage von elektronischem Schriftgut nur wenige und einheitliche Datenformate zu benutzen.

Dazu gehören (zum Zeitpunkt der Veröffentlichung dieser Technischen Richtlinie):

4.2.1.1 Text (ASCII)

ASCII (American Standard Code for Information Interchange) steht für einen Zeichensatz und für ein Textformat. Ein ASCII-Text beschreibt ein Dokument, das nur aus Zeichen des ASCII-Zeichensatzes besteht, also keine Layoutinformationen beinhaltet und eignet sich somit besonders für einfache Textinformationen und Metadaten. Der ASCII-Code wurde 1972 von der International Organization for Standardization als ISO 646⁵¹ spezifiziert und bietet aus heutiger Sicht die besten Voraussetzungen für eine andauernde Verkehrsfähigkeit. Da ASCII per Definition keine Zeichensätze einbindet, ist eine korrekte Darstellung nicht in jedem Falle gewährleistet.

Das Format besteht ursprünglich aus 7 bit, mit denen 128 (Zeichen-) Kombinationen dargestellt werden können, also ein Zeichensatz, der auf dem lateinischen Alphabet basiert, wie es in der modernen englischen Sprache verwendet wird. Um einige Sonderzeichen, z. B. die Umlaute der deutschen Sprache, abzubilden, wurden 8-bit-ASCII-Zeichensätze definiert.

Eine Weiterentwicklung der ASCII-Kodierung stellt der sogenannte Unicode-Standard⁵² dar. Er basiert je nach Kodierungstabelle auf 8 (UTF-8), 16 (UTF-16) oder 32 (UTF-32) Bit pro Zeichencode.

Verwendung

Für einfache, unformatierte Texte (Textdateien): Dokumente (Textdaten) in ausschließlich lateinischen Schriftzeichen sollen den im ISO Standard ISO 646:1991 (ASCII) definierten Zeichensatz verwenden **[ANSIX3.4]**.

Dokumente (Textdaten) mit nicht ausschließlich lateinischen Schriftzeichen sollen die aktuelle Version des Unicode Standards **[UNICODE]** verwenden.

Unicode ist funktional äquivalent zum Standard ISO 10646-1:2000. Unicode konforme Texte werden grundsätzlich in UTF-8 oder UTF-16 kodiert.

4.2.1.2 PDF/A

PDF/A-1 ist ein als ISO-Norm verabschiedeter Standard auf Basis von **[PDF 1.4]**⁵³ für die Langzeitarchivierung elektronischer Dokumente.

Veröffentlicht als ISO 19005-1:2011⁵³, legt PDF/A-1 Anforderungen an ein Norm-konformes PDF fest und regelt die Verwendung von PDF unter dem Gesichtspunkt der langfristigen Stabilität und Reproduzierbarkeit.

⁴⁷ Siehe https://www.verwaltung-innovativ.de/DE/Verwaltungsdigitalisierung/orgkonzept_everwaltung/orgkonzept_everwaltung_node.html

⁴⁸ Siehe https://www.cio.bund.de/Web/DE/Architekturen-und-Standards/Architekturrichtlinie-IT-Bund/architekturrichtlinie_it_bund_node.html

⁴⁹ Siehe DIN 31644:2012 Information und Dokumentation - Kriterien für vertrauenswürdige digitale Langzeitarchive

⁵⁰ Siehe ISO 14721:2012 Space data and information transfer systems — Open archival information system (OAIS) — Reference model

⁵¹ Siehe <http://www.iso.org/>.

⁵² Diese Spezifikation ist unter <http://www.unicode.org/> verfügbar.

⁵³ Siehe <https://www.iso.org/standard/60603.html>.

Die Norm spezifiziert zwei Konformitätsebenen:

- PDF/A-1a - Level A conformance: sowohl eindeutige visuelle Reproduzierbarkeit wie auch durchgängige Verwendung von Unicode und inhaltliche Strukturierung des Dokuments.
- PDF/A-1b - Level B conformance: eindeutige visuelle Reproduzierbarkeit.

PDF/A ist als Normreihe angelegt. Weitere Teile wurden innerhalb des zuständigen ISO-Komitees (ISO TC171 SC2 WG5) erarbeitet. 2011 erschien ein zweiter Normteil PDF/A-2 (basierend auf PDF 1.7) für die Langzeitspeicherung, der auf der ISO-Version des PDF-Formats (ISO32000⁵⁴) aufsetzt und einige zwischenzeitlich eingeflossene technischen Neuerungen, wie beispielsweise JPEG2000 transparente Elemente, Zusammenfassung mehrerer PDF/A-Dokumente in einem Container PDF-Dokument (Kollektion) und PDF-Ebenen berücksichtigt. Auch in PDF/A-2 gelten die Konformitätsebenen A und B. Hinzu kommt die Ebene u für Unicode, welche die Durchsuchbarkeit und Kopierbarkeit von Unicode Text erleichtert⁵⁵.

Der dritte Normteil PDF/A-3⁵⁶, basierend auf **[PDF 1.7]**, wurde im Jahr 2018 zuletzt geprüft und bestätigt, der den Standard um die Einbettung beliebiger Datentypen (XML, CAD, usw.) zur Bewahrung erweitert. PDF/A-3 ist insofern vorrangig ein Datencontainer, in dem Metadaten und weiterer Content zum Datenaustausch enthalten sind. Die langfristige Lesbarkeit solcher eingebetteten Dateien, die nicht selbst PDF/A-konform sind, ist durch die anwendende Institution sicherzustellen. Darüber hinaus ist die Beziehung, in welcher die eingebettete Datei zum PDF/A Dokument steht, zwingend anzugeben. Der Standard unterscheidet hier zwischen Alternative, Source, Data, oder Supplement

Der vierte Normteil PDF/A-4⁵⁷ basierend auf **[PDF 2.0]** wurde im Jahr 2020 veröffentlicht. Im Gegensatz zu PDF/A 1-3 beinhaltet PDF/A-4 zwei Varianten:

- PDF/A-4 f ermöglicht auch Dateianhänge, die nicht den PDF/A-Vorgaben entsprechen
- PDF/A-4 e für Engineering, also Integration von 3D-Inhalten, beschränkt auf die Formate U3D und PRC.

Im Kern ist PDF/A-4 eine weitere Option von PDF/A für spezifische Anwendungsfälle, in denen die Konformität zum klassischen PDF/A-Standard nicht im Vordergrund steht oder die Bewahrung technischer Dokumente notwendig ist (hierzu wäre PDF/A-4 e geeigneter). Die Verwendung ist durch die anwendende Institution zu prüfen.

Mit PDF/UA⁵⁸ wurde ein Standard für barrierefreie PDF-Dokumente geschaffen. PDF/UA beruht im Kern auf PDF/A-2 und ermöglicht die universelle Zugänglichkeit des Dokuments. Es ist in **[ISO 14289-1]** genormt und beschreibt Methoden zur Strukturierung von PDF-Dokumenten, die notwendig sind, um den Inhalt z.B. durch Screenreader lesen zu können. PDF/UA ist eine einfache Variante zur Erzeugung und Validierung barrierefreier wie archivfähiger PDF-Dokument ohne den vergleichsweise hohen technischen Aufwand für PDF/A-1/2a zu benötigen. In dem Fall wird das Dokument zunächst in PDF/A- 1b oder 2b erzeugt und anschließend in PDF/UA verlustfrei konvertiert

⁵⁴ Siehe <https://www.iso.org/standard/50655.html>.

⁵⁵ Vgl. <http://www.digitalpreservation.gov/formats/fdd/fdd000322.shtml> sowie <http://www.digitalpreservation.gov/formats/fdd/fdd000318.shtml>.

⁵⁶ Siehe <https://www.iso.org/standard/57229.html>.

⁵⁷ Siehe <https://www.iso.org/standard/71832.html>

⁵⁸ Vgl. <http://www.pdfa.org/wp-content/uploads/2013/08/PDFUA-kompakt-PDFUA.pdf>.

Da mit PDF/X⁵⁹ – für den Austausch von Druckvorlagen in der grafischen Industrie – bereits seit 2001 eine weitere auf PDF basierende Normenreihe existiert, wurde bei der Erarbeitung von PDF/A darauf geachtet, dass eine gültige PDF/A-Datei gleichzeitig auch eine gültige PDF/X-Datei sein kann.

PDF/A in der jeweiligen Version unterstützt, wie bereits PDF Versionen 1.4 und 1.7, verschiedene Sicherheitsmechanismen, insbesondere auch die Einbettung elektronischer Signaturen, Siegeln bzw. Zeitstempeln im international anerkannten PKCS#7-Format [PKCS#7], woraus das inzwischen in ISO32000 und [ETSI EN 319 142-1] standardisierte PAdES-Format entstanden ist, welches auch im Durchführungsrechtsakt 2015/1506 zur [eIDAS-VO] als durch öffentliche Stellen verbindlich anzuerkennendes Signaturformat enthalten ist.

PDF/A-2 wird u.a. in der Architekturrichtlinie Bund [AR-Bund] als Format für die Langzeitspeicherung empfohlen.

Verwendung

Für sämtliche (vorwiegend zeichenorientierte) statischen Dokumente.

Empfehlung

PDF Dateien sollen den Standard ISO 19005-2 (PDF/A-2) erfüllen. Bei Verwendung von PDF/A-2 gelten je nach Konformitätsebene folgende Einschränkungen:

- **Konvertierung:** Wird die Datei von einem anderen Dateiformat, z. B. Microsoft Word, in PDF/A konvertiert, dann muss dabei ein Programm verwendet werden, das explizit die Erstellung von PDF/A-konformen Dateien ermöglicht. Alternativ kann eine Ausgabe in Version 1.7 von PDF erfolgen. Auch in diesem Fall ist aber eine abschließende Konvertierung in PDF/A mit einem entsprechenden Konverter durchzuführen, da PDF 1.4 und 1.7 Strukturen und Inhalte erlaubt, die in PDF/A untersagt sind.
- **Tagged PDF:** Tagged PDF schreibt für den textuellen Inhalt einer PDF-Datei eine definierte interne Struktur vor, um die Inhalte mit geeigneten Werkzeugen auslesen und für andere Zwecke weiterverarbeiten zu können. Mögliche Anwendungen sind die Übertragung von Inhalten in Dateiformate wie XML, HTML oder RTF. Soweit im konkreten Anwendungsfall notwendig sollen Dateien so erstellt werden, dass sie Tagged PDF, wie in [PDF 1.4] bzw. [PDF1.7] beschrieben, entsprechen. Damit kann die Konformitätsebene A (PDF/A-2a) erreicht werden. Alternativ kann zunächst ein PDF/A in Konformitätsebene B erzeugt und anschließend nach PDF/UA transformiert werden, welches Tagged PDF erfüllt.
- **Metadaten:** Eine aufbewahrbare Datei soll möglichst selbstbeschreibend sein, was unter anderem auch durch die Speicherung von Metadaten auf Basis der von Adobe entwickelten eXtensible Metadata Platform (XMP)⁶⁰ [ISO16684] in der Datei selbst gewährleistet werden kann. Die Ablage von Metadaten im PDF/A-Dokument muss im XMP-Format durchgeführt werden. Dies betrifft vor allem die originären PDF-Metadaten wie Autor, Titel, Erstellungswerkzeug etc. sofern diese im Dokument verwendet werden. Neben der Verwendung von zehn, bereits im XMP-Standard enthaltenen Metadaten-Schemata (z. B. Dublin-Core) ist ab PDF/A-2 die Verwendung weiterer Metadaten möglich, sofern sie über ein PDF/A-Erweiterungsschema definiert wurden.
- **Verschlüsselung und andere Sicherheitseinstellungen:** Der lesende Zugriff auf eine Datei darf keinerlei Beschränkungen unterliegen, insbesondere dürfen keine Passwörter verwendet werden, um die Anwendung bestimmter Funktionen auf die Datei zu unterbinden. Damit ist z. B. gewährleistet, dass die Datei zu Zwecken der Informationserhaltung in andere Formate übertragen werden kann, wenn dies zur Erhaltung ihres Inhalts notwendig ist. Druckeinschränkungen sind zu vermeiden. Darüber hinaus gehende Sicherheitsoptionen, wie Einschränkungen bzgl. der Extraktion von Textpassagen, oder Kopieroptionen dürfen nicht verwendet werden. Der Verschlüsselungsstandard AES wird ab PDF/A-2 unterstützt.
- **Authentizität und Integrität:** Die Verwendung kryptographischer Verfahren (elektronische Signaturen, Siegel und Zeitstempel) zum Nachweis der Authentizität und Integrität einer PDF-Datei,

⁵⁹ PDF/X ist in den ISO-Standards 15929 und 15930 genormt, ISO 15929 definiert den PDF/X-Ansatz insgesamt, ISO 15930 definiert konkrete Normteile, mehr unter <http://www.iso.org/>.

⁶⁰ Siehe XMP Adobe eXtensible Metadata Platform; mehr unter <http://www.adobe.com/products/xmp>.

wird, wenn nicht durch gesetzliche Vorschriften ausdrücklich geboten, empfohlen. Für die Einbettung von elektronischen Signaturen, Siegeln und Zeitstempel in PDF-Dokumente ist, wie in [2015/1506/EU] gefordert, das PAdES Baseline Profile gemäß [ETSI TS 103 172] bzw. zukünftig die Europäische Norm [ETSI EN 319 142-1] zu berücksichtigen.

- **Text:** Jede im Text der Datei verwendete Schriftart muss in die Datei eingebettet sein, die Datei muss also neben dem eigentlichen Text auch die grafischen Beschreibungen aller darin verwendeten Schriftarten enthalten. Zur Optimierung müssen nur die für die Beschreibung der aktuell im Text verwendeten Zeichen eingebettet werden (Subsetting). Diese Maßnahme gewährleistet, dass PDF-Anzeigeprogramme die Datei stets in der vom Autor/von der Autorin beabsichtigten Form darstellen können, ohne auf Ersatz-Schriftarten zurückgreifen zu müssen, welche die visuelle Darstellung der Datei verändern könnten. Auch die von Adobe zur Verfügung gestellten Standard-Schriftarten sind, sofern sie im Dokument verwendet werden, immer einzubetten. Grundsätzlich sollen nur öffentlich verfügbare Schriftarten verwendet werden, die keinerlei von den Inhabern an deren Rechten festgelegten Beschränkungen unterliegen. Möglichst alle in der Datei enthaltenen Zeichen sollen in maschinenlesbar codierter Form vorliegen und nicht als digitalisierte Bilder von Zeichen, damit sie für Suchfunktionen zur Verfügung stehen. Für Textabschnitte, die Zeichen enthalten, welche über den Zeichensatz ISO-8859-1 für US-amerikanische und westeuropäische Sprachen [ISO-Latin-1]⁶¹ hinausgehen, soll der Zeichensatz [UNICODE] verwendet werden. Ein Autor/eine Autorin soll Texte also entweder in [ISO-Latin-1] oder in UNICODE codieren, was konkret durch die Auswahl entsprechender Schriftarten, die über eine durchgängige Codierung verfügen, realisiert wird, z. B. Arial Unicode MS.
- **Grafiken:** Enthält die Datei grafische Darstellungen (Abbildungen), dann müssen diese entweder mit ihren Quellfarbprofilen (z. B. CalRGB) eingebunden sein, oder allen Grafiken wird ein einheitliches Zielfarbprofil bzw. OutputIntent (z. B. Standard Red-Green-Blue [sRGB]⁶²) zugewiesen. Damit wird eine geräteunabhängige Farbraumspezifikation erreicht. Der als visueller Effekt in Version 1.4 von PDF eingeführte so genannte Transparenzschlüssel, der es ermöglicht, einander überlappende und durchscheinende Grafiken zu erzeugen, darf generell nicht verwendet werden. Alternative Darstellungen (z. B. so genanntes Downsampling für die schnelle Vorschau im Web) sind weder für Farb- noch für Grauwertbilder zulässig. Das gleiche gilt für Ebenen. Ab PDF/A-2 ist auch JPEG 2000 als Bildformat in einer PDF/A-Datei erlaubt.
- **Einbindungen:** Alle für die Darstellung des Dokuments notwendigen Inhalte müssen in der Datei selbst enthalten sein, so dass kein Laden von Datenströmen aus externen Quellen erforderlich ist. Die Datei darf kein eingebundenes Objekt enthalten, dessen Darstellung ein externes Anwendungsprogramm erfordern würde. Eine Darstellung, die ein Rendering für spezifische Ausgabegeräte erfordert, ist nicht zulässig. Die Einbindung von PDF/A-Dateien in PDF/A-Dokumente (Kollektion) ist möglich, ebenso sind PDF-Ebenen ab PDF/A-2 möglich.
- **Audio/Video:** Aus den im vorigen Absatz angeführten Gründen darf eine Datei auch weder Audio- noch Video-Datenströme enthalten. Dies ist erst ab PDF/A-3 möglich.
- **Verknüpfungen (Links):** Interne Verknüpfungen, die auf Sprungmarken innerhalb der Datei, wie etwa Überschriften, verweisen, sind zulässig. Externe Verknüpfungen, die auf Sprungmarken außerhalb der Datei verweisen, beispielsweise Hyperlinks zu Ressourcen im Internet, sollen nach Möglichkeit so aufgebaut sein, dass alle symbolischen Adressen dieser Marken, also Dateipfade, Uniform Resource Locators (URL) oder Persistent Identifiers, im Text der Datei enthalten sind und ohne zusätzliche Maßnahmen am Bildschirm angezeigt bzw. auf Papier ausgedruckt werden können. Ein Beispiel: statt „Website des BSI“ mit dahinter liegendem Link sollte geschrieben werden: „Website des BSI (<http://www.bsi.de>)“.
- **Kommentare:** Die Verwendung von Kommentaren ist zulässig, solange diese keine Audio- oder Video-Datenströme und keine sonstigen Dateianhänge enthalten.

⁶¹ Siehe ISO/IEC 8859-1:1998 Information technology - 8-bit single-byte coded graphic character sets, Part 1: Latin alphabet No. 1, siehe unter: <http://www.iso.org/standard/28245.html>.

⁶² Siehe IEC-Standard: IEC 61966-2-1 – Ed. 1.0 – Bilingual Multimedia systems and equipment - Colour measurement and management - Part 2-1: Colour management - Default RGB colour space - sRGB, siehe unter: <http://webstore.iec.ch/webstore/webstore.nsf/artnum/025408>.

- **Ausführbare Aktionen:** Die Datei darf keine Aufrufe von ausführbaren Anweisungsfolgen, wie z. B. Scripts, beinhalten, insbesondere darf weder innerhalb von Feldern in Formularen noch an anderer Stelle JavaScript eingebunden sein. Formularfelder selbst sind zulässig.

4.2.1.3 ODF

Das Open Document Format (ODF) wurde von OASIS⁶³ als XML-basiertes Dokumentenformat für Texte, Tabellenkalkulationen, Präsentationen und andere Office-Dokumente standardisiert. Inhalt der Dokumente und Informationen über ihr Layout sind voneinander getrennt und können dadurch unabhängig verarbeitet werden. Es kann zum Austausch von komplexen Dokumenten eingesetzt werden, die zur Weiterbearbeitung vorgesehen sind.

Im November 2006 erfolgte die Veröffentlichung von OpenDocument v1.0 unter dem Namen ISO/IEC 26300:2006⁶⁴ als Standard. Das OpenDocument Format wird u. a. durch das plattformunabhängige, lizenzkostenfreie und offene Office-Paket von OpenOffice.org⁶⁵ unterstützt. ODF Version 1.2 wurde am 29.09.2011⁶⁶ und ODF V1.3 am 10.01.2020 als OASIS Standard bestätigt⁶⁷.

Verwendung

Für sämtliche (vorwiegend) zeichenorientierte Dokumente.

4.2.1.4 TIFF⁶⁸

Das „Tagged Image File Format“ (TIFF) erlaubt das Speichern von Grafikinformaten ohne Informationsverlust und ist nach ISO 12639 für die medienunabhängige Bildverarbeitung standardisiert worden. Die Kodierung des Formats erlaubt es, mehrere Darstellungen (z. B. Thumbnails) oder Versionen einer Grafik oder auch Textinformation als Metadaten in einer Datei abzulegen.

Der Einsatz von TIFF ist vor allem immer dann angezeigt, wenn die grafischen Informationen eines Dokuments von maßgeblicher Bedeutung für die Aussagekraft sind. Unterstützt wird TIFF durch alle gängigen Grafik- und Präsentationsprogramme.

Um maximale Interoperabilität zu erreichen, sollen ausschließlich Eigenschaften aus der „Baseline TIFF“⁶⁹ eingesetzt werden. TIFF kann zum Einsatz kommen, wenn die Fähigkeit des Formats benötigt wird, mehrseitige Dokumente darzustellen. Für eingescannte Textdokumente ist TIFF besonders geeignet.

Verwendung

Für Abbildungen (Non Coded Information, bspw. Rasterbilder).

TIFF Dateien sollen die Basis-TIFF-Spezifikation in der Version 6.0 erfüllen [TIFF6].

Darüber hinaus können folgende Erweiterungen genutzt werden:

- CCITT Bilevel Kodierung und
- LZW Kompression.

⁶³ Siehe <http://www.oasis-open.org/committees/download.php/12572/OpenDocument-v1.0-os.pdf>.

⁶⁴ Siehe <http://www.iso.org/>.

⁶⁵ Siehe <http://de.openoffice.org/>.

⁶⁶ Siehe <https://www.oasis-open.org/news/pr/odf-1-2-approval>.

⁶⁷ Siehe <https://www.oasis-open.org/news/announcements/open-document-format-for-office-applications-opendocument-v1-3-from-the-opendocum>.

⁶⁸ Siehe <https://www.adobe.io/open/standards/TIFF.html> sowie <http://www.kost-ceco.ch/wiki/whelp/KaD/index.php?ld=http://www.kost-ceco.ch/wiki/whelp/KaD/pages/TIFF.html>.

⁶⁹ Unter „Baseline TIFF“ sind Eigenschaften von TIFF-Dateien zusammengefasst, die jedes TIFF-fähige Programm unterstützen sollte. Beispielsweise gehören zu „Baseline TIFF“ ausschließlich die beiden Komprimierungsverfahren „Huffman“ und „Packbits“, während „LZW“, „JPEG“, „ZIP“ und „CCITT“ optionale Erweiterungen sind, die nicht in jedem TIFF-fähigen Programm implementiert sind.

4.2.1.5 JPEG und JPEG 2000

Das JPEG-Format⁷⁰ (Joint Photographic Experts Group) steht für verlustfreie und verlustbehaftete Kompressionsverfahren und ein Grafikformat und ist eines der häufigsten im Internet verwendeten Grafikformate. Eine erste Normung von JPEG erfolgte in [ISO10918-1]. Mit JPEG 2000 liegt ein Format vor, genormt in [ISO15444] und bietet sich vor allem für große Bilddateien an. Teil 6 der Spezifikation beschreibt eine verlustfreie bzw. reversible Komprimierung. Das in Teil 9 der JPEG 2000 Spezifikation enthaltene JPEG 2000 Interactive Protocol vereinfacht die Anwendung, indem eine gezielte Auswahl der zu komprimierenden/dekomprimierenden Teile der Bilddatei in der Rendition möglich ist. Darüber hinaus sind zwei Typen von JPEG 2000 zu unterscheiden:

- JP2: beschränkt die möglichen Farbprofile auf drei ICC-Profile [ISO15444-1]
- JPX: ermöglicht die Nutzung unbegrenzter Farbprofile was im Hinblick auf eine langfristige Bewahrung kritisch zu prüfen ist⁷¹

Verwendung

Für Abbildungen (Non Coded Information, bspw. Rasterbilder).

4.2.1.6 PNG

PNG⁷² (Portable Network Graphics Format) wurde von der späteren „PNG Development Group“ als Alternative zum GIF-Format entwickelt und eignet sich wegen der Möglichkeit der verlustfreien Kompression und inkrementellen Anzeige der Grafiken vor allem für Anwendungen im Internet. Die PNG-Spezifikation ist offengelegt und wurde als ISO / IEC 15948 im Jahr 2003 zum internationalen Standard erhoben⁷³. Das PNG-Format wird von den meisten Bildverarbeitungsprogrammen standardmäßig unterstützt. So genannte Kalibrierungs-Datenblöcke erlauben die Kalibrierung der Darstellung damit z. B. der Ausdruck eines Bildes genauso aussieht, wie es der Autor an seinem Bildschirm gesehen hat.

Verwendung

Für Abbildungen (Non Coded Information, bspw. Rasterbilder).

4.2.2 SVG – Scalable Vector Graphics

SVG ist ein vom World Wide Web Consortium (W3C) herausgegebenes XML-basierendes Grafikformat für zweidimensionale Vektorgrafiken. Der große Vorteil gegenüber anderen Grafikformaten ist, dass Grafiken im SVG-Format ohne Qualitätsverlust skaliert werden können. Zudem ist SVG aufgrund des XML-Formats leichter maschinell zu verarbeiten.

Die aktuelle Version 1.1 von SVG wurde am 16.8.2011 veröffentlicht.⁷⁴ Derzeit wird für SVG die Version 2 erarbeitet.⁷⁵ [AR-Bund]

Verwendung:

Für Abbildungen (Vektorgrafiken).

4.2.3 Multi-Media Formate

„Multimedia“ ist durch die Möglichkeit der gleichzeitigen Verwendung diverser digitaler Darstellungsformen von Informationen charakterisiert (Video, Ton, Bild und Text).

⁷⁰ Siehe <http://www.jpeg.org/index.html?langsel=de>, standardisiert als ISO/IEC 10918-1:1994, siehe <http://www.iso.org/>, standardisiert als ITU-T.81, siehe <http://www.itu.int/rec/T-REC-T.81/en>.

⁷¹ Vgl. http://www.dpconline.org/component/docman/doc_download/526-jp2knov2010vanderkniff.

⁷² Siehe <http://www.w3.org/TR/PNG/>.

⁷³ Siehe <http://www.iso.org/>, <http://www.w3.org/TR/2003/REC-PNG-20031110>.

⁷⁴ Siehe <https://www.w3.org/TR/SVG11/>.

⁷⁵ Siehe <https://www.w3.org/TR/SVG2/>.

Ein Multimedia-Format ist in diesem Zusammenhang ein selbst-beschreibendes Dateiformat, welches die Inhaltsdaten enthält und (im Fall von Containerformaten, siehe unten) deren Struktur und Zwischenbeziehungen definiert.

Zu unterscheiden sind drei Multimedia-Formate:

- Ein Audioformat beschreibt den Aufbau einer Audiodatei
- Ein Videoformat beschreibt den Aufbau einer Videodatei
- Ein Containerformat kann mehrere Datenelemente (genannt „Streams“) mit unterschiedlichen Formaten enthalten. Dies ist zunächst nicht nur auf Multimedia-Formate beschränkt. Der Container regelt auch das Zusammenspiel und die Reihenfolge der einzelnen Datenströme. So kann ein Container z. B. mehrere Audiostreams zu einem Videostream enthalten und es besteht die Möglichkeit der Einbettung von Untertiteln (z. B. in Form von Grafiken). Die Audiostreams können auch mit verschiedenen Codecs⁷⁶, die zur Kodierung bzw. Dekodierung analoger Signale bzw. digitaler Audio- oder Videodaten verwendet werden, codiert sein und so z. B. unterschiedliche Tonqualitäten abbilden.

In den folgenden Abschnitten werden die häufigsten Vertreter der Dateiformatklassen „Audio“ und „Video“ sowie „Container“ kurz erläutert. Die gängigen Containerformate werden normalerweise nicht als Containerformate, sondern als Audio- bzw. Videoformate wahrgenommen. Daher werden die Containerformate in den beiden folgenden Abschnitten mit dargestellt. Die empfohlenen Dateiformate sind entsprechend gekennzeichnet.

HINWEIS 11

Die heute gängigen und in die in diesem Dokument empfohlenen Multimediaformate wurden noch nicht hinreichend auf ihre Eignung für die Langzeitspeicherung vergleichbar zu PDF/A untersucht. Demzufolge kann es für eine langfristige Verfügbarkeit der archivierten Daten notwendig sein, neben den Rohdaten weitere Daten wie z. B. Sound-Modelle oder Video-Treiber zu archivieren.

Selbstverständlich kann hier nur ein kleiner Ausschnitt zu empfehlender Multimediaformate gegeben werden. Es existieren noch zahlreiche anderer Formate, die im Rahmen der digitalen Speicherung von Ton- und Videoinformationen Verwendung finden. Im Kontext der Langzeitspeicherung können diese Formate jedoch derzeit nur bedingt oder gar nicht empfohlen werden, da ihnen entweder die Verbreitung fehlt, keine offen gelegte Spezifikation vorliegt oder die rechtliche Situation (insbesondere die Lizenzbedingungen) eine Verwendung erschweren bzw. nicht empfehlen. Diese Richtlinie orientiert sich daher – soweit möglich und sinnvoll – an der Architekturrichtlinie Bund [AR-Bund].

4.2.3.1 Audioformate

4.2.3.1.1 WAVE-Audio File Format

Die bekannteste Methode zu Codierung von Audiodateien ist Pulse Code Modulation (LPCM). In dem Fall werden die Klangwellen einzeln in Intervallen codiert. In LPCM codierte Daten sollen in im WAVE-Audio File Format gespeichert werden- Das Format beschreibt eine unbegrenzte Anzahl sog. „chunk“ oder Teile zur Aufnahme von Metadaten. Die WAVE-Ausprägung Broadcast WAVE Format (BWF) integriert darüber hinaus einen zusätzlichen Teil den sog. „bext chunk“. Dieser ermöglicht es, zusätzliche z. B. zur Bewahrung

⁷⁶ Codec, Kunstwort für Coder / Decoder; ein Programm oder Verfahren (Format) mit den Multimediadaten (Audio- und/oder Videosignale) für die Übertragung durch digitale Dienste / Netze digital kodiert und beim Abspielen wieder dekodiert werden. In den meisten Fällen werden beim Kodiervorgang die analogen Signale nicht verlustfrei digitalisiert, sondern es wird eine Dynamikreduktion des analogen Signals sowie eine Datenkompression des digitalen Signals vorgenommen, die je nach Ausmaß und Verfahren maßgeblich die Qualität bei der Rückwandlung des digitalen Datenstroms in analoge Signale beeinflusst. Vornehmliches Ziel der Dynamikreduktion und Kompression ist eine Verringerung der für die Übertragung des digitalen Signals notwendigen Bandbreite, bzw. eine Verringerung der für die Speicherung notwendigen Speicherkapazität. Die komprimierten Dateien werden in dem speziellen Code-Format abgelegt.

relevante Metadaten einzubetten. BWF ist das meist genutzte Masterformat zur Bewahrung von Mono- und Stereo-Audiodateien. Ein wesentlicher Vorteil besteht darin, dass die Formatdokumentation vollständig frei zur Verfügung steht⁷⁷.

4.2.2.1.2 Ogg Encapsulation Format

Ogg [RFC3533] ist ein Containerformat für Multimedia-Dateien. Die Entwicklung des Container-Formats wird von der Xiph.Org Foundation⁷⁸ geleitet. Es werden ebenfalls Codecs zur Verfügung gestellt.

Der weit verbreitetste Codec ist der Audio-Codec **Vorbis**⁷⁹. Für eine bessere Qualität kann als Alternative der verlustfreie Audio-Codec **FLAC**⁸⁰ verwendet werden.

Ogg ist eine von Softwarepatenten freie und unbeschränkte Alternative zu proprietären Formaten und ist daher geeignet für eine elektronische Langzeitspeicherung, insbesondere durch die zur Verfügung stehende und detaillierte Format-Spezifikation.

Ogg wird in der Architekturrichtlinie Bund **[AR-Bund]** als Audioformat empfohlen.

Das Format hat zudem den Vorteil, dass es auch als empfohlenes Videoformat (mit anderem Codec) genutzt werden kann, was dazu beiträgt, die Anzahl der insgesamt für die Langzeitspeicherung geeigneten Datenformate zu reduzieren.

4.2.3.1.2 MP4 / MPEG-4 Part 14

MP4 ist ebenfalls ein Containerformat für Multimedia-Dateien. Die Entwicklung des Container-Formats wurde von der Moving Picture Experts Group geleitet und ist in ISO/IEC 14496-12 und -14 (MPEG-4 Teil 12 und 14)⁸¹ standardisiert.

MPEG-4 als Audioformat ist ein offener, herstellerunabhängiger Standard und ist geeignet für eine elektronische Langzeitspeicherung. Das Audioformat wird in der Architekturrichtlinie Bund **[AR-Bund]** empfohlen.

MP4 hat zudem den Vorteil, dass es auch als empfohlenes Videoformat (mit anderem Codec) genutzt werden kann, was dazu beiträgt, die Anzahl der insgesamt für die Langzeitspeicherung geeigneten Datenformate zu reduzieren.

4.2.3.1.3 Advanced Audio Coding (AAC)

Advanced Audio Coding (AAC) ist ein standardisiertes verlustbehaftetes Audio-Format. AAC ist spezifiziert in ISO/IEC 13818-7⁸² und wird als verbesserter Nachfolge von MP3 gehandelt, da die Qualität bei gleicher Bitrate meist höher ist.

AAC ist Teil der MPEG-2 und MPEG-4 Spezifikation und wurden von der Moving Pictures Experts Group entwickelt. AAC selbst ist frei von Patenten und Lizenzen, allerdings müssen Hersteller von Codecs für AAC Lizenzgebühren zahlen.

Die Architekturrichtlinie Bund **[AR-Bund]** erwähnt AAC nicht. Dessen ungeachtet kann es aufgrund seiner Offenheit und zu erwartenden Verbreitung in der Zukunft durchaus für die Langzeitspeicherung empfohlen werden.

⁷⁷ Vgl. <https://www.loc.gov/preservation/digital/formats/fdd/fdd000003.shtml> mit File Format Guidelines for Management and Long -Term Retention of electronic records. North Carolina State Archive. 2012. Sowie <https://kost-ceco.ch/cms/wav.html>.

⁷⁸ Siehe unter <https://xiph.org/flac/>.

⁷⁹ Siehe unter <http://www.vorbis.com>.

⁸⁰ Siehe unter <http://flac.sourceforge.net/>.

⁸¹ Siehe unter <https://www.iso.org/standard/75929.html>.

⁸² Siehe unter <http://www.iso.org/>.

4.2.3.2 Videoformate

4.2.3.2.1 Ogg Encapsulation Format

Wie bereits oben erwähnt ist Ogg [RFC3533] ein Containerformat für Multimedia-Dateien. Als passenden Codec gibt es den Video-Codec **Theora**⁸³ der ebenfalls von der Xiph.Org Foundation entwickelt wurde.

Ogg ist eine von Softwarepatenten freie und unbeschränkte Alternative zu proprietären Formaten und ist geeignet für eine elektronische Langzeitspeicherung, insbesondere durch die zur Verfügung stehende und detaillierte Format-Spezifikation.

Ogg wird in der Architekturrichtlinie Bund [AR-Bund] als Videoformat empfohlen.

4.2.3.2.2 MP4 / MPEG-4 Part 14

MP4 ist ebenfalls ein Containerformat für Multimedia-Dateien. Die Entwicklung des Container-Formats wurde von der Moving Picture Experts Group geleitet und ist in ISO/IEC 14496-12 und -14 (MPEG-4 Teil 12 und 14)⁸⁴ standardisiert.

MPEG-4 als Videoformat ist ein offener, herstellerunabhängiger Standard und ist geeignet für eine elektronische Langzeitspeicherung. Das Videoformat wird in der Architekturrichtlinie Bund [AR-Bund] empfohlen.

MP4 hat zudem den Vorteil, dass es auch als empfohlenes Audioformat (mit anderem Codec) genutzt werden kann, was dazu beiträgt, die Anzahl der insgesamt für die Langzeitspeicherung geeigneten Datenformate zu reduzieren.

4.3 Base64 Kodierung

Sämtliche (binären) Primärinformationen (Inhaltsdaten), die in Kapitel 3.1.4 vorgestellt wurden, sollen innerhalb der `dataObjectsSection` der XAIP Struktur des Archivdatenobjektes (siehe Kapitel 3.1.4) gespeichert werden. Gleichzeitig besteht auch hier die Anforderung, dass das Format und die Daten über eine lange Zeit hinweg und auch von anderen Systemen und Plattformen in der Zukunft lesbar sein sollen.

Das unveränderte Einfügen von binären Daten in eine XML-Struktur ist zwar technisch möglich, verursacht in der Regel jedoch einige Probleme und führt sicher zu einer gewissen Abhängigkeit von einer Software oder einer Plattform. Daher müssen die Binärdaten zunächst in eine plattformunabhängige Form gebracht werden. Aus rechtlichen Gründen darf die dafür notwendige Codierung am Inhalt der Binärdaten keine Änderungen durchführen; es muss sich also um eine bijektive Abbildung⁸⁵ handeln.

Die meistverbreitete Methode ist die BASE64 Codierung nach [RFC4648]. Sie findet vor allem auch im Internet-Standard MIME⁸⁶ (Multipurpose Internet Mail Extensions) Anwendung und wird damit hauptsächlich zum Versenden von E-Mail-Anhängen verwendet. Nötig ist dies, um den problemlosen Transport von beliebigen Binärdaten zu gewährleisten, da das Simple Mail Transfer Protocol (SMTP)⁸⁷ in seiner ursprünglichen Fassung nur für den Versand von 7-Bit-ASCII-Zeichen ausgelegt war.

Bei der BASE64 Codierung werden jeweils 3 Byte (24 Bit) der Originaldatei auf 4 Blöcke mit jeweils 6 Bit der Zielformat abgebildet. Die Reihenfolge der Bits bleibt dabei erhalten. 6 Bit pro Block erlauben $2^6 = 64$ mögliche Zeichen. Daher auch der Name des Verfahrens. Für diese 64 möglichen Zeichen wurden die Zeichen A–Z, a–

⁸³ Siehe unter <http://www.theora.org/>

⁸⁴ Siehe unter <http://www.iso.org/>

⁸⁵ Eine Abbildung $f:A \rightarrow B$ heißt bijektiv (oder umkehrbar eindeutig), wenn verschiedene Elemente einer Originalmenge A auf verschiedene Elemente einer Bildmenge B abgebildet werden, und umgekehrt jedes Element aus B als (Ab-)Bild genau eines Elementes der Originalmenge A vorkommt.

⁸⁶ Siehe unter <http://www.ietf.org/rfc/rfc2045.txt>

⁸⁷ Siehe unter <http://www.ietf.org/rfc/rfc0821.txt>

z, 0–9, + und / ausgewählt. Diese Zeichen sind sowohl in ASCII als auch in EBCDIC enthalten und unabhängig von einer Codepage. So können Daten auch zwischen nicht-ASCII Systemen ausgetauscht werden.

Durch die Abbildung von 3 Bytes auf 4 Zeichen steigt der Platzbedarf um 33%. Dieser Nachteil wird in der Regel jedoch in Kauf genommen.⁸⁸

Für Kapselung von Inhaltsdaten in einem zu dieser Technischen Richtlinie konformen XAIP Dokument soll folgendes Verfahren zur Anwendung kommen:

- Bestehen die Inhaltsdaten ausschließlich aus Text (ASCII) (siehe Kapitel 4.2.1.1) oder XML, können diese Daten ohne Umkodierung in die XAIP Datenstruktur eingefügt werden.
- Alle anderen Datenformate müssen BASE64 codiert werden, bevor sie in die XAIP Datenstruktur eingefügt werden.
- Für BASE64 kodierte Daten soll in dem Metadatenabschnitt, der Informationen darüber enthält, die möglicherweise für eine adäquate Repräsentation der Inhaltsdaten benötigt werden, ein Eintrag für das entsprechende MIME-Format der Inhaltsdaten vorgesehen werden

⁸⁸ Es existieren noch andere Kodierungsverfahren Diese sind jedoch zumeist nur für spezielle Anwendungen verbreitet (z.B. im Post-Script Dateiformat von Adobe oder zur Adresskodierung von IPv6 Adressen) oder haben gar keine Verbreitung gefunden.

5 Kryptographische Datenformate

Der folgende Abschnitt beschreibt einige kryptographische Datenformate, die zum Zeitpunkt der Veröffentlichung dieser Technischen Richtlinie für die vertrauenswürdige Langzeitspeicherung empfohlen werden.

5.1 Signatur- und Siegelformate

Zu den in der Praxis gebräuchlichen Signatur- bzw. Siegelformaten zählen insbesondere

- die ASN.1-basierten digitalen Signaturen der CMS-Familie gemäß
 - des im Durchführungsrechtsakt **[2015/1506/EU]** referenzierten CAdES Baseline Profile **[ETSI TS 103 173]** bzw. zukünftig der Europäischen Norm **[ETSI EN 319 122-1]**,
- die XML-basierten digitalen Signaturen gemäß
 - des im Durchführungsrechtsakt **[2015/1506/EU]** referenzierten XAdES-Baseline Profile **[ETSI TS 103 173]** bzw. zukünftig gemäß der Europäischen Norm **[ETSI EN 319 132-1]**,
- die PDF-basierten digitalen Signaturen gemäß
 - des im Durchführungsrechtsakt **[2015/1506/EU]** referenzierten PAdES-Baseline Profile **[ETSI TS 103 172]** bzw. zukünftig gemäß der Europäischen Norm **[ETSI EN 319 142-1]** und schließlich
- die ZIP-basierten ASiC-Container für digitale Signaturen, elektronische Zeitstempel und Evidence Records gemäß
 - des im Durchführungsrechtsakt **[2015/1506/EU]** referenzierten ASiC-Baseline Profile **[ETSI TS 103 174]** und der Europäischen Norm **[ETSI EN 319 162-1]**.

Sofern im Umfeld der vertrauenswürdigen Langzeitspeicherung Einfluss auf die verwendeten Signatur- bzw. Siegel- bzw. Zeitstempelformate genommen werden kann, sollen diese Formate verwendet werden, wobei die Empfehlungen in den Abschnitten 5.1.1, 5.1.2, 5.1.3 und 5.1.4 berücksichtigt werden sollen.

Grundsätzlich kann jedoch – abhängig von den anwendungsspezifischen Anforderungen – auch der Einsatz von anderen Signatur- bzw. Siegelformaten (vgl. **[HK 06b]**) notwendig und sinnvoll sein. In diesem Fall sollen die Empfehlungen in Abschnitt 5.1.5 berücksichtigt werden.

5.1.1 PKCS#7 / CMS / CAdES

Das auf **[PKCS#7]** zurückgehende Cryptographic Message Syntax (CMS) Signaturformat gemäß **[RFC3852]** bzw. **[RFC5652]** ist das in der Praxis am häufigsten eingesetzt ASN.1-basierte Signatur- bzw. Siegelformat. Da bei diesem Signatur- bzw. Siegelformat die zu signierenden Daten lediglich als binäre Objekte – ohne Betrachtung einer internen Struktur – behandelt werden, können zwar beliebige Daten signiert bzw. gesiegelt werden, die digitalen Signaturen können aber nicht ohne weiteres in die Nutzdaten eingebettet werden.

Aufbauend auf der grundlegenden CMS-Struktur sind in **[ETSI TS 101 733]** bzw. **[RFC5126]** spezifische Erweiterungen definiert, die den besonderen Anforderungen für fortgeschrittene elektronische Signaturen gemäß **[eIDAS-VO]**, Artikel 26 bzw. den fortgeschrittenen elektronischen Siegeln gemäß **[eIDAS-VO]**, Artikel 36 Rechnung tragen. Beispielsweise sind hier Attribute für das Gegenzeichnen einer digitalen Signatur (CounterSignature), das Einfügen von Attributzertifikaten (SignatureAttributes), Zeitstempeln (ContentTimeStamp, SignatureTimeStampToken, ESCTimeStampToken, TimestampedCertsCRLs und ArchiveTimeStampToken), Zertifikaten (CertificateValues) und Sperrinformationen (RevocationValues) vorgesehen. Für die langfristige Aufbewahrung von CAdES-basierten digitalen Signaturen existiert darüber hinaus ein internationales Standardprofil **[ISO14533-1]** und im CAdES Baseline Profile **[ETSI TS 103 173]**, das im Durchführungsrechtsakt **[2015/1506/EU]** referenziert wird, sowie in **[ETSI EN 319 122]** finden sich Festlegungen, die auf eine Steigerung der Interoperabilität abzielen.

Für die Behandlung von CMS- und CAdES-basierten digitalen Signaturen sieht die vorliegende Richtlinie Folgendes vor:

(A5.1-1) CMS-basierte digitale Signaturen müssen vor der Erstellung des initialen Archivzeitstempels geprüft werden. Hierbei müssen im Einklang mit [ISO14533-1] und [ETSI TS 103 173]⁸⁹ gemäß [2015/1506/EU] alle bei der Prüfung verwendeten Zertifikate im Element `SignedData.certificates` abgelegt werden. In entsprechender Weise müssen auch die für die Prüfung herangezogenen Sperrinformationen im Element `SignedData.crls` abgelegt werden, wobei Sperrlisten in der Option `SignedData.crls.crl` und OCSP-Responses in der Option `SignedData.crls.other` abzulegen sind. Alternativ, im Falle, dass die aufzubewahrenden Daten nicht verändert werden dürfen, müssen die o. g. Prüfdaten in den entsprechenden Unterelementen (z. B. `xaip:certificateValues`, `xaip:revocationValues` etc.) des Elements `xaip:credential` (vgl. Kap. 3.1.5) abgelegt und mit der korrespondierenden Signatur verlinkt werden.

5.1.2 Digitale XML Signaturen / XAdES

Neben den oben erläuterten CMS-basierten digitalen Signaturen werden in der Praxis zunehmend auch XML-basierte digitale Signaturen gemäß [XMLDSIG] eingesetzt. Der Vorteil dieses Signatur- bzw. Siegelformates ist, dass den spezifischen Besonderheiten von XML-basierten Daten Rechnung getragen wird und deshalb beispielsweise nur explizit definierte Teile eines Dokumentes signiert werden können und digitale Signaturen selbst in die Nutzdaten eingebettet werden können. Auch hier existieren spezifische Erweiterungen für fortgeschrittene elektronische Signaturen gemäß [eIDAS-VO], Artikel 26 bzw. für fortgeschrittene elektronische Siegel gemäß [eIDAS-VO], Artikel 36, die in [ETSI TS 101 903], [XAdES] bzw. [ETSI EN 319 132-1], [ETSI EN 319 132-2] und [ETSI EN 319 132-3] definiert sind.

Beispielsweise sind hier Signatur- bzw. Siegeleigenschaften (Properties) für das Gegenzeichnen einer digitalen Signatur (CounterSignature), das Einfügen von Attributzertifikaten (SignerRole), Zeitstempeln (AllDataObjectsTimeStamp, IndividualDataObjectsTimeStamp, SignatureTimeStamp, SigAndRefsTimeStamp, RefsOnlyTimeStamp und ArchiveTimeStamp), Zertifikaten (CertificateValues) und Sperrinformationen (RevocationValues) vorgesehen. Für die langfristige Aufbewahrung von XAdES-basierten digitalen Signaturen existiert darüber hinaus ein internationales Standardprofil [ISO14533-2] und im XAdES Baseline Profile [ETSI TS 103 171], das im Durchführungsrechtakt [2015/1506/EU] referenziert wird, sowie in [ETSI EN 319 132-1] finden sich Festlegungen, die auf eine Steigerung der Interoperabilität abzielen.

Für die Behandlung von XML- und XAdES-basierten digitalen Signaturen sieht die vorliegende Richtlinie Folgendes vor:

(A5.1-2) XML-basierte digitale Signaturen müssen vor der Erstellung des initialen Archivzeitstempels geprüft werden. Hierbei müssen im Einklang mit [ISO14533-2] und [ETSI TS 103 171]⁹⁰ gemäß [2015/1506/EU] die für die Signatur- bzw. Siegelprüfung verwendeten Zertifikate, die nicht bereits im `ds:KeyInfo`-Element enthalten sind, in das `xades:CertificateValues`-Element der XAdES-Signatur bzw. des XAdES-Siegels eingefügt werden. In ähnlicher Weise müssen die bei der Signatur- bzw. Siegelprüfung herangezogenen Sperrinformationen, die nicht bereits im `ds:KeyInfo`-Element vorhanden sind, in das `xades:RevocationValues`-Element der XAdES-Signatur bzw. des XAdES-Siegels eingefügt werden. Alternativ, im Falle, dass die aufzubewahrenden Daten nicht verändert werden dürfen, müssen die o. g. Prüfdaten in den entsprechenden Unterelementen (z. B. `xaip:certificateValues`, `xaip:revocationValues` etc.) des Elements `xaip:credential` (vgl. Kap. 3.1.5) abgelegt und mit der korrespondierenden Signatur verlinkt werden.

5.1.3 Digitale PDF-Signaturen / PAdES

Für die langfristige Aufbewahrung von PAdES-basierten digitalen Signaturen existiert darüber hinaus ein internationales Standardprofil [ISO14533-3] und im PAdES Baseline Profile [ETSI TS 103 172], das im

⁸⁹ sowie zukünftig [ETSI EN 319 122-1]

⁹⁰ sowie zukünftig [ETSI EN 319 132-1]

Durchführungsrechtakt [2015/1506/EU] referenziert wird, sowie in [ETSI EN 319 142] finden sich Festlegungen, die auf eine Steigerung der Interoperabilität abzielen.

Für die Behandlung von digitalen PAdES-Signaturen sieht die vorliegende Richtlinie folgende Empfehlungen vor:

(A5.1-3) PAdES-basierte digitale Signaturen müssen vor der Erstellung des initialen Archivzeitstempels geprüft werden. Hierbei müssen im Einklang mit [ISO14533-3] und [ETSI TS 103 172]⁹¹ gemäß [2015/1506/EU] die für die Signatur- bzw. Siegelprüfung verwendeten Zertifikate und Sperrinformationen in entsprechende Elemente der PAdES-Signatur, z. B. DSS Dictionary bzw. Signature VRI Dictionary, abgelegt werden. Alternativ, im Falle, dass die aufzubewahrenden Daten nicht verändert werden dürfen, müssen die o.g. Prüfdaten in den entsprechenden Unterelementen (z. B. `xaip:certificateValues`, `xaip:revocationValues` etc.) des Elements `xaip:credential` (vgl. Kap. 3.1.5) abgelegt und mit der korrespondierenden Signatur verlinkt werden.

5.1.4 Digitale AdES-Signaturen im ASiC-Container

Für die Behandlung von digitalen AdES-Signaturen in ASiC-Containern sieht die vorliegende Richtlinie folgende Empfehlungen vor:

(A5.1-4) Die im ASiC-Container enthaltenen digitalen AdES-Signaturen müssen vor der Erstellung des initialen Archivzeitstempels des neuen Evidence-Records geprüft werden. Hierbei müssen im Einklang mit [ETSI TS 103 174]⁹² gemäß [2015/1506/EU] die für die Signatur- bzw. Siegelprüfung verwendeten Zertifikate und Sperrinformationen in entsprechende Elemente der jeweiligen Signatur bzw. des jeweiligen Siegels gemäß Abschnitt 5.1 abgelegt werden.

5.1.5 Sonstige Signatur- bzw. Siegelformate

Für die Behandlung von sonstigen digitalen Signaturen, bei denen es sich nicht um standardkonforme digitale PKCS#7/CMS/CAdES-, XML/XAdES- oder PDF/PAdES-Signaturen handelt, sieht die vorliegende Richtlinie folgende Empfehlungen vor:

(A5.1-5) Auch diese digitalen Signaturen sollen vor der Erstellung des initialen Archivzeitstempels geprüft werden. Die im Rahmen der Signatur- bzw. Siegelprüfung verwendeten Zertifikate und Sperrinformationen sollen in entsprechenden `<credential>`-Elementen abgelegt werden, so dass bei Bedarf auf diese Informationen zugegriffen werden kann. Hierbei sollen Zertifikate im `<certificateValues>`-Element und Sperrinformationen im `<revocationValues>`-Element innerhalb eines `<credential>`-Elementes abgelegt werden.

5.2 Zertifikatsformate

Unter den verschiedenen standardisierten Zertifikatsformaten sind für die vertrauenswürdige Langzeitspeicherung insbesondere die Public-Key- und Attribut-Zertifikate gemäß [X.509] bedeutsam.

Im Umfeld der vertrauenswürdigen Langzeitspeicherung werden Zertifikate insbesondere bei der Prüfung von (qualifizierten) digitalen Signaturen genutzt. Hierbei sollen entsprechende Prüfungen des Zertifikatsstatus (vgl. Abschnitt 5.3) durchgeführt werden.

(A5.2-1) Die hierbei gebildeten Zertifikatsketten müssen bis hin zu einer vertrauenswürdigen Wurzelinstanz oder einem Vertrauensanker gemäß der vom [TR-ESOR-PEPT] abgeleiteten und veröffentlichten Preservation Policy (PEP) des TR-ESOR-Produktes bzw. Bewahrungsdiensts geprüft werden und wie in Abschnitt 5.1 erläutert, in die digitale Signatur bzw. den XAIP-Container eingefügt werden.

⁹¹ Siehe zukünftig [ETSI EN 319 142-1].

⁹² Siehe zukünftig [ETSI EN 319 162-1].

5.3 Zertifikatsvalidierungsformate

(A5.3-1) Sofern die Gültigkeit der Zertifikate in den bei der Signatur- bzw. Siegelprüfung gebildeten Zertifikatspfaden nicht anderweitig sichergestellt ist, muss eine explizite Prüfung des Gültigkeitsstatus der Zertifikate erfolgen.

Hierfür soll das Online Certificate Status Protocol (OCSP) gemäß [RFC2560] bzw. [RFC6960] eingesetzt werden. Alternativ dazu kann das Server-Based Certificate Validation Protocol (SCVP) gemäß [RFC5055] eingesetzt werden, sofern sich dieses auf OCSP-Auskünfte der Zertifikatsaussteller stützt.

Sofern im Umfeld der vertrauenswürdigen Langzeitspeicherung Einfluss auf die für die verwendeten Zertifikate verfügbaren Mechanismen zur Prüfung des Zertifikatsstatus genommen werden kann, sollen keine Sperrlisten gemäß [RFC5280] eingesetzt werden, da hierdurch die Gültigkeit einer digitalen Signatur im Allgemeinen nicht zweifelsfrei bestimmt werden kann.

5.3.1 Online Certificate Status Protocol (OCSP / RFC 2560 / RFC 6960)

Beim Online Certificate Status Protocol (OCSP) gemäß [RFC2560] bzw. [RFC6960] werden aktuelle Auskünfte über den Zertifikatsstatus über ein Challenge-Response-Protokoll i. d. R. direkt beim Aussteller des Zertifikates angefordert. Da die Auskunft des Ausstellers praktisch zum Zeitpunkt der Signatur- bzw. Siegelprüfung erfolgt, ist die Aktualität der Sperrinformationen – anders als beispielsweise bei Einsatz von Sperrlisten – sichergestellt. In Umgebungen mit einer hohen Anzahl an OCSP-Transaktionen können die Empfehlungen gemäß [RFC5019] beachtet werden.

(A5.3-2) OCSP-basierte Statusinformationen müssen wie in Abschnitt 5.1 beschrieben in die entsprechende digitale Signatur bzw. in den XAIP-Container eingefügt werden.

5.3.2 Server-Based Certificate Validation Protocol (SCVP / RFC 5055)

Beim Server-Based Certificate Validation Protocol (SCVP) gemäß [RFC5055] können die unter Umständen sehr komplexen Aufgaben, die im Rahmen der Validierung eines Zertifikates notwendig sind, an einen hierfür spezialisierten Dienst ausgelagert werden. Dies ist beispielsweise dann sinnvoll, wenn eine komplexe Signatur- bzw. Siegelprüfung von einer Komponente mit geringer Rechenleistung (z. B. mobiles Endgerät) durchgeführt werden muss. Für diese Zwecke schickt der SCVP-Client eine Anfrage an den SCVP-Server, indem beispielsweise das zu prüfende Zertifikat übermittelt oder anderweitig spezifiziert ist. Daraufhin bildet und prüft der SCVP-Server den kompletten Zertifikatspfad, wobei er sich auf die primären – typischerweise vom Aussteller des Zertifikates bereit gestellten – Sperrinformationen in Form von OCSP-Responses oder Sperrlisten stützt.

Beim Aufruf des SCVP-Servers soll im wantBack-Element (im query-Element des CVRequest) durch Angabe der OID id-swb-pkc-revocation-info klargestellt werden, dass entsprechende Sperrinformationen zurückgeliefert werden sollen.

(A5.3-3) Die in der CVResponse (im replyObjects/replyWantBacks/revInfoWantBack/RevocationInfos) zurückgelieferten Sperrinformationen sollen extrahiert und wie in Abschnitt 5.1 beschrieben in die entsprechende digitale Signatur bzw. in den XAIP-Container eingefügt werden.

5.4 Zeitstempel

Für den Bezug von (qualifizierten) Zeitstempeln soll insbesondere⁹³ das Time-Stamp Protocol (TSP) gemäß [RFC3161], [RFC5816], [RFC5652] und [ETSI EN 319 422] genutzt werden.

(A5.4-1) Damit die spätere Prüfung der Zeitstempel leicht möglich wird, soll im `SignedData`-Container nur die Signatur bzw. das Siegel des Zeitstempeldienstes enthalten sein und es muss nach dem Erstellen eines Zeitstempels und dem Ablauf der so genannten „Grace Period“ (vgl. [ETSI TS 101 733], Abschnitt 4.4.2) automatisch die Prüfung des Zeitstempels erfolgen. Hierbei müssen im Einklang mit [ETSI TS 103 173], das im Durchführungsrechtsakt [2015/1506/EU]⁹⁴ referenziert wird, alle bei der Prüfung verwendeten Zertifikate im Element `SignedData.certificates` abgelegt werden. In entsprechender Weise müssen auch die für die Prüfung herangezogenen Sperrinformationen im Element `SignedData.crls` abgelegt werden, wobei Sperrlisten in der Option `SignedData.crls.crl` und OCSP-Responses in der Option `SignedData.crls.other` abzulegen sind.

5.5 Beweisdatenbericht (Evidence Record gemäß RFC 4998 /RFC 6283)

Ein Evidence Record ist gemäß dem Evidence Record Syntax (ERS) Standard der IETF [RFC4998] bzw. [RFC6283]⁹⁵ eine Dateneinheit, mit der die Existenz gespeicherter Daten und Dokumente zu einem definierten Zeitpunkt technisch nachgewiesen werden kann. Sie enthält kryptographische Beweisdaten, mit denen die Integrität und Authentizität elektronisch gespeicherter Daten und Dokumente jederzeit verifiziert werden können. Technisch basiert der ERS-Standard auf dem Ansatz, dass kryptographische Prüfsummen (Hashwerte) der Archivdatenobjekte (XAIP-Dokumente) als kryptographisch eindeutige Repräsentanten der aufzubewahrenden Daten bei der Ablage im Archivsystem in einem Hashbaum (nach Merkle [MER 1980]) angeordnet werden und die Wurzel des Hashbaumes für den Nachweis der Integrität mit einem qualifizierten Zeitstempel gesichert („versiegelt“) werden (siehe auch Anlage [TR-ESOR-M.3]). Dieser erste Zeitstempel wird gemäß dem ERS-Standard [RFC4998] bzw. [RFC6283] auch als initialer Archivzeitstempel bezeichnet.

Vertrauensanker für den Archivzeitstempel und damit für eine rechtskonforme Signatur- bzw. Siegel- bzw. Zeitstempelerneuerung gemäß § 15 des Vertrauensdienstegesetzes [VDG] ist der qualifizierte Zeitstempel. Seine Datenstruktur soll die Anforderungen des „Time-Stamp Protocol (TSP)“ [RFC3161] und der „Cryptographic Message Syntax (CMS)“ gemäß [RFC5816] bzw. [RFC5652] und [ETSI EN 319 422] sowie [ISO14533-1] erfüllen.

Bei einer notwendigen Signatur- bzw. Siegel- bzw. Zeitstempelerneuerung, die ausreichend ist, sofern nur das eingesetzte digitale Signaturverfahren seine Sicherheitseignung einzubüßen droht, aber der genutzte Hashalgorithmus weiterhin geeignet ist, umschließt ein neuer Archivzeitstempel den Hashwert des ursprünglichen Zeitstempels in einem neu zu bildenden Hashbaum mit einem neuen qualifizierten Zeitstempel als Abschluss, so dass eine sichere und nachweisliche, chronologische Beweiskette aus kryptographisch miteinander verknüpften Archivzeitstempeln entsteht. Der hierdurch entstehende Evidence Record enthält im bereits vorher existierenden `ArchiveTimeStampChain`-Element gemäß [RFC4998] bzw. [RFC6283] ein zusätzliches `ArchiveTimeStamp`-Element.

(A5.5-1) Sofern (auch) die Sicherheitseignung des eingesetzten Hashalgorithmus bedroht ist, muss eine Hashbaum-Erneuerung vorgenommen werden. Hierbei wird das Archivdatenobjekt mit einem geeigneten Algorithmus gehasht und ein *neues* `ArchiveTimeStampChain`-Element mit einem

⁹³ Darüber hinaus empfiehlt sich in Umgebungen mit einer hohen Anzahl an notwendigen individuellen Zeitstempeln die Anwendung der darauf aufbauenden und in [RFC4998] bzw. [RFC6283] standardisierten Hashbaum-basierten Archivzeitstempel (`ArchiveTimeStamp`) oder die in [HK 09] erläuterte Konstruktion der so genannten „Intervall-qualifizierten Zeitstempel“.

⁹⁴ sowie zukünftig [ETSI EN 319 122-1]

⁹⁵ RFC 4998 muss, RFC 6283 kann unterstützt werden.

entsprechenden `ArchiveTimeStamp`-Element in das `ArchiveTimeStampSequence`-Element eingefügt. Weitere Informationen hierzu finden sich auch in [RFC4998] bzw. [RFC6283].

Der technische Nachweis der Aufrechterhaltung der Integrität und deshalb ggf. der Authentizität der im elektronischen Langzeitspeicher abgelegten Daten erfolgt dann, neben der Vorlage der eigentlichen Archivdaten und der zugehörigen, gültigen Zertifikate vorhandener digitaler Signaturen, vor allem über den Nachweis der Integrität der kryptographischen Repräsentanten der Archivdatenobjekte, d. h. der Hashwerte und Archivzeitstempel.

Der ERS-Standard spezifiziert für diese Zwecke einen so genannten „Beweisdatenbericht“ (Evidence Record). Dieser Beweisdatenbericht enthält insbesondere eine Folge von Archivzeitstempeln mit denen die Integrität und Authentizität der Archivdatenobjekte nachgewiesen werden kann. Ein Archivzeitstempel enthält wiederum alle notwendigen Daten aus dem Hashbaum (`reducedHashTree`), die für die Prüfung der Zugehörigkeit des Archivdatenobjektes zum Hashbaum notwendig sind. Die Wurzel des Hashbaumes wird mit einem qualifizierten Zeitstempel versehen (siehe auch Anlage [TR-ESOR-M.3]).

(A5.5-2) Die Erzeugung eines ERS gem. [RFC4998] muss unterstützt werden. Die Erzeugung eines ERS gem. [RFC6283] kann unterstützt werden. Der Aufbau und die Belegung der Evidence Records ist dem Anhang [TR-ESOR-ERS] zu entnehmen. Im nachfolgenden Text wird ein grober Überblick gegeben.

5.5.1 EvidenceRecord gemäß RFC 4998

Ein Evidence Record gemäß [RFC4998] ist eine folgendermaßen definierte ASN.1-Struktur:

```
EvidenceRecord ::= SEQUENCE {
    version                INTEGER { v1(1) },
    digestAlgorithms        SEQUENCE OF AlgorithmIdentifier,
    cryptoInfos             [0] CryptoInfos OPTIONAL,
    encryptionInfo         [1] EncryptionInfo OPTIONAL,
    archiveTimeStampSequence ArchiveTimeStampSequence
}
CryptoInfos ::= SEQUENCE SIZE (1..MAX) OF Attribute
```

Das `version` Feld (bspw. `v1`) beschreibt die aktuelle Version des ERS-Standards.

Das Feld `digestAlgorithms` enthält eine Aufzählung sämtlicher Hashalgorithmen, mit denen Hashwerte über die gespeicherten Daten während des Aufbewahrungszeitraums erzeugt wurden.

Das optionale Feld `cryptoInfos` ermöglicht den Transport von Daten, die für die Validierung der im Feld `archiveTimeStampSequence` enthaltenen Informationen benötigt werden.

Das ebenfalls optionale Feld `encryptionInfo` enthält notwendige Informationen zum korrekten Umgang mit verschlüsselten Inhalten.

Das Feld `archiveTimeStampSequence` enthält eine Folge verketteter Archivzeitstempelketten (`ArchiveTimeStampChain`-Elemente), die im Verlaufe des Aufbewahrungszeitraums über die gespeicherten Daten gebildet wurden. Ein `ArchiveTimeStampChain`-Element beinhaltet eine Sequenz von zumindest einem oder mehreren Elementen vom Typ `ArchiveTimeStamp`, die aufsteigend nach der Zeit des beinhalteten Zeitstempels sortiert sind.

```
ArchiveTimeStampSequence ::= SEQUENCE OF ArchiveTimeStampChain
ArchiveTimeStampChain ::= SEQUENCE OF ArchiveTimeStamp
```

`ArchiveTimeStampChain` und `ArchiveTimeStampSequence` sind zeitlich geordnet nach dem Zeitpunkt der abschließenden Zeitstempel. Innerhalb einer Archivzeitstempelkette haben alle reduzierten Hashbäume denselben Hashalgorithmus zur Grundlage.

Ein Archivzeitstempel ist nach dem ERS-Standard der IETF wie folgt definiert:

```
ArchiveTimeStamp ::= SEQUENCE {
    digestAlgorithm      [0] AlgorithmIdentifier OPTIONAL,
    attributes           [1] Attributes OPTIONAL,
    reducedHashtree      [2] SEQUENCE OF PartialHashtree OPTIONAL,
    timeStamp            ContentInfo -- TimeStampToken nach [RFC3161]
}
TimeStampToken ::= ContentInfo
-- [RFC3161]
-- contentType is id-signedData ([RFC3852])
-- content is SignedData ([RFC3852])
```

Das Feld `digestAlgorithm` identifiziert den verwendeten Hashalgorithmus. Ist das Feld nicht vorhanden, geht der ERS-Standard davon aus, dass für die Hashwertbildung der Hashalgorithmus des Zeitstempels verwendet wurde.

- Das optionale Feld `attributes` kann zusätzliche Informationen zu den für die Signatur- bzw. Siegelerneuerung oder Anwendung der Archivzeitstempel angewendeten Regeln aufnehmen.

Das Feld `reducedHashtree` enthält sämtliche Hashwerte, die für die mathematische Verifikation der Hashwert-Knoten, in die der ursprüngliche Hashwert des Archivdatenobjektes inklusive des abschließenden qualifizierten Zeitstempels eingeflossen ist, benötigt werden.

Das Feld `timeStamp` enthält einen qualifizierten Zeitstempel, der unter Verwendung einer fortgeschrittenen elektronischen Signatur bzw. Siegel erstellt wurde, als kryptographische Bestätigung der Integrität der mit dem Evidence Record zurückgelieferten Daten.

5.5.2 <EvidenceRecord> gemäß RFC 6283

Ein <EvidenceRecord>-Element gemäß [RFC6283] ist vom Typ **EvidenceRecordType**, der folgende Struktur besitzt:

```
<xs:element name="EvidenceRecord" type="EvidenceRecordType" />

<xs:complexType name="EvidenceRecordType">
  <xs:sequence>
    <xs:element name="EncryptionInformation" type="EncryptionInfo" minOccurs="0"/>
    <xs:element name="SupportingInformationList" type="SupportingInformationType"
      minOccurs="0" />
    <xs:element name="ArchiveTimeStampSequence"
      type="ArchiveTimeStampSequenceType"/>
  </xs:sequence>
  <xs:attribute name="Version" type="xs:decimal" use="required" fixed="1.0"/>
</xs:complexType>
```

<EncryptionInformation> [optional]

Das <EncryptionInfo>-Element kann bei Bedarf Informationen für die Behandlung von verschlüsselten Daten enthalten. Weitere Details zum **EncryptionInfo**-Typ finden sich in [RFC6283].

<SupportingInfoList> [optional]

Das <SupportingInfoList>-Element kann weitere unterstützende Informationen, wie beispielsweise Informationen zur Sicherheitseignung von kryptographischen Algorithmen gemäß [RFC5698] enthalten. Weitere Details zum **SupportingInformationType** finden sich in [RFC6283].

<ArchiveTimeStampSequence> [required]

Das <ArchiveTimeStampSequence>-Element enthält ähnlich wie beim ASN.1-basierten Evidence Record gemäß [RFC4998] eine Folge <ArchiveTimeStampChain>-Elementen, die wiederum eine Folge von <ArchiveTimeStamp>-Elementen enthält. Weitere Details zum **ArchiveTimeStampSequenceType** finden sich in [RFC6283].

6 Anhang – Umgang mit digitalen Signaturtechniken innerhalb von (L)XAIP

Gem. eIDAS-VO und dem dazugehörigen Durchführungsrechtsakt [2015/1506/EU] müssen aktuell folgende vier Arten der elektronischen Signaturen / Siegel⁹⁶ EU-weit anerkannt werden:

- CAdES, gem. CAdES Baseline Profile [ETSI TS 103 173], v2.2.1 – vgl. Kap. 6.1,
- XAdES, gem. XAdES Baseline Profile [ETSI TS 103 171], v.2.1.1 – vgl. Kap. 6.2,
- PAdES, gem. PAdES Baseline Profile [ETSI TS 103172], v.2.2.2 – vgl. Kap. 6.3,
- ASiC, gem. ASiC Baseline Profile [ETSI TS 103 174], v.2.2.1 – vgl. Kap. 6.4.

Außer den o.g. Signaturen/Siegel können auch weitere beweisrelevante kryptographische Artefakte innerhalb eines XAIP⁹⁷ abgelegt werden. Dazu gehören:

- CAdES, gem. CAdES Baseline Profile [ETSI EN 319 122-1], v1.1.1 und höher – vgl. Kap. 6.1,
- XAdES, gem. XAdES Baseline Profile [ETSI EN 319 132-1], v.1.1.1 und höher – vgl. Kap. 6.2,
- PAdES, gem. PAdES Baseline Profile [ETSI EN 319 142-1], v.1.1.1 und höher – vgl. Kap. 6.3,
- ASiC, gem. ASiC Baseline Profile [ETSI EN 319 162-1], v.1.1.1 und höher – vgl. Kap. 6.4.
- Zeitstempel, gem. [RFC3161] – vgl. Kap. 6.5,
- Evidence Records, gem. [RFC4998] oder [RFC6283] – vgl. Kap. 6.6.

HINWEIS 12

Grundsätzlich wird bei der Ablage der signierten Daten und Signaturen innerhalb von XAIP folgende Grundregel stets angewandt:

- *Handelt sich dabei um die abgesetzte (detached) Signatur, so werden die signierten Daten innerhalb der Dataobjects-Section und die Signatur innerhalb der Credentials-Section abgelegt,*
- *Im Falle von verbundenen (attached) Signaturen werden die Daten in Abhängigkeit dessen, ob die Signatur oder die signierten Daten die äußere Hülle bilden abgelegt. Bilden die signierten Daten die äußere Hülle, so werden die Daten (hier Daten zusammen mit Signatur) innerhalb Dataobjects-Section abgelegt, andererseits (die Signatur bildet die äußere Hülle) sind die Daten innerhalb der Credentials-Section abzulegen.*

Innerhalb eines XAIP gem. [TR-ESOR-F] können die Signaturen, abhängig von deren Ausprägung, an diversen Stellen abgelegt werden, das sind insbesondere:

- `xaip:credential` – primärer Ort für die Ablage beweisrelevanter Daten (Credentials-Section),
- `xaip:metaDataObject` – Zielort für die Ablage von Metadaten, die ggf. auch signiert werden können (Metadata-Section),
- `xaip:dataObject` – Ablage von Inhaltsdaten, die ggf. auch signiert werden können (Dataobjects-Section).

Folgende Kapitel stellen kurz das Vorgehen bei der Ablage der einzelnen zuvor erwähnten Signaturarten dar.

HINWEIS 13

Grundsätzlich könnten auch weitere Varianten der Signaturen, sowie Konstellationen, wie die Signaturen und die geschützten Daten dargestellt werden, vorkommen. Diese können anhand der in diesem Dokument dargestellten Ablageregeln entsprechend einsortiert werden.

6.1 Umgang mit CAdES

Es können grundsätzlich zwei Arten von CAdES unterschieden werden:

⁹⁶ Sofern dies nicht explizit anders angegeben ist, gelten die Aussagen in diesem Dokument, die für **elektronische Signaturen** getroffen werden, stets auch für **elektronische Siegel**.

⁹⁷ Mit dem Begriff **XAIP** wird, soweit nicht explizit anders dargestellt, sowohl ein klassisches **XAIP** wie auch ein **LXAIP** gemeint.

1. Verbundene (attached⁹⁸) CAdES – die äußere Hülle stellt die Signatur selbst dar und beinhaltet die signierten Daten (nur eine Datei),
2. Abgesetzte (detached⁹⁹) CAdES – die Signatur selbst beinhaltet keine Daten, diese werden in einer separaten Datei abgelegt (zwei Dateien).

(A6.1-1) Im ersten Fall (**verbundene CAdES**) MUSS die Signatur innerhalb der Credentials-Section abgelegt werden. Im Falle einer **abgesetzten CAdES** MUSS die Signatur in der Credentials-Section und die signierten Daten im dataObjectsSection-Element untergebracht werden. Die Zusammengehörigkeit MUSS über die entsprechende Eingabe im Attribut `relatedObjects` des entsprechenden Elements `xaip:credential` erfolgen.

6.2 Umgang mit XAdES

Die XAdES-Signaturen können grundsätzlich wie folgt unterschieden werden:

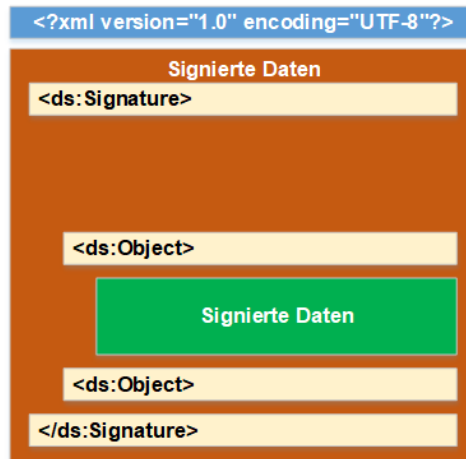


Abbildung 15: Schematische Darstellung einer enveloping XAdES.

1. Abgesetzte (detached) XAdES – die signierten Daten und die Signatur selbst liegen in zwei (oder mehr) getrennten Dateien¹⁰⁰,
2. Enveloping XAdES – die Signatur stellt die äußere Hülle dar, die signierten Daten sind darin eingebettet (nur eine Datei),
3. Enveloped XAdES – die signierten Daten stellen die äußere Hülle dar, die Signatur ist darin eingebettet (nur eine Datei).

⁹⁸ Angelehnt an die Situation bei XML-Signaturen wird hier bisweilen auch von einer „enveloping signature“ gesprochen.

⁹⁹ In [RFC 2315] wird bei der abgesetzten Signatur auch von "external signatures" gesprochen.

¹⁰⁰ Der Sonderfall sog. „detached sibling signature“ MUSS entsprechend den Ablageregeln für die Enveloped XAdES behandelt werden.

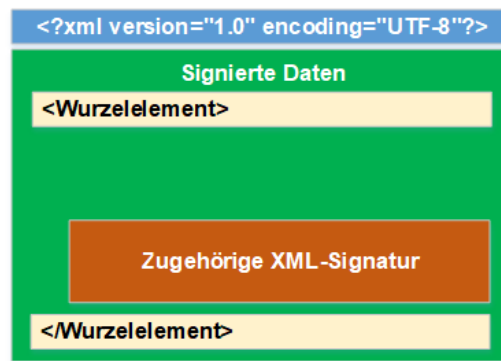


Abbildung 16: Schematische Darstellung einer enveloped XAdES.

(A6.2-1) Im ersten Fall (**abgesetzte XAdES**) MÜSSEN die Daten innerhalb der Dataobjects-Section und die Signatur innerhalb des Credentials-Section abgelegt werden. Die Zugehörigkeit der signierten Daten und der Signatur MUSS mit Hilfe der Referenzen innerhalb des Attributs relatedObjects des entsprechenden Elements `xaip:credential` hergestellt werden.

HINWEIS 14

Grundsätzlich ist es möglich, die XML-Daten sowohl die signierten Daten (vgl. Abbildung 17) als auch die Signatur selbst (vgl. Abbildung 18) entsprechend als XML innerhalb vom XAIP abzulegen.

Der Umgang mit XML-Signaturen (insbesondere signierten XML-Daten) beim Einbetten in ein anderes XML-Dokument (hier (L)XAIP) kann allerdings einige Herausforderungen mit sich bringen. Beispielsweise dürfen die einzubettenden Daten keine Präambel beinhalten und das Encoding der beiden Inhalte muss gleich sein, etc.

Aus dem o. g. Grund wird es empfohlen, die XML-Daten analog zu binären Daten zu behandeln und diese unverändert in dem Elementen `xaip:binaryData` respektive abzulegen.

Analog dazu wird es empfohlen im Falle von XML-Signaturen, diese im Element `dss:Base64Signature`-Element innerhalb des `dss:SignatureObject`-Elements abzulegen.

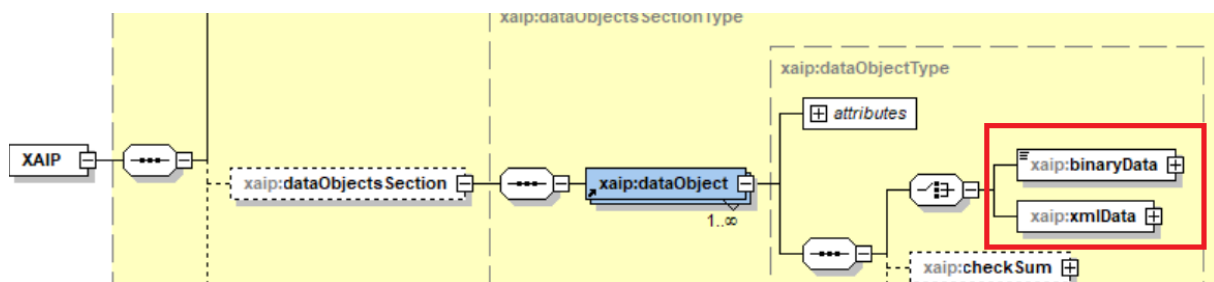


Abbildung 17: Möglichkeiten der Ablage der XML-Inhaltsdaten innerhalb vom (L)XAIP.

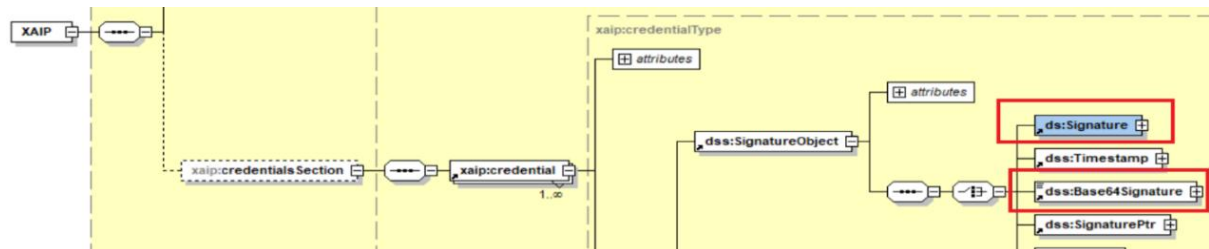


Abbildung 18: Möglichkeiten der Ablage der XML-Signaturen innerhalb vom (L)XAIP.

(A6.2-2) Im Falle von **enveloping XAdES** MUSS der Inhalt innerhalb der Credentials-Section abgelegt werden.

Die **enveloped XAdES** MUSS innerhalb des Dataobjects-Section abgelegt werden.

6.3 Umgang mit PAdES

(A6.3-1) Eine **PAdES** wird grundsätzlich in das geschützte Dokument eingebettet (das geschieht unabhängig davon, ob es sich um eine sichtbare oder nicht sichtbare Signatur handelt). Aus diesem Grund MÜSSEN signierte PDF-Dokumente (genau wie unsignierte) in der Dataobjects-Section abgelegt werden.

6.4 Umgang mit ASiC

(A6.4-1) Ein **ASiC** in seiner Container-Funktion beinhaltet sowohl die Daten als auch Signaturen, Evidence Records oder Zeitstempel. Unabhängig von dem Inhalt wird es empfohlen, ein ASiC (als eines der im Durchführungsrechtsakt [2015/1506/EU] erwähnten Formate) innerhalb der Credentials-Section abzulegen.

6.5 Umgang mit dem Zeitstempel

Die Zeitstempel können in unterschiedlichen Konstellationen verwendet werden:

1. Abgesetzter Zeitstempel – eine separate Datei für den Zeitstempel (i. d. R. ein gem. [RFC3161] ausgestellte Zeitstempel)
2. In eine Signatur eingebetteter Zeitstempel – keine separate Datei, der Zeitstempel ist als Teil der Signatur abgelegt,
3. In Evidence Record eingebettete Zeitstempel – es handelt sich um einen Archivzeitstempel, der ein integraler Teil des Evidence Records ist.

(A6.5-1) Im Falle eines **abgesetzten Zeitstempels** MUSS dieser analog wie eine abgesetzte CAdES oder XAdES innerhalb der Credentials-Section abgelegt werden. Die Zugehörigkeit zu den geschützten Daten MUSS über das Attribut `relatedObjects` des entsprechenden Elements `xaip:credential` sichergestellt werden.

(A6.5-2) Wenn der **Zeitstempel in eine Signatur eingebettet** ist (z. B. als ein Attribut eines CAdES oder eine XAdES, oder innerhalb des ASiC abgelegt), so MUSS gem. der Empfehlung für die Signatur (vgl. Kap. 6.1, 6.2, 6.3, 6.4), in der er eingebettet ist, vorgegangen werden.

(A6.5-3) Im letzten Fall (der Zeitstempel fungiert als ein **Archivzeitstempel innerhalb eines Evidence Records**) MÜSSEN die Ablageregeln des Evidence Records ihre Anwendung finden (vgl. hierzu Kap. 6.6).

6.6 Umgang mit dem Evidence Record

Grundsätzlich können Evidence Records gem. [RFC4998] und [RFC6283] betrachtet werden. Im weiteren Verlauf wird stellvertretend von Evidence Record gesprochen, wobei damit die beiden Typen gemeint sind.

Die Evidence Records werden i.d.R. wie folgt verwendet:

1. Abgesetzter Evidence Record – eine getrennte Datei zu den abgesicherten Inhaltsdaten,
2. Abgesetzter Evidence Record, der sich auf eine Version eines XAIP bezieht,
3. Evidence Records eingebettet in ein ASiC – äußere Hülle stellt hier ein ASiC dar, der Evidence Record ist darin enthalten,
4. Evidence Record eingebettet in eine CAdES (vgl. [RFC4998] Appendix A) – ein Evidence Record als ein nicht-signierte Attribut einer CAdES gemäß [ETSI TS 119 122-3]
5. Evidence Record eingebettet in eine XAdES – ein Evidence Record als ein nicht-signiertes Attribut einer XAdES gemäß [ETSI TS 119 132-3].

(A6.6-1) Ein **abgesetzter Evidence Record** MUSS innerhalb der Credentials-Section im dafür dedizierten Element `xaip:evidenceRecord` abgelegt werden. Mit Hilfe des Attributs `relatedObjects`¹⁰¹ des zugehörigen Elternelements `xaip:credential` oder der Attribute `AOID` und `VersionID` des Elements `xaip:evidenceRecord` (bevorzugte Variante) MUSS die Zugehörigkeit der geschützten Daten signalisiert werden.

Ein **abgesetzter Evidence Record mit einem Bezug auf ein XAIP** MUSS innerhalb Credentials-Section im Element `xaip:evidenceRecord` abgelegt werden. Die Attribute `AOID` und `VersionID` MÜSSEN dabei verwendet werden, um die XAIP und die konkrete Version, auf die sich der Evidence Record bezieht, zu kennzeichnen.

¹⁰¹ Verweis auf die zugehörige VersionID.

(A6.6-2) Ein **Evidence Record**, der **innerhalb eines ASiC** mitgeführt wird, MUSS gem. den Ablageregeln für den ASiC behandelt werden (vgl. hierzu Kap. 6.4).

Ein **in eine CAdES eingebetteter Evidence Record** MUSS gem. den Ablageregeln für die CAdES behandelt werden (vgl. hierzu Kap. 6.1).

Ein **in eine XAdES eingebetteter Evidence Record** MUSS gem. den Ablageregeln für die XAdES behandelt werden (vgl. hierzu Kap. 6.2).

7 Anhang - Umgang mit signierten Metadaten

Grundsätzlich soll der Umgang mit den signierten Metadaten die im Anhang 6 definierten Regeln befolgen.

HINWEIS 15

Im Falle der abgesetzten Signaturen (z. B. detached CAdES oder detached XAdES etc.) sollen die Metadaten entsprechend in der `metaDataSection` und zugehörige Signatur in der `credentialsSection` abgelegt werden (vgl. auch hierzu HINWEIS 12). Die `credential-Elemente` sollen mit Hilfe des `relatedObjects`-Attributs entsprechend auf die zugehörigen Instanzen der `metaDataObject-Elemente` verweisen.

HINWEIS 16

Im Falle der verbundenen Signaturen, soll sich das Vorgehen entsprechend daran ausrichten, ob die äußere Hülle die Signatur (z. B. attached CAdES oder envelopping XAdES) oder die Metadaten (z. B. envelopped XAdES oder PAdES) bilden:

- *Signatur – das Objekt soll sowohl als ein `credential-Element` als auch als ein `metaDataObject-Element` abgelegt werden. Eine Verknüpfung analog dem HINWEIS 14 soll erfolgen.*
- *Metadaten – das Objekt soll ausschließlich als ein `metaDataObject-Element` abgelegt werden.*

8 Anhang – XML-Schema-Definition

```

<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xaip="http://www.bsi.bund.de/tr-esor/xaip"
  xmlns:xs="http://www.w3.org/2001/XMLSchema"
    xmlns:ds="http://www.w3.org/2000/09/xmldsig#"
    xmlns:xades="http://uri.etsi.org/01903/v1.3.2#"
    xmlns:ers="urn:ietf:params:xml:ns:ers"
    xmlns:vr="urn:oasis:names:tc:dss-x:1.0:profiles:verificationreport:schema#"
    xmlns:dss="urn:oasis:names:tc:dss:1.0:core:schema"
    xmlns:saml="urn:oasis:names:tc:SAML:2.0:assertion"
    xmlns:xmime="http://www.w3.org/2005/05/xmlmime"
    targetNamespace="http://www.bsi.bund.de/tr-esor/xaip"
    elementFormDefault="qualified" attributeFormDefault="unqualified"
    version="1.3.0">
  <xs:import namespace="http://www.w3.org/2000/09/xmldsig#"
    schemaLocation="./deps/xmldsig-core-schema.xsd"/>
  <xs:import namespace="http://uri.etsi.org/01903/v1.3.2#"
    schemaLocation="./deps/XAdES01903v132-201601.xsd"/>
  <xs:import namespace="urn:ietf:params:xml:ns:ers"
    schemaLocation="./deps/xml-ers-rfc6283.xsd"/>
  <xs:import namespace="urn:oasis:names:tc:dss-x:1.0:profiles:verificationreport:schema#"
    schemaLocation="./deps/oasis-dssx-1.0-profiles-verification-report-cs1.xsd"/>
  <xs:import namespace="urn:oasis:names:tc:dss:1.0:core:schema"
    schemaLocation="./deps/oasis-dss-core-schema-v1.0-os.xsd"/>
  <xs:import namespace="urn:oasis:names:tc:SAML:2.0:assertion"
    schemaLocation="./deps/saml-schema-assertion-2.0.xsd"/>
  <xs:import namespace="http://www.w3.org/2005/05/xmlmime"
    schemaLocation="./deps/xmlmime.xsd"/>
  <!--=====-->
  <!-- Version 1.3.0 vom 14.03.2022 -->
  <!--=====-->
  <!-- XAIP gesamt -->
  <!--=====-->
  <xs:element name="XAIP" type="xaip:XAIPType"/>
  <xs:complexType name="XAIPType">
    <xs:sequence>
      <xs:element name="packageHeader" type="xaip:packageHeaderType"/>
      <xs:element name="metaDataSection" type="xaip:metaDataSectionType" minOccurs="0"/>
      <xs:element name="dataObjectsSection" type="xaip:dataObjectsSectionType"
        minOccurs="0"/>
      <xs:element name="credentialsSection" type="xaip:credentialsSectionType"
        minOccurs="0"/>
    </xs:sequence>
    <xs:attribute name="XAIPVersion" use="required" fixed="1.3.0"/>
  </xs:complexType>
  <!--=====-->
  <!-- packageHeaderType -->
  <!--=====-->
  <xs:complexType name="packageHeaderType">
    <xs:sequence>
      <xs:element name="AOID" type="xs:string" minOccurs="0"/>
      <xs:element name="packageInfo" type="xs:string" minOccurs="0"/>
      <xs:element name="versionManifest" type="xaip:versionManifestType"
        maxOccurs="unbounded"/>
      <xs:element ref="ds:CanonicalizationMethod" minOccurs="0"/>
      <xs:element name="extension" type="xaip:extensionType" minOccurs="0"/>
    </xs:sequence>
    <xs:attribute name="packageID" type="xs:ID" use="required"/>
  </xs:complexType>
  <xs:complexType name="extensionType">
    <xs:sequence>
      <xs:any processContents="lax" minOccurs="0" maxOccurs="unbounded"/>
    </xs:sequence>
  </xs:complexType>
  <xs:complexType name="versionManifestType">
    <xs:sequence>
      <xs:element name="versionInfo" type="xs:string" minOccurs="0"/>
    </xs:sequence>
  </xs:complexType>

```

```

<xs:element name="preservationInfo" type="xaip:preservationInfoType"/>
<xs:element name="submissionInfo" type="xaip:submissionInfoType" minOccurs="0"/>
<xs:element name="packageInfoUnit" type="xaip:packageInfoUnitType"
  minOccurs="0" maxOccurs="unbounded"/>
<xs:element ref="xaip:idAssignmentList" minOccurs="0"/>
<xs:element name="extension" type="xaip:extensionType" minOccurs="0"/>
</xs:sequence>
<xs:attribute name="VersionID" type="xs:ID" use="required"/>
</xs:complexType>
<xs:complexType name="preservationInfoType">
  <xs:sequence>
    <xs:element name="retentionPeriod" type="xs:date"/>
    <xs:element name="status" type="xs:string" minOccurs="0"/>
    <xs:element name="extension" type="xaip:extensionType" minOccurs="0"/>
  </xs:sequence>
</xs:complexType>
<xs:complexType name="submissionInfoType">
  <xs:sequence>
    <xs:element name="clientID" type="saml:NameIDType"/>
    <xs:element name="submissionUnit" type="saml:NameIDType" minOccurs="0"/>
    <xs:element name="submissionAuthor" type="saml:NameIDType" minOccurs="0"/>
    <xs:element name="submissionTime" type="xs:dateTime" minOccurs="0"/>
    <xs:element name="extension" type="xaip:extensionType" minOccurs="0"/>
  </xs:sequence>
</xs:complexType>
<xs:complexType name="packageInfoUnitType">
  <xs:sequence>
    <xs:element name="unitType" type="xs:string" minOccurs="0"/>
    <xs:element name="textInfo" type="xs:string" minOccurs="0"/>
    <xs:element name="protectedObjectPointer" type="xs:IDREF" maxOccurs="unbounded"/>
    <xs:element name="unprotectedObjectPointer" type="xs:IDREF" minOccurs="0"
      maxOccurs="unbounded"/>
    <xs:element name="packageInfoUnit" type="xaip:packageInfoUnitType" minOccurs="0"
      maxOccurs="unbounded"/>
    <xs:element name="extension" type="xaip:extensionType" minOccurs="0"/>
  </xs:sequence>
  <xs:attribute name="packageUnitID" type="xs:ID" use="required"/>
</xs:complexType>
<!--=====-->
<!-- MetadataSection -->
<!--=====-->
<xs:complexType name="metaDataSectionType">
  <xs:sequence>
    <xs:element ref="xaip:metaDataObject" maxOccurs="unbounded"/>
  </xs:sequence>
</xs:complexType>
<xs:element name="metaDataObject" type="xaip:metaDataObjectType"/>
<xs:complexType name="metaDataObjectType">
  <xs:sequence>
    <xs:choice>
      <xs:element ref="xaip:binaryMetaMeta">
      <xs:element name="xmlMetaMeta" type="dss:AnyType"/>
    </xs:choice>
    <xs:element ref="xaip:checkSum" minOccurs="0"/>
  </xs:sequence>
  <xs:attribute name="metaDataID" type="xs:ID" use="required"/>
  <xs:attribute name="relatedObjects" type="xs:IDREFS" use="required"/>
  <xs:attribute name="category" type="xs:string"/>
  <xs:attribute name="classification" type="xs:string"/>
  <xs:attribute name="type" type="xs:string"/>
</xs:complexType>
<xs:element name="binaryMetaMeta" type="xaip:binaryMetaMetaType"/>
<xs:complexType name="binaryMetaMetaType" xmlns:expectedContentTypes="*/*">
  <xs:simpleContent>
    <xs:extension base="xs:base64Binary">
      <xs:attribute name="MimeType" type="xs:string" use="optional"/>
    </xs:extension>
  </xs:simpleContent>
</xs:complexType>
<!--=====-->
<!-- DataObjectsSection -->

```

```

<!--=====-->
<xs:complexType name="dataObjectsSectionType">
  <xs:sequence>
    <xs:element ref="xaip:dataObject" maxOccurs="unbounded"/>
  </xs:sequence>
</xs:complexType>
<xs:element name="binaryData" type="xaip:binaryDataType"/>
<xs:complexType name="binaryDataType" xmlns:mime="http://www.w3.org/2003/11/rfc2045" mime:expectedContentTypes="*/*">
  <xs:simpleContent>
    <xs:extension base="xs:base64Binary">
      <xs:attribute name="MimeType" type="xs:string" use="optional"/>
    </xs:extension>
  </xs:simpleContent>
</xs:complexType>
<xs:element name="dataObject" type="xaip:dataObjectType"/>
<xs:complexType name="dataObjectType">
  <xs:sequence>
    <xs:choice>
      <xs:element ref="xaip:binaryData"/>
      <xs:element name="xmlData" type="dss:AnyType"/>
    </xs:choice>
    <xs:element ref="xaip:checksum" minOccurs="0"/>
    <xs:element name="transformInfo" type="xaip:transformInfoType" minOccurs="0"/>
  </xs:sequence>
  <xs:attribute name="dataObjectID" type="xs:ID" use="required"/>
  <xs:attribute name="relatedObjects" type="xs:IDREFS"/>
</xs:complexType>
<xs:element name="checksum" type="xaip:checksumType"/>
<xs:complexType name="checksumType">
  <xs:sequence>
    <xs:element name="checksumAlgorithm" type="xs:anyURI"/>
    <xs:element name="checksum" type="xs:hexBinary"/>
  </xs:sequence>
</xs:complexType>
<xs:complexType name="transformInfoType">
  <xs:sequence>
    <xs:element name="transformObject" type="xaip:transformObjectType"
      maxOccurs="unbounded"/>
  </xs:sequence>
</xs:complexType>
<xs:complexType name="transformObjectType">
  <xs:sequence>
    <xs:element name="transformAlgorithm" type="xs:anyURI"/>
    <xs:element name="Parameters" type="xs:anyType" minOccurs="0"/>
  </xs:sequence>
  <xs:attribute name="transformObjectID" type="xs:ID" use="required"/>
  <xs:attribute name="order" type="xs:string"/>
</xs:complexType>
<!--=====-->
<!-- CredentialsSection -->
<!--=====-->
<xs:complexType name="credentialsSectionType">
  <xs:sequence>
    <xs:element ref="xaip:credential" maxOccurs="unbounded"/>
  </xs:sequence>
</xs:complexType>
<xs:element name="credential" type="xaip:credentialType"/>
<xs:complexType name="credentialType">
  <xs:choice>
    <xs:element ref="dss:SignatureObject"/>
    <xs:element name="certificateValues" type="xades:CertificateValuesType"/>
    <xs:element name="revocationValues" type="xades:RevocationValuesType"/>
    <xs:element ref="xaip:evidenceRecord"/>
    <xs:element ref="vr:VerificationReport"/>
    <xs:element name="other" type="xaip:extensionType"/>
  </xs:choice>
  <xs:attribute name="relatedObjects" type="xs:IDREFS"/>
  <xs:attribute name="credentialID" type="xs:ID" use="required"/>
</xs:complexType>
<xs:element name="evidenceRecord" type="xaip:EvidenceRecordType"/>
<xs:complexType name="EvidenceRecordType">

```

```

<xs:choice>
  <xs:element name="xmlEvidenceRecord" type="ers:EvidenceRecordType"/>
  <xs:element name="asn1EvidenceRecord" type="xs:base64Binary"/>
</xs:choice>
<xs:attribute name="AOID" type="xs:string"/>
<xs:attribute name="VersionID" type="xs:string"/>
</xs:complexType>
<!-- =====>
<!-- Delta-XAIP -->
<!-- =====>
<xs:element name="DXAIP" type="xaip:DXAIPType"/>
<xs:complexType name="DXAIPType">
  <xs:complexContent>
    <xs:extension base="xaip:XAIPType">
      <xs:sequence>
        <xs:element name="updateSection" type="xaip:updateSectionType"/>
      </xs:sequence>
    </xs:extension>
  </xs:complexContent>
</xs:complexType>
<xs:complexType name="updateSectionType">
  <xs:sequence>
    <xs:element name="prevVersion" type="xs:string"/>
    <xs:element name="placeholder" type="xaip:placeholderType" minOccurs="0"
      maxOccurs="unbounded"/>
  </xs:sequence>
</xs:complexType>
<xs:complexType name="placeholderType">
  <xs:attribute name="objectId" type="xs:ID" use="required"/>
</xs:complexType>
<xs:element name="idAssignmentList" type="xaip:idAssignmentListType"/>
<xs:complexType name="idAssignmentListType">
  <xs:sequence>
    <xs:element ref="xaip:idAssignmentPointer" maxOccurs="unbounded"/>
  </xs:sequence>
  <xs:attribute name="idAssignmentListID" type="xs:ID" use="required"/>
</xs:complexType>
<xs:element name="idAssignmentPointer" type="xaip:idAssignmentPointerType"/>
<xs:complexType name="idAssignmentPointerType">
  <xs:sequence>
    <xs:element ref="xaip:checksum"/>
  </xs:sequence>
  <xs:attribute name="objectRef" type="xs:IDREF" use="required"/>
</xs:complexType>
</xs:schema>

```